STUDY MATERIAL

Algorithm Design

FOR

COMPUTER SCIENCE & INFORMATION TECHNOLOGY

BY

SIDDHARTH SHUKLA

i-Gate Systematics Gate Preparation (COMPUTER SCIENCE & INFORMATION TECHNOLOGY)

Website: www.igate.guru

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

Copyright©,i-gate publication

First edition 2020

All right reserved

No part of this book or parts thereof may be reproduced, stored in a retrieval system or transmitted in any language or by any means, electronic, mechanical, photocopying, recording or otherwise without the prior written permission of the publisher.

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

ALGORITHMS (ADA)

Syllabus –

- Analysis
  - Asymptotic notation
  - Notions of space and time complexity
  - Worst and average case analysis
- Design
  - Greedy approach
  - Dynamic programming
  - Searching
  - Sorting
  - Asymptotic analysis (best, worst, average cases) of time and space
  - Divide-and-conquer
  - Tree and graph traversals
  - Connected components
  - Spanning trees
  - Shortest paths
  - Hashing
  - upper and lower bounds
  - Basic concepts of complexity classes – P, NP, NP-hard, NP-complete..

By Siddharth Shukla(B.E., M.E.)

Textbooks – "algorithm"

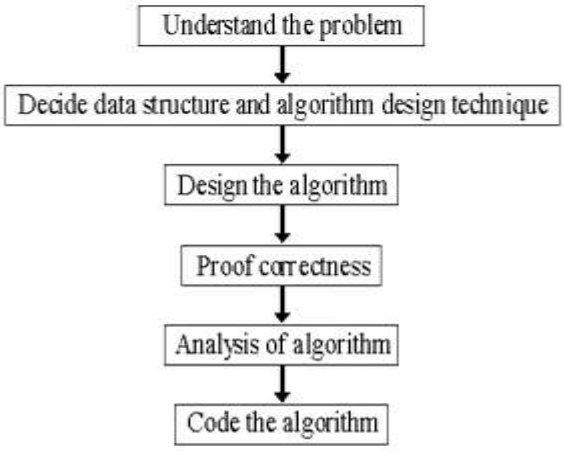
- "Introduction to algorithm" by Corman
- "Computer Algorithm" by Horowitz and Sahani
- "Algorithms" by Dr. M. N. Seetaramanth Tata McGraw Hill publication
- Introduction Algo:CLRS, Dasgupta, Vazirani

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

Algorithm –

- An algorithm is any well-defined computational procedure that takes some value, or set of values, as input and produces some value, or set of values, as output.
- An algorithm is a sequence of computational steps that transform the input into the output.
- An algorithm is an abstraction of a program to be executed on a physical machine (model of computation).
- **An algorithm must have following properties –**
  - **Finiteness** - Algorithm must complete after a finite number of instruction have been executed.
  - **Absence of ambiguity** - Each step must be clearly defined, having only one interpretation.
  - **Definition of sequence** - Each step must have a unique defined preceding and succeeding step. The first step (start step) and last step (halt step) must be clearly noted.
  - **Input/output** - Number and types of required inputs and results must be specified.
  - **Feasibility** - It must be possible to perform each instruction.
- **Characteristics of an algorithm –**
  - **Input** – Zero or more quantity are externally supplied.
  - **Output** – At-least one quantity is produce.
  - **Definiteness** – Each instruction should be clear and unambiguous.
  - **Effectiveness** – Each instruction should be very basic that it can be perform manually.
  - **Termination** – Algorithm should terminate after a finite number of steps.

Process and design of algorithm –



Analyzing algorithms – Analyzing an algorithm has come to mean predicting the resources (such as memory, communication bandwidth) that the algorithm requires.

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

Goal of analysis – Better utilization of memory, speed up the process (reduce the execution time) and reduce the development cost.

Running time of an algorithm – running time on a particular input is the numbers of basic operations that are perform.

OR

It is defined as number of steps executed by algorithm on input. It is depend upon size of inputs. The running time should be independent.

Example – **Insertion sort** –

INSERTION-SORT(A)		*cost*	*times*
1	**for** $j \leftarrow 2$ **to** *length*$[A]$	c_1	n
2	**do** $key \leftarrow A[j]$	c_2	$n-1$
3	▷ Insert $A[j]$ into the sorted sequence $A[1..j-1]$.	0	$n-1$
4	$i \leftarrow j-1$	c_4	$n-1$
5	**while** $i > 0$ and $A[i] > key$	c_5	$\sum_{j=2}^{n} t_j$
6	**do** $A[i+1] \leftarrow A[i]$	c_6	$\sum_{j=2}^{n}(t_j-1)$
7	$i \leftarrow i-1$	c_7	$\sum_{j=2}^{n}(t_j-1)$
8	$A[i+1] \leftarrow key$	c_8	$n-1$

In insertion sort 'for' loop depend on the size of input whereas 'while' loop depend on the type of inputs.

Computational time for insertion sort – Let the cost associated with each instruction 'i' be 'c_i' , therefore the cost of 1st , 2nd , to 8th instruction be c_1 , c_2 . . . . c_8.

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5\sum_{j=2}^{n} t_j + c_6\sum_{j=2}^{n}(t_j-1) + c_7\sum_{j=2}^{n}(t_j-1) + c_8(n-1)$$

where t_j = number of times the 'while' loop will executed.

Asymptotic –

Asymptotic analysis is based on two simplifying assumptions, which hold in most. But it is important to understand these assumptions and limitations of asymptotic analysis.

- **Large input sizes** : We are most interested in how the running time grows for large values of n.
- **Ignore constant factors** : The actual running time of the program depends on various constant factors in the implantation (coding tricks, optimizations in compilation, speed of the underlying hardware etc.). Therefore we will ignore constant factors.

Asymptotic notations – Asymptotic notations are mostly used in computer science to describe the running time, efficiency, performance, time complexity, and space complexity of an algorithm. It also shows the order of growth of function.

5 Asymptotic Notations:

By Siddharth S. Shukla (BE, ME, PhD*)

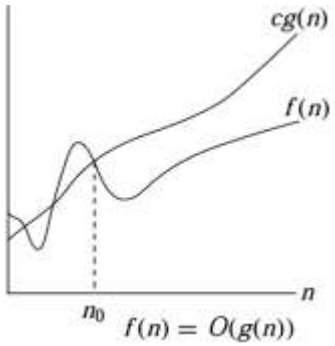
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352

No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

- O (Big - oh)
- Θ (Theta)
- Ω (Omega)
- o (Little - oh)
- ω (Little - omega)

O (Big-oh) Notation - The O - notation is used for asymptotically upper bounding a function. We would use O (big-oh) notation to represent a set of functions that upper bounds a particular function. For a given function *g(n)*, we denote by *O(g(n))* and pronounced "big-oh of *g* of *n*". The set of functions -

O(g(n)) = { *f (n)* : there exist positive constants *c* and n_0 such that $0 \le f(n) \le cg(n)$ for all $n \ge n_0$ }



$f(n) \le c.g(n)$ means $f(n)$ is slower than $c.g(n)$.

Let *f(n)* and *g(n)* are two functions and their exist a relationship –

$f(n) = O(g(n))$ if $\lim_{n\to\infty} \frac{f(n)}{g(n)} = c < \infty$

Where 'c' is constant.

Example 1 – Find the big – oh (O) notation for the following functions –

(a) $f(n) = 4n^3 + 2n + 3$

Solution – For $n \ge 3$, $4n^3 + 2n + 3 \le 4n^3 + 2n + n$

$4n^3 + 2n + 3 \le 4n^3 + 3n$

For $n^3 \le 3n$, $4n^3 + 3n \le 4n^3 + n^3$

$4n^3 + 3n \le 5n^3$

$f(n) \le c.g(n)$

thus , $c = 5$ and $n_0 = 3$

hence, $f(n) = O(n^3)$

(b) $f(n) = 10n^2 + 7$

Solution – For $n \ge 7$, $10n^2 + 7 \le 10n^2 + n$

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

$10n^2 + 7 \leq 10n^2 + n$

For $n^2 \geq n$, $10n^2 + n \leq 10n^2 + n^2$

$10n^2 + n \leq 11n^2$

$f(n) \leq c.g(n)$

thus , $c = 11$ and $n_0 = 7$

hence, $f(n) = O(n^2)$

Example 2 – Solve the following –

(a) Is $(n+1) = O(n^2)$?

Solution – For big – oh notation, $f(n) = O(g(n))$

means, $f(n) \leq c.g(n)$

Function is valid if -

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = c < \infty$$

Let $f(n) = n+1$

$g(n) = n^2$

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \frac{n+1}{n^2} = \left(\frac{n}{n^2} + \frac{1}{n^2}\right) = \left(\frac{1}{n} + \frac{1}{n^2}\right) = 0 < \infty$$

hence the given function is valid.

(b) Is $n^2 + 5n + 1 = O(n^2)$?

Solution – For big – oh notation, $f(n) = O(g(n))$

means, $f(n) \leq c.g(n)$

Function is valid if -

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = c < \infty$$

Let $f(n) = n^2 + 5n + 1$

$g(n) = n^2$

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \frac{n^2 + 5n + 1}{n^2} = \left(\frac{n^2}{n^2} + \frac{5n}{n^2} + \frac{1}{n^2}\right) = \left(1 + \frac{5}{n} + \frac{1}{n^2}\right) = 1 < \infty$$

hence the given function is valid.

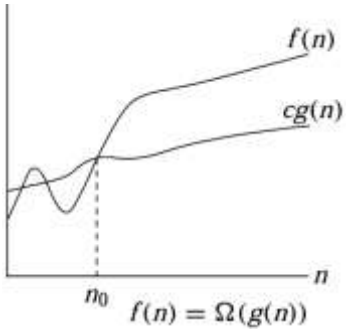
Ω (Omega) Notation – The *Ω*-notation is used for asymptotically lower bounding a function. We would use *Ω* (omega) notation to represent a set of functions that lower bounds a particular function. For a given function *g(n)*, we denote by - *Ω (g(n))* (pronounced "big-omega of *g* of *n*") . The set of functions -

By Siddharth S. Shukla (BE, ME, PhD*)

i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352

No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

$\Omega(g(n)) = \{ f(n)$: there exist positive constants c and n_0 such that $0 \le cg(n) \le f(n)$ for all $n \ge n_0\}$.



Let *f(n)* and *g(n)* are two functions and their exist a relationship –

f(n) = O(g(n)) if $\lim_{n\to\infty} \frac{f(n)}{g(n)} = c > 0$

Where 'c' is constant.

Example 1 - Find the big - omega (Ω) notation for the following functions –

(a) $f(n) = 5n^3 + n^2 + 3n + 2$

Solution – For all n, $5n^3 < 5n^3 + n^2 + 3n + 2$

$f(n) > c.g(n)$

thus , $c = 5$

hence, $f(n) = \Omega(n^3)$

(b) $f(n) = 3*2^n + 4n^2 + 5n + 2$

Solution – For all $n \ge n_0$, $3*2^n \le 3*2^n + 4n^2 + 5n + 2$

$f(n) \ge c.g(n)$

thus , $c = 3$ and $g(n) = 2^n$

hence, $f(n) = \Omega(2^n)$

Example 2 – Solve the following –

(a) Is $(10n^2 + 5n + 7) = \Omega(n)$?

Solution – For big – omega (Ω) notation, $f(n) = \Omega(g(n))$

means, $f(n) \ge c.g(n)$

Function is valid if -

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = c > 0$$

Let $f(n) = 10n^2 + 5n + 7$

$g(n) = n$

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

$$\lim_{n\to\infty}\frac{f(n)}{g(n)} = \frac{10n^2 + 5n + 7}{n} = \left(\frac{10n^2}{n} + \frac{5n}{n} + \frac{7}{n}\right)$$
$$= \left(\frac{10n}{1} + \frac{5}{1} + \frac{7}{n}\right) = \infty > 0$$

hence the given function is valid.

(b) Is $n^2 + n + 1 = \Omega(n)$?

Solution – For big – omega notation, $f(n) = \Omega(g(n))$

means, $f(n) \geq c.g(n)$

Function is valid if -

$$\lim_{n\to\infty}\frac{f(n)}{g(n)} = c > 0$$

Let $f(n) = n^2 + n + 1$

$g(n) = n$

$$\lim_{n\to\infty}\frac{f(n)}{g(n)} = \frac{n^2 + n + 1}{n} = \left(\frac{n^2}{n} + \frac{n}{n} + \frac{1}{n}\right) = \left(n + 1 + \frac{1}{n}\right) = \infty > 0$$

hence the given function is valid.

Θ **(Theta) Notation** – The Θ-notation is used for asymptotically bounding a function from both above and below. We would use Θ (Theta) notations to represent a set of functions that bounds a particular function from above and below. For a given function *g(n)*, we denote by Θ*(g(n))* the *set of functions* -

Θ*(g(n))* = { *f (n)* : there exist positive constants *c*1, *c*2, and n_0 such that $0 \leq c1g(n) \leq f(n) \leq c2g(n)$ for all $n \geq n_0$} .

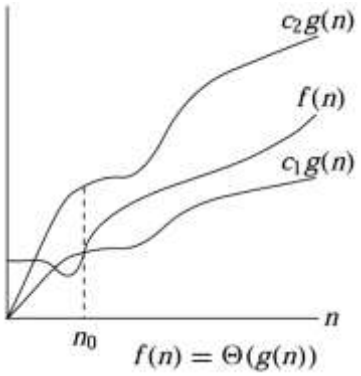
A function *f (n)* belongs to the set Θ*(g(n))* if there exist positive constants *c*1 and *c*2 such that it can be "sandwiched" between *c*1*g(n)* and *c*2*g(n)*, for sufficiently large *n*. In other words, for all $n \geq n_0$, the function *f (n)* is equal to *g(n)* to within a constant factor. We say that *g(n)* is an *asymptotically tight bound* for *f (n)*.

Let *f(n)* and *g(n)* are two functions and their exist a relationship –

f(n) = Θ*(g(n))* if $\lim_{n\to\infty}\frac{f(n)}{g(n)} = c$, where $0 < c < \infty$

Where 'c' is constant.

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.



Theorem – For any two functions *f (n)* and *g(n)*,
we have *f (n)* = Θ *(g(n))*
if and only if *f (n)* = *O(g(n))* and *f (n)* = *Ω (g(n))*.

Example 1 - Find the Theta (Θ) notation for the following functions –

(a) $f(n) = n^2 + n + 1$

Solution – $c_1.g(n) \le f(n) \le c_2.g(n)$

For all n, $n^2 \le n^2 + n + 1 \le n^2 + n + n$

$n^2 \le n^2 + n + 1 \le n^2 + 2n$

$n^2 \le n^2 + n + 1 \le n^2 + 2n^2$

$n^2 \le n^2 + n + 1 \le 3n^2$

thus , $c_1 = 1$, $c_2 = 3$ and $g(n) = n^2$

hence, $f(n) = \Theta(g(n))$

$f(n) = \Theta(n^2)$

(b) $f(n) = 3*2^n + 4n^2 + 5n + 2$

Solution – Finding theta – notation -

For all $n \ge n_0$, $3*2^n \le 3*2^n + 4n^2 + 5n + 2$

$f(n) \ge c.g(n)$

thus , $c = 3$ and $g(n) = 2^n$

hence, $f(n) = \Omega(2^n)$

Now find big – oh notation –

For all $n \ge 2$, $3*2^n + 4n^2 + 5n + 2 \le 3*2^n + 4n^2 + 5n + n$

$3*2^n + 4n^2 + 5n + 2 \le 3*2^n + 4n^2 + 6n$

For all $n^2 \ge 6n$, $3*2^n + 4n^2 + 5n + 2 \le 3*2^n + 4n^2 + n^2$

$3*2^n + 4n^2 + 5n + 2 \le 3*2^n + 5n^2$

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

$3*2^n + 4n^2 + 5n + 2 \leq 8 \cdot 2^n$

$f(n) = O(g(n))$

$f(n) = O(2^n)$

hence, $f(n) = \Theta(2^n)$

Example 2 – Solve the following –

(a) Is $(21n^3 + 2n + 1) = \Theta(n^{10})$?

Solution – For theta (Θ) notation, $f(n) = \Theta(g(n))$

Function is valid if -

$\lim_{n\to\infty} \frac{f(n)}{g(n)} = c$ where $0 < c < \infty$

Let $f(n) = 21n^3 + 2n + 1$

$g(n) = n^{10}$

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \frac{21n^3 + 2n + 1}{n^{10}} = \left(\frac{21n^3}{n^{10}} + \frac{2n}{n^{10}} + \frac{1}{n^{10}}\right) = 0$$

$$\neq 0 < c < \infty$$

hence the given function is not valid.

(b) Is $2n + 1 = \Theta(n)$?

Solution – For theta notation, $f(n) = \Theta(g(n))$

Function is valid if -

$\lim_{n\to\infty} \frac{f(n)}{g(n)} = c$ where $0 < c < \infty$

Let $f(n) = n + 1$

$g(n) = n$

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \frac{n + 1}{n} = \left(\frac{n}{n} + \frac{1}{n}\right) = \left(1 + \frac{1}{n}\right) = 1 + 0 = 1$$

$= \quad 0 < c < \infty$

hence the given function is valid.

o (Little – oh) Notation – The asymptotic upper bound provided by *O*-notation may or may not be asymptotically tight. The bound $2n^2 = O(n^2)$ is asymptotically tight, but the bound $2n = O(n^2)$ is not. We use *o*-notation to denote an upper bound that is not asymptotically tight. We formally define $o(g(n))$ ("little-oh of g of n") as the set of function –

$o(g(n)) = \{ f(n)$: for any positive constant $c > 0$,

there exists a constant $n_0 > 0$ such that $0 \leq f(n) < c.g(n)$ for all $n \geq n_0\}$.

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

Let *f(n)* and *g(n)* are two functions and their exist a relationship –

f(n) = o(g(n)) if $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$

The definitions of *O*-notation and *o*-notation are similar.

The difference is that in $f(n) = O(g(n))$, the bound $0 \le f(n) \le cg(n)$ holds for *some* constant $c > 0$, but in $f(n) = o(g(n))$, the bound $0 \le f(n) < cg(n)$ holds for *all* constants $c > 0$.

Example 1 - Solve the following –

(a) Is $(21n^3 + 10\,n^2 + 5n + 1) = o(n^5)$?

Solution – For little –oh (o) notation, f(n) = o (g(n))

Function is valid if -

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$$

Let $f(n) = 21n^3 + 10\,n^2 + 5n + 1$

$g(n) = n^5$

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \frac{21n^3 + 10\,n^2 + 5n + 1}{n^5} = \left(\frac{21n^3}{n^5} + \frac{10n^2}{n^5} + \frac{5n}{n^5} + \frac{1}{n^5}\right) = 0$$

hence the given function is valid.

(b) Is $2^{2n+1} = o(2^n)$?

Solution – For little - oh notation, f(n) = o (g(n))

Function is valid if -

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$$

Let $f(n) = 2^{2n+1}$

$g(n) = 2^n$

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \frac{2^{2n+1}}{2^n} = \left(\frac{2^{2n+1-n}}{1}\right) = \left(\frac{2^{n+1}}{1}\right) = 2^n.2 = \infty$$

hence the given function is not valid.

ω (Little – omega) Notation – ω -notation is to Ω-notation as *o*-notation is to *O*-notation. We use ω -notation to denote a lower bound that is not asymptotically tight. we define *ω(g(n))* ("little-omega of *g* of *n*") as the set of function –

$\omega(g(n)) = \{ f(n)$: for any positive constant $c > 0$,

there exists a constant $n_0 > 0$ such that $0 \le c.g(n) < f(n)$ for all $n \ge n_0\}$.

By Siddharth S. Shukla (BE, ME, PhD*)

i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352

No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

Let *f(n)* and *g(n)* are two functions and their exist a relationship –
f(n) = ω (g(n))

if $\lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty$

OR

if $\lim_{n\to\infty} \frac{g(n)}{f(n)} = 0$

Example 1 - Solve the following –

(a) Is (2n +1) = ω (1) ?

Solution – For little –omega (ω) notation, f(n) = ω(g(n))

Function is valid if -

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty$$

Let f(n) = 2n +1

g(n) = 1

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \frac{2n\ +1}{1} = \ (2n+1)\ = \infty + 1 = \ \infty$$

hence the given function is valid.

(b) Is (2n +1) = ω (n) ?

Solution – For little –omega (ω) notation, f(n) = ω(g(n))

Function is valid if -

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty$$

Let f(n) = 2n +1

g(n) = n

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \frac{2n\ +1}{n} = \ \left(2+\frac{1}{n}\right)\ = 2 + 0 = \ 0$$

hence the given function is not valid.

NOTE - The asymptotic comparison of two functions *f* and *g* and the comparison of two real numbers *a* and *b* -

a = Θ (b) → a = b

a = O(b) → a ≤ b

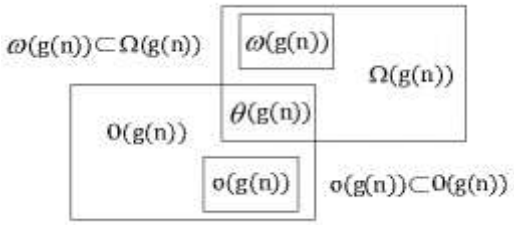
By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

a = Ω (b) → a ≥ b
a = o(b) → a < b
a = ω (b) → a > b

Functions - $1 < logn < \sqrt{n} < n < n.\log n < n^2 < n^2.\log n < n^3 < 2^n < n! < n^n$

$Constant < \log function < linear\ function < Quadratic\ function < Cubic\ function < Exponential$

Relationship between all notations –



Θ is a subset of Ω and of O. and

- $\omega(g(n)) \cup \theta(g(n))$ *is subset of* $\Omega(g(n))$
- $o(g(n)) \cup \theta(g(n))$ *is subset of* $O(g(n))$
- $\theta(g(n))$ *is the intersection of* $\Omega(g(n))$ *and* $O(g(n))$

Comparison of functions –

Asymptotic notation properties -

- Reflexivity
- Transitivity
- Symmetry
- Transpose symmetry

Reflexivity –

$f(n) = \Theta(f(n))$,
$f(n) = O(f(n))$,
$f(n) = \Omega(f(n))$.

Transitivity –

$f(n) = \Theta(g(n))$ and $g(n) = \Theta(h(n))$ imply $f(n) = \Theta(h(n))$,
$f(n) = O(g(n))$ and $g(n) = O(h(n))$ imply $f(n) = O(h(n))$,
$f(n) = \Omega(g(n))$ and $g(n) = \Omega(h(n))$ imply $f(n) = \Omega(h(n))$,
$f(n) = o(g(n))$ and $g(n) = o(h(n))$ imply $f(n) = o(h(n))$,
$f(n) = \omega(g(n))$ and $g(n) = \omega(h(n))$ imply $f(n) = \omega(h(n))$.

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

Symmetry –

$f(n) = \Theta(g(n))$ if and only if $g(n) = \Theta(f(n))$.

Transpose symmetry –

$f(n) = O(g(n))$ if and only if $g(n) = \Omega(f(n))$.

$f(n) = o(g(n))$ if and only if $g(n) = \omega(f(n))$.

----------◄►----------

Recurrences –

Recurrences - A recurrence is an equation or inequality that describes a function in terms of its value on smaller inputs. Three methods for solving recurrences –

- Substitution method
- Recursion-tree method
- Master method

The Substitution method - Here we guess a bound and then use mathematical induction to prove our guess correct. The substitution method for solving recurrences involves two steps:

1. Guess the form of the solution.
2. Use mathematical induction to find the constants and show that the solution works.

The substitution method can be used to establish either upper or lower bounds on a recurrence.

Example - let us determine an upper bound on the recurrence -

$$T(n) = 2T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + n$$

Let we guess that the solution is T (n) = O(n lg n).

To prove that $T(n) \leq c\, n \lg n$

Assuming that bound holds for $\left\lfloor \frac{n}{2} \right\rfloor$, that is, that $T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) \leq c \left\lfloor \frac{n}{2} \right\rfloor \lg\left(\left\lfloor \frac{n}{2} \right\rfloor\right)$.

$$\begin{aligned} T(n) &\leq 2\left(c \left\lfloor \frac{n}{2} \right\rfloor \lg\left(\left\lfloor \frac{n}{2} \right\rfloor\right)\right) + n \\ &\leq cn \lg(n/2) + n \\ &= cn \lg n - cn \lg 2 + n \\ &= cn \lg n - cn + n \\ &\leq cn \lg n \end{aligned}$$

Thus our guess is correct.

Checking for the boundary conditions – let us assume T (1) = 1

Then for n = 1, the bound –

T (n) ≤ cn lg n

By Siddharth S. Shukla (BE, ME, PhD*)

i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352

No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

$T(1) \leq c1 \lg 1 = 0$
which is at false with $T(1) = 1$.

The iteration method – In iteration method the basic idea is to expand the recurrence and express it as a summation of terms dependent only on 'n' and the initial conditions.

Example – Solve the recurrence relation by iteration : $T(n) = T(n-1) + n^4$

Solution – $T(n) = T(n-1) + n^4$
$T(n) = [\ T(n-2) + (n-1)^4\ ] + n^4$
$T(n) = T(n-2) + (n-1)^4 + n^4$
$T(n) = T(n-3) + (n-2)^4 + (n-1)^4 + n^4$
……
$T(n) = n^4 + (n-1)^4 + (n-2)^4 + \ldots\ldots + 2^4 + 1^4 + T(0)$
$T(n) = \sum_{i=1}^{n} i^4 + T(0)$
$T(n) = \Theta(n^4)$

Example – Solve the recurrence relation by iteration : $T(n) = T(n-1) + 1$ and $T(1) = \Theta(1)$.

Solution – $T(n) = T(n-1) + 1$
$T(n) = [\ T(n-2) + 1\ ] + 1$
$T(n) = T(n-2) + 1 + 1$
$T(n) = [\ T(n-3) + 1\ ] + 1 + 1 = T(n-3) + 3$
$T(n) = T(n-4) + 4$
$T(n) = T(n-5) + 5$
…
…
$T(n) = T(n-k) + k$ where $k = n-1$
$T(n-k) = T(n-(n-1)) = T(1) = \Theta(1)$
$T(n) = \Theta(1) + k$
$T(n) = \Theta(1) + (n-1)$
$T(n) = 1 + n - 1$
$T(n) = n$
$T(n) = \Theta(n)$

The recursion-tree method converts the recurrence into a tree whose nodes represent the costs incurred at various levels of the recursion; we use techniques for bounding summations to solve the recurrence.

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

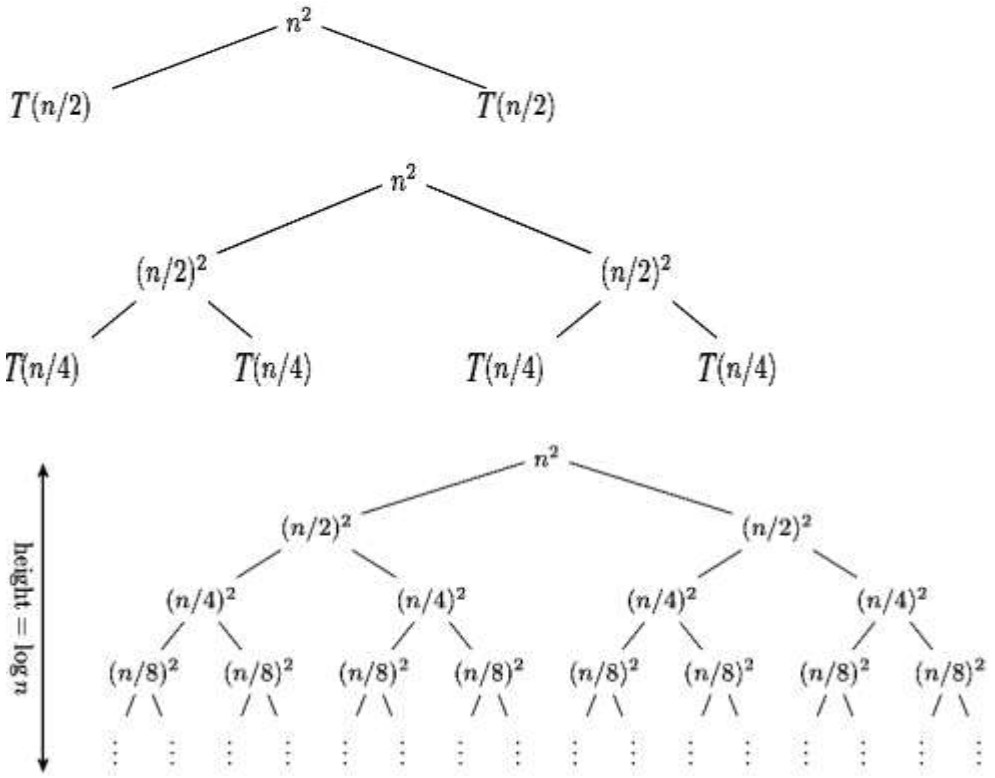
OR

Recursion tree method is a pictorial representation of an iteration method, which is in the form of a tree, where at each level nodes are expanded. In general, we consider second term in recurrence as root. It is useful when divide and conquer algorithm is used.

Example – Consider $T(n) = 2T\left(\frac{n}{2}\right) + n^2$

Obtain asymptotic notation using recursion tree method.

Solution – The recursion tree for the given recurrence is –



$T(n) = n^2 + \frac{n^2}{2} + \frac{n^2}{4} + \ldots\ldots \log n \ times$

$T(n) \leq n^2 \sum_{i=0}^{\infty} \left(\frac{1}{2^i}\right)$

$T(n) \leq n^2 \left(\frac{1}{1-1/2}\right)$

$T(n) \leq 2n^2$

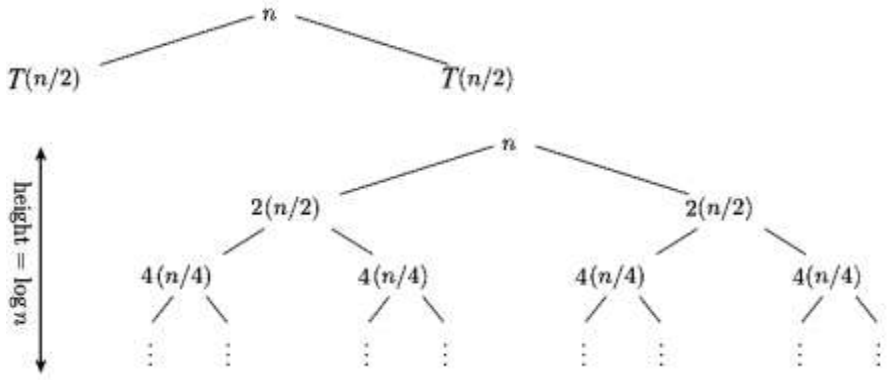
By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

$T(n) = \Theta(n^2)$

Example – Consider $T(n) = 4T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + n$

Obtain asymptotic notation using recursion tree method.

Solution – The recursion tree for the given recurrence is –



We have $T(n) = n + 2n + 4n + \ldots\ldots \log_2 n \ times$

$T(n) = n(1 + 2 + 3 + 4 + \ldots\ldots \log_2 n \ times)$

$T(n) = n \frac{(2^{\log 2\, n} - 1)}{(2 - 1)}$

$T(n) = \frac{n(n-1)}{1} = n^2 - n = \Theta(n^2)$

$T(n) = \Theta(n^2)$

The master method provides bounds for recurrences of the form T (n) = aT (n/b) + f (n),

Where, a ≥ 1, b > 1, and f (n) is a given function

Let a ≥ 1 and b > 1 be constants, let f (n) be a function, and let T (n) be defined on the non-negative integers by the recurrence.

$T(n) = aT(n/b) + f(n)$

where we interpret n/b to mean either $\left\lfloor \frac{n}{b} \right\rfloor$ *or* $\left\lceil \frac{n}{b} \right\rceil$. Then T (n) can be bounded asymptotically as follows –

1. If $f(n) = O(n^{\log_b a - \varepsilon})$ for some constant $\varepsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \lg n)$.

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

3. If $f\ (n)\ =\ \Omega(n^{\log_b a+\varepsilon})$ for some constant $\varepsilon > 0$, and if $a\ f\ \left(\frac{n}{b}\right) \leq\ c\ f\ (n)$ (called – regularity condition) for some constant c < 1 and all sufficiently large n, then $T\ (n)\ =\ \Theta(\ f\ (n))$.

Example – Solve the recurrence using Master method - Consider $T(n) =\ 9T\left(\frac{n}{3}\right) +\ n$.

Solution – Given recurrence $T(n) =\ 9T\left(\frac{n}{3}\right) +\ n$

Compare this with $T\ (n)\ =\ aT\ \left(\frac{n}{b}\right)\ +\ f\ (n)$

we get : a = 9 , b = 3 , f(n) = n

Now : $n^{\log_b a}\ =\ n^{\log_3 9}\ =\ n^2\ =\ \Theta\ (n^2)$

$f(n) = n\ =\ n^{\log_3 9 -1}\ =\ n^{2-1}$

Case 1 satisfy , hence the solution is : $\Theta\left(n^{\log_3 9}\right) =\ \Theta\ (n^2)$

Example – Solve the recurrence using Master method - $T(n) =\ T\left(\frac{2n}{3}\right) +\ 1$.

Solution – Given recurrence : $T(n) =\ T\left(\frac{2n}{3}\right) +\ 1$

Compare this with $T\ (n)\ =\ aT\ \left(\frac{n}{b}\right)\ +\ f\ (n)$

we get : a = 1 , b = 3/2 , f(n) = 1

Now : $n^{\log_b a}\ =\ n^{\log_{3/2} 1}\ =\ n^0\ =\ 1$

$f(n) =\ n^{\log_b a}$

applying case -2

$T(n) = \Theta\left(n^{\log_b a}\ \lg n\right) =\ \Theta\ (\lg n)$.

$T(n) =\ \Theta\ (\lg n)$

Searching –

Searching -

- Linear search
- Binary search

Linear search –

Linear search / sequential search is a method for finding a particular value in, list, that consists of checking every one of its elements, one at a time and in sequence, until the desired one is found. Linear search is a special case of brute force search. Its worst case cost is proportional to the number of elements in the list.

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

Implementation –

```
boolean linear search (int [ ] arr, int target)
{
        int i = 0;
        While (i < arr.length)
        {
                if(arr [ i ] = = target )
                {
                        return true;
                }
        }
        ++i;
        }
        return false;
}
```

Example – Consider the array –

10	7	1	3	-4	2	20

Search for 3 –

10	7	1	3	-4	2	20

Element not found, Move to next element –

10	**7**	1	3	-4	2	20

Element not found, Move to next element –

10	7	**1**	3	-4	2	20

Element not found, Move to next element –

10	7	1	**3**	-4	2	20

Element found; stop the search.

Binary search -

A binary search algorithm is a technique for finding a particular value in a linear array, by ruling out half of the data at each step, a binary search finds the median, makes comparison, to determine whether the desired value comes before or after it, and then searches the remaining half in the same manner. A binary search is an example of divide and conquer algorithm.

Implementation –

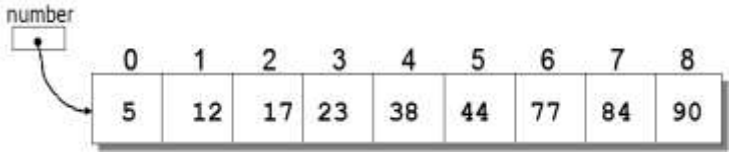
```
function binarysearch (a, value, left, right)
```


By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

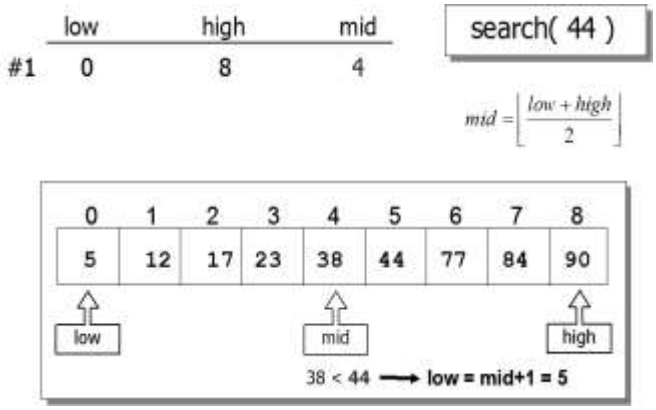
```
if right < left
        return not found
    mid: = floor ((right - left)/2) + left
    if a [mid] = value
        return mid
    if value < a[mid]
        return binarysearch(a, value, left, mid - 1)
    else
        return binarysearch (a, value, mid +1, right)
```

Example – The Binary Search algorithm depends on the array being already sorted

Find element **44** -

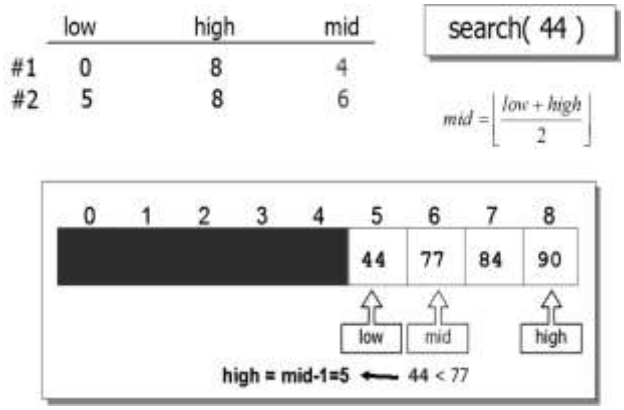


We KNOW that the value we are searching for MUST be in the UPPER HALF of the array, since it is larger than the midpoint element value! So in one comparison, we have discarded the lower half of the array as elements that we need to search! But we still haven't found the number, so let's continue with the next figure.

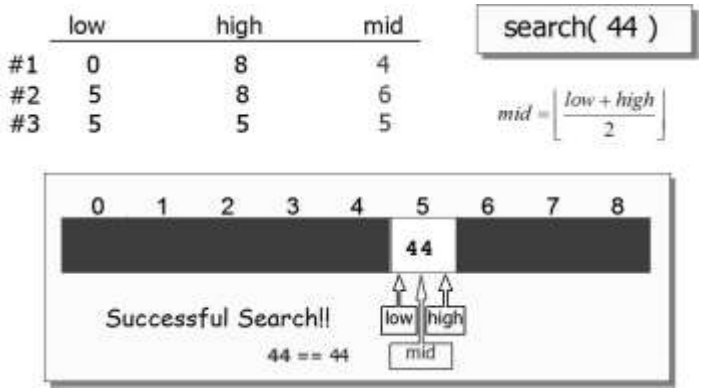


So now we take the 2nd pass in the algorithm. We know that we just need to search the upper half of the array. The mid pointed to above is NOT the value that we are looking for. So, we now reset our "low" pointer to point to the index of the element that is one higher than our previous midpoint. It is lower (44 is less than 77). So now we are going to reset our pointers and does a third pass.

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.



In the third pass, we reset the HIGH pointer since our search value was lower than the value of the element at the midpoint. In the figure below, we can see that we reset the high pointer to point to one less than the previous mid pointer (since we already knew that the mid pointer did NOT point to our value). We leave the low pointer alone.



We have successfully searched for and found our value in three comparison steps.

Sorting –

Sorting –

- Selection sort
- Bubble sort
- Insertion sort

Selection sort –

Selection sort is a sorting algorithm, specifically an in - place comparison sort. It has $O(n^2)$ complexity making it inefficient on large lists.

Implementation :-

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

1. Find the minimum Value in the list
2. Swap it with the value in the first position.
3. Repeat the steps above for the remainder of the list (starting at the second position and advancing each time)

Algorithm –

```
for i ← 1 to n-1 do
        min j ← i;
        min x ← A[i]
        for j ← i + 1 to n do
                If A[j] < min x then
                        min j ← j
                        min x ← A[j]
        A[min j] ← A [i]
        A[i] ← min x
```

ANALYSIS - Selection sort is not difficult to analyze compared to other sorting algorithms since none of the loops depend on the data in the array selecting the lowest element requires scanning all n elements (this takes n - 1 comparisons) and then swapping it into the first position. Finding the next lowest element requires scanning the remaining n - 1 elements and so on, for $(n - 1) + (n - 2) + \ldots\ldots + 2 + 1 = n(n - 1)/2 \in \theta(n^2)$ comparisons.
Each of these scans requires one swap for n - 1 elements (the final element is already in place)

Example - The selection sort algorithm using numbers.

Original array:

	6	3	5	4	9	2	7	
1st pass -	2	3	5	4	9	6	7	(2 and 6 were swapped)
2nd pass -	2	3	4	5	9	6	7	(4 and 5 were swapped)
3rd pass -	2	3	4	5	6	9	7	(6 and 9 were swapped)
4th pass -	2	3	4	5	6	7	9	(7 and 9 were swapped)
5thpass -	2	3	4	5	6	7	9	(no swap)
6th pass -	2	3	4	5	6	7	9	(no swap)

Note : -
There are 7 keys in the list and thus 6 passes were required. However, only 4 swaps took place.

Bubble sort –

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

Bubble sort is a simple sorting algorithm that works by repeatedly stepping through the list to be sorted, comparing each pair of adjacent items, and swapping them if they are in the wrong order. The pass through the list is repeated until no swaps are needed. This indicates that the list is sorted. The algorithm gets its name from the way smaller elements 'bubble' to the top of the list.
Number of comparison made - $1 + 2 + 3 + \ldots + (n - 1) = n(n - 1)/2 = O(n^2)$

Implementation -

```
for i ← 1 to length [A] do
  for j ← length [A] downto i +1 do
    If A[A] < A[j-1] then
      Exchange A[j] ↔ A[j-1]
```

Example - The array of numbers "5 1 4 2 9", and sort the array from lowest number to greatest number using bubble sort algorithm. In each step, elements written in **bold** are being compared.

First Pass: Considering length n

(**5 1** 4 2 9) ⟶ (**1 5** 4 2 9), Here,
algorithm compares the first two elements, and swaps them.
(1 **5 4** 2 9) ⟶ (1 **4 5** 2 9), Swap since 5 > 4
(1 4 **5 2** 9) ⟶ (1 4 **2 5** 9), Swap since 5 > 2
(1 4 2 **5 9**) ⟶ (1 4 2 **5 9**), Now,
since these elements are already in order (8 > 5), algorithm does not swap them.

Second Pass: Considering length n - 1

(**1 4** 2 5 9) ⟶ (**1 4** 2 5 9)
(1 **4 2** 5 9) ⟶ (1 **2 4** 5 9), Swap since 4 > 2
(1 2 **4 5** 9) ⟶ (1 2 **4 5** 9)

Third Pass: Considering length n -2

(**1 2** 4 5 9) ⟶ (**1 2** 4 5 9)
(1 **2 4** 5 9) ⟶ (1 **2 4** 5 9)

Fourth Pass: Considering length n -3

(**1 2** 4 5 9) ⟶ (**1 2** 4 5 9)

Sorted array – 1 2 4 5 9

Insertion sort -

Insertion sort is a comparison sort in which the sorted array is built one entry at a time. It is much less efficient on large lists than more advanced algorithms such a Quick sort, heap sort, (or) merge sort.

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

Implementation –

```
INSERTION_SORT (A)
1.  FOR j ← 2 TO length[A]
2.      DO  key ← A[j]
3.          {Put A[j] into the sorted sequence A[1 . . j − 1]}
4.          i ← j − 1
5.          WHILE i > 0 and A[i] > key
6.                  DO A[i +1] ← A[i]
7.                      i ← i − 1
8.          A[i + 1] ← key
```

Coding –

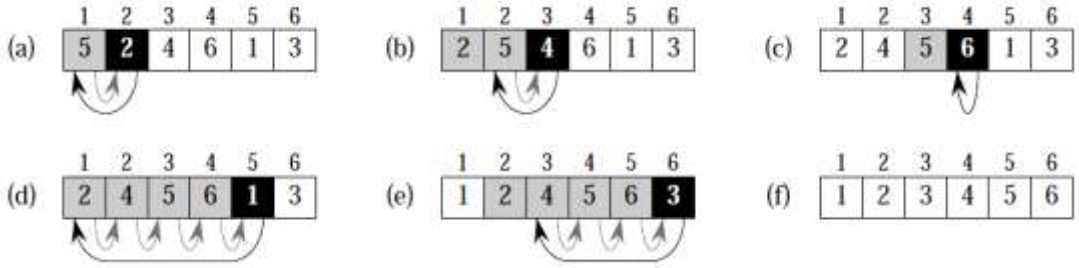
```
void insertionSort(int numbers[ ], int array_size)
{
 int i, j, index;

 for (i = 1; i < array_size; i++)
 {
     index = numbers[i];
     j = i;
     while ((j > 0) && (numbers[j − 1] > index))
     {
         numbers[j] = numbers[j − 1];
         j = j − 1;
     }
     numbers[j] = index;
 }
}
```

- This algorithm does not require extra memory.
- For Insertion sort we say the worst-case running time is $\theta(n^2)$, and the best-case running time is $\theta(n)$.
- Insertion sort use no extra memory it sort in place.
- The time of Insertion sort is depends on the original order of a input. It takes a time in $\Omega(n^2)$ in the worst-case, despite the fact that a time in order of n is sufficient to solve large instances in which the items are already sorted.

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

Example - The operation of INSERTION-SORT on the array A = 5, 2, 4, 6, 1, 3



(a)–(e) The iterations of the for loop of lines 1–8. In each iteration, the black rectangle holds the key taken from A[j], which is compared with the values in shaded rectangles to its left in the test of line 5. Shaded arrows show array values moved one position to the right in line 6, and black arrows indicate where the key is moved to in line 8.

(f) The final sorted array.

Divide and conquer –

Divide and conquer is a top down technique for designing algorithms that consists of dividing the problem into smaller sub problems hoping that the solutions of the sub problems are easier to find and then composing the partial solutions into the solution of the original problem.

- **Divide** the problem into a number of sub-problems
  - Similar sub-problems of smaller size
- **Conquer** the sub-problems
  - Solve the sub-problems recursively
  - Sub-problem size small enough ⇒ solve the problems in straightforward manner
- **Combine** the solutions of the sub-problems
  - Obtain the solution for the original problem

The **divide-and-conquer** strategy solves a problem by:

1. Breaking it into sub-problems that are themselves smaller instances of the same type of problem.
2. Recursively solving these sub-problems.
3. Appropriately combining their answers.

Examples -

- Sorting: Merge sort and quick sort
- Binary tree traversals
- Binary Search

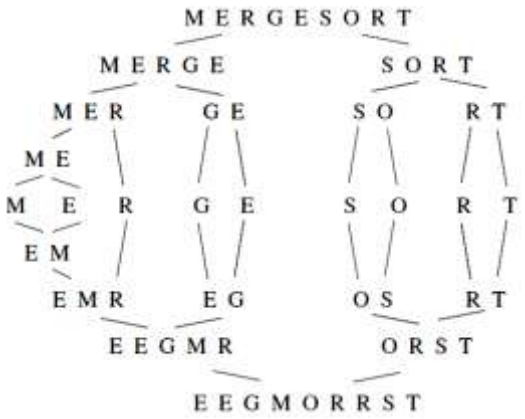
By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

- Multiplication of large integers
- Matrix Multiplication : Strassen's algorithm
- Closest pair and Convex hull algorithm

Merge sort –

Merge sort is a sorting algorithm for rearranging lists (or any other data structure that can only be accessed sequentially, e.g., file streams) into a specified order.

Merge sort in action –



To sort $A[p .. r]$:

1. **Divide Step -** If a given array A has zero or one element, simply return; it is already sorted. Otherwise, split $A[p .. r]$ into two subarrays $A[p .. q]$ and $A[q + 1 .. r]$, each containing about half of the elements of $A[p .. r]$. That is, q is the halfway point of $A[p .. r]$.

2. **Conquer Step -** Conquer by recursively sorting the two subarrays $A[p .. q]$ and $A[q + 1 .. r]$.

3. **Combine Step -** Combine the elements back in $A[p .. r]$ by merging the two sorted subarrays $A[p .. q]$ and $A[q + 1 .. r]$ into a sorted sequence. To accomplish this step, we will define a procedure MERGE (A, p, q, r).

Algorithm: Merge Sort - To sort the entire sequence A[1 .. n], make the initial call to the procedure MERGE-SORT (A, 1, n).

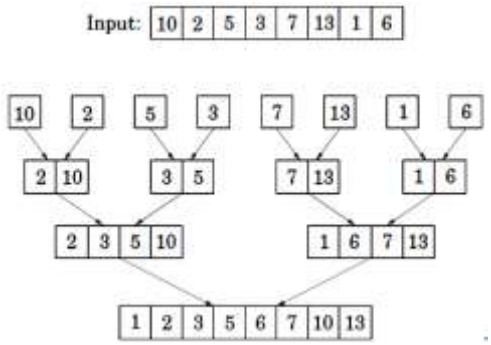
MERGE-SORT (A, p, r)

1. IF $p < r$ // Check for base case
2. THEN $q = \text{FLOOR}[(p + r)/2]$ // Divide step

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

3. MERGE (A, p, q) // Conquer step.
4. MERGE (A, $q + 1$, r) // Conquer step.
5. MERGE (A, p, q, r) // Conquer step.

Example: Bottom-up view of the above procedure for n = 8.



The **pseudocode** of the MERGE procedure is as follow:

MERGE (A, p, q, r)

1. $n_1 \leftarrow q - p + 1$
2. $n_2 \leftarrow r - q$
3. Create arrays L[1 . . $n_1 + 1$] and R[1 . . $n_2 + 1$]
4. **FOR** $i \leftarrow 1$ **TO** n_1
5. **DO** L[i] $\leftarrow$ A[$p + i - 1$]
6. **FOR** $j \leftarrow 1$ **TO** n_2
7. **DO** R[j] $\leftarrow$ A[$q + j$]
8. L[$n_1 + 1$] $\leftarrow \infty$
9. R[$n_2 + 1$] $\leftarrow \infty$
10. $i \leftarrow 1$
11. $j \leftarrow 1$
12. **FOR** $k \leftarrow p$ **TO** r
13. **DO IF** L[i] $\leq$ R[j]
14. **THEN** A[k] $\leftarrow$ L[i]
15. $i \leftarrow i + 1$
16. **ELSE** A[k] $\leftarrow$ R[j]
17. $j \leftarrow j + 1$

Example – Using merge – sort algorithm to sort the following elements :
15 , 10 , 5 , 20 , 25 , 30 , 40 , 35.

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

Solution- Suppose A[ ]

1	2	3	4	5	6	7	8
15	10	5	20	25	30	40	35
p			q				r

Here p = 1; r = 8

1 < 8 then q = ⌊(1 + 8)/2⌋ = ⌊4.5⌋ = 4

i.e., q = 4

call MERGE-SORT (A, 1, 4)

MERGE-SORT (A, 5, 8)

MERGE (A, 1, 4, 8)

First call MERGE-SORT (A, 1, 4)

Here p = 1 , r = 4

1 < 4, so q = $\left\lfloor \frac{1+4}{2} \right\rfloor$ = ⌊2.5⌋ = 2

q = 2

then call MERGES0RT (A, 1, 2)

MERGE-SORT (A, 3,4)

MERGE (A, 1, 2, 4)

Now call **MERGE-SORT (A, 1, 2)** firstly

Here p = 1 , r = 2

1 < 2, then q = $\left\lfloor \frac{1+2}{2} \right\rfloor$ = ⌊1.5⌋ = 1

So, call MERGE-SORT (A, 1, 1)

MERGE-SORT (A, 2, 2)

MERGE (A, 1, 1, 2)

call **MERGE-SORT (A, 1,1)**

Here 1 ≮ 1 No change.

call **MERGE-SORT (A, 2, 2)**

Here 2 ≮ 1 so, no change.

call **MERGE (A, 1, 1, 2)**

Here p = 1, q = 1, r = 2

so, n_1 = q - p + 1 = 1 - 1 + 1

n_1 = 1

n_2 =2 - 1

n_2 = 1

Create arrays L [1...2] and R [1...2]

for L= 1 to 1

By Siddharth S. Shukla (BE, ME, PhD*)

i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352

No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

L[1] = A[1+1 - 1]
L[1] = A[1]
for j = 1 to 1
R[1]= A[1+1]
R[1] = A[2]
Now L[2] = ∞ and i=1
R[2]= ∞ j=1
For k = 1 to 2
k=1
if L[1] ≤ R[1]
15 ≤ 10 (false)
So A[1] = R[1] i.e., A[1] = 10
and j = 1 + 1 = 2
k = 2 and j = 2
L[1] ≤ R[2]
15 ≤ ∞ (true)
So i = 1 + 1 = 2 and A[2] = 15
Now A[ ] =

1	2	3	4	5	6	7	8
10	15						

call **MERGE-SORT (A, 3,4)**
Here p = 3 , r = 4
3 < 4, then q = $\left\lfloor \frac{p+r}{2} \right\rfloor = \left\lfloor \frac{3+4}{2} \right\rfloor = \lfloor 3.5 \rfloor = 3$
q = 3
call MERGE-SORT (A, 3, 3) (no change)
MERGE-SORT (A, 4,4) (no change)
MERGE (A, 3, 3, 4)
So, **call MERGE (A, 3, 3, 4)**
Here p = 3, q = 3
So, $n_1 = 1$ r = 4.
So, create array L and R.
for i = 1 to 1
L[1]= A[3 + 1 - 1]
L[1] = A[3] i.e., L[1] = 5

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

for j = 1 to 1

R[1] = A[3 + 1]

R[1]=20

i.e.,

L[2] = ∞ , and R[2] = ∞

and i=1 and j = 1

Now for k = 3 to 4

k=3, i = 1, j = 1

L[1] = 5

R[1] =20

Here 5 ≤ 20 (True)

then A[3] = 5 and i = 1 + 1 = 2

k = 4, i = 2, j = 1

L[2] = ∞ , R[1] = 20

L[2] ≮ R[1] , so

A[4] = 20

Now –

1	2	3	4	5	6	7	8
10	15	5	20				

call **MERGE (A, 1, 2, 4)**

Here p = 1, q = 2, r = 4

n1 = 2 - 1 + 1

n1 = 2

n2 = 4 - 2

n2 = 2

Create array L[1. .3] and R [1. 3]

For i = 1 to 2

i = 1, L[1] = A[1 + 1 - 1]

L[1] = A[1] i.e., L[1] = 10

i=2, L[2] = A[1 + 2 - 1]

L[2] = A[2]

L[2]=15

i.e., L[ ]

1	2	3
10	15	∞

By Siddharth S. Shukla (BE, ME, PhD*)

i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352

No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

For j = 1 to 2

R[1] = A[2 + 1] = 5

R[2] = A[2 + 2] = 20

i.e., R[ ]

1	2	3
5	20	∞

Now i = 1; j = 1

For k = 1 to 4

k = 1, L[1] ≤ R[1] i.e., 10 ≤ 5 (False)

so A[1] = R [1]

i.e., A[1] = 5 and j =1+1 = 2

k = 2, L[1] ≤ R[2] i.e., 10 ≤ 20 (True)

so A[2] = L[1]

i.e., A[2] = 10 and j = 1+1 = 2

k = 3, L[2] ≤ R[2] i.e., 15 ≤ 20 (True)

so A[3] = L[2]

i.e., A[3] = 15 and i = 2+1 = 3

k = 4, L [3] ≤ R [2] i.e., ∞ ≤ 20 (False)

so A[4] = R[2]

i.e., A[4] = 20 and j = 2+1 = 3

i.e., now array A[ ] is –

1	2	3	4	5	6	7	8
10	15	5	20				

call **MERGE-SORT (A, 5, 8)**

Here p = 5, r = 8

5 < 8, so, q = $\left\lfloor \frac{5+8}{2} \right\rfloor$ = ⌊6.5⌋ = 6

i.e., q = 6

then call MERGE-SORT (A, 5, 6)

MERGE-SORT A, 7,8)

MERGE (A, 5, 6, 8)

we call **MERGE-SORT (A, 5, 6)**

Here p = 5, r = 6

5 < 6 then q = $\left\lfloor \frac{5+6}{2} \right\rfloor$ = ⌊5.5⌋ = 5

q = 5

By Siddharth S. Shukla (BE, ME, PhD*)

i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352

No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

then call MERGE-SORT (A, 5, 5)

MERGE-SORT (A, 6,6)

MERGE (A, 5, 5, 6)

we call **MERGE-SORT (A, 5,5)**

Here $5 < 5$ (False) so, No change

call **MERGE-SORT (A, 6, 6)**

Here $6 < 6$ (False) so, No change

call **MERGE (A, 5, 5, 6)**

Here $p = 5$, $q = 5$, $r = 6$

$n_1 = 1$

$n_2 = 1$

Create arrays L[1...2] and R [1...2]

L[1] = A[5] i.e., L[1] = 25

R[1] = A[6] i.e.,R[1] = 30

$L[2] = \infty$

i.e., L[ ]

1	2
25	∞

$R[2] = \infty$

i.e., R[ ]

1	2
30	∞

$i = 1$ and $j = 1$

For $k = 5$ to 6

$k = 5$, $L[1] \leq R[1]$ i.e., $25 \leq 30$ (True)

so, A[5] = 25

and $i = 1 + 1 = 2$

$k = 6$, $L[2] \leq R[1]$ i.e., $\infty \leq 30$ (False)

so A[6] = 30

and $j = 1 + 1 = 2$

Thus now array A[ ] is –

1	2	3	4	5	6	7	8
5	10	15	20	25	30		

call **MERGE-SORT (A, 7,8)**

we get A [ ] -

1	2	3	4	5	6	7	8

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

5	10	15	20	25	30	35	40

call **MERGE (A, 5, 6,8)**

We get the array A[ ] -

1	2	3	4	5	6	7	8
5	10	15	20	25	30	35	40

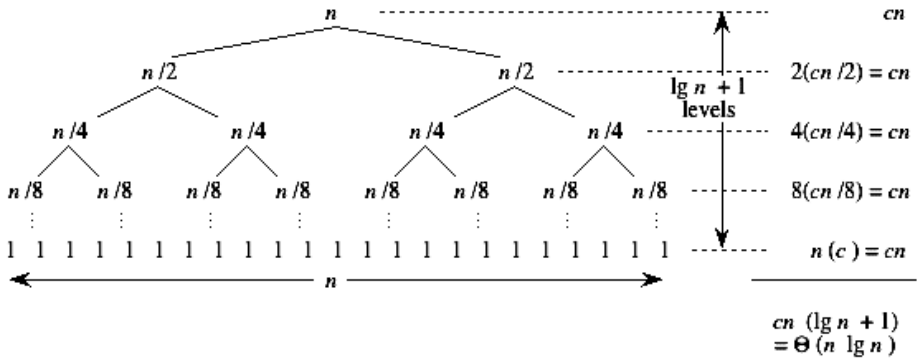
call **MERGE (A, 1, 4, 8)**

We get the sorted array A[ ] as –

1	2	3	4	5	6	7	8
5	10	15	20	25	30	35	40

This is final sorted array.

Complexity of merge sort –



where n = number of element = upper bound – lower bound + 1.

Quick sort –

Quick sort is an example of divide and conquer strategy. In quick sort, we divide the array of items to be sorted into two partitions and then call the quick sort procedure recursively to sort the two partitions, i.e. we divide the problem into two smaller ones and conquer by solving the smaller ones.

Divide: Partition (rearrange) the array $A[p\,..\,r]$ into two (possibly empty) subarrays $A[p\,..\,q-1]$ and $A[q+1\,..\,r]$ such that each element of $A[p\,..\,q-1]$ is less than or equal to $A[q]$, which is, in turn, less than or equal to each element of $A[q+1\,..\,r]$. Compute the index q as part of this partitioning procedure.

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

Conquer: Sort the two subarrays $A[p\,..\,q-1]$ and $A[q+1\,..\,r]$ by recursive calls to quick sort.

Combine: Since the subarrays are sorted in place, no work is needed to combine them: the entire array $A[p\,..\,r]$ is now sorted.

The following procedure implements quicksort.

QUICKSORT *(A, p, r)*

1. if $p < r$
2. then $q \leftarrow$ PARTITION *(A, p, r)*
3. QUICKSORT *(A, p, q − 1)*
4. QUICKSORT *(A, q + 1, r)*

As a first step, Quick Sort chooses as pivot one of the items in the array to be sorted. Then array is partitioned on either side of the pivot. Elements that are less than or equal to pivot will move toward the left and elements that are greater than or equal to pivot will move toward the right.

Partitioning the array – The key to the algorithm is the PARTITION procedure, which rearranges the subarray $A[p\,..\,r]$ in place.

PARTITION *(A, p, r)*

1. $x \leftarrow A[r]$
2. $i \leftarrow p - 1$
3. **for** $j \leftarrow p$ **to** $r - 1$
4. **do if** $A[j] \le x$
5. **then** $i \leftarrow i + 1$
6. exchange $A[i] \leftrightarrow A[j]$
7. exchange $A[i + 1] \leftrightarrow A[r]$
8. **return** $i + 1$

The running time of the partition procedure is Θ(n) where $n = r - p + i$ which is the number of keys in the array. Another argument that running time of PARTITION on a sub array of size Θ(n) is as follows: Pointer 'i' and pointer 'j' start at each end and move towards each other, conveying somewhere in the middle. The total number of times that *i* can be incremented and *j* can be decremented is therefore *O(n)*. Associated with each increment or decrement there are *O(1)* comparisons and swaps. Hence, the total time is *O(n).*

Example - Sort : 36, 15, 40, 1, 60, 20, 55, 25, 50, 20 , by using quick sort algorithm.

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

Is it a stable sorting algorithm ?

Solution - **Stable sorting** - We say that a sorting algorithm is stable if, when two records have the same key, they stay in their original order.

Let A[ ] =

1	2	3	4	5	6	7	8	9	10
36	15	40	1	60	20	55	25	50	20
									x

Here, $p = 1$, $r = 10$.

$x = A[10]$ i.e., $x = 20$

$i = p - 1$ i.e., $i = 0$

$j = l$ to 9

Now, $j = 1$ and $i = 0$

$A[j] = A[1] = 36$ and $36 \not\leq 20$

So, $j = 2$ and $i = 0$

$A[2] = 15 \leq 20$ (True)

then, $i = 0 + 1 = 1$ and $A[1] \leftrightarrow A[2]$

i.e.,

1	2	3	4	5	6	7	8	9	10
15	36	40	1	60	20	55	25	50	20

Now, $j = 3$ and $i = 1$

$A[3] = 40 \not\leq 20$

$j = 4$ and $i = 1$

$A[4] = 1 \leq 20$ (True)

then, $i = 1+1 = 2$ and $A[2] \leftrightarrow A[4]$

i.e.,

1	2	3	4	5	6	7	8	9	10
15	1	40	36	60	20	55	25	50	20

Now, $j = 5$, $i = 2$

$A[5] = 60 \not\leq 20$.

$j = 6$ and $i = 2$

$A[j] = 20 \leq 20$ (True)

then, $i = 1+1$ i.e., $i = 2+1 = 3$

and $A[3] \leftrightarrow A[6]$

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

i.e.,

1	2	3	4	5	6	7	8	9	10
15	1	20	36	60	40	55	25	50	20

Now, $j = 7, i = 3$

$A[7] = 55 \not\leq 20$

$j = 8, i = 3$

$A[8] = 25 \not\leq 20$

$j = 9, i = 3$

$A[9] = 50 \not\leq 20$

Now, $A[i + 1]$ i.e., $A[4] \leftrightarrow A[10]$

i.e.,

1	2	3	4	5	6	7	8	9	10
15	1	20	20	60	40	55	25	50	36

and two sub-arrays are -

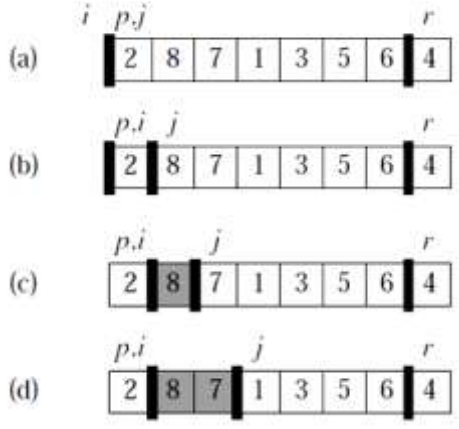
15	1	20

and,

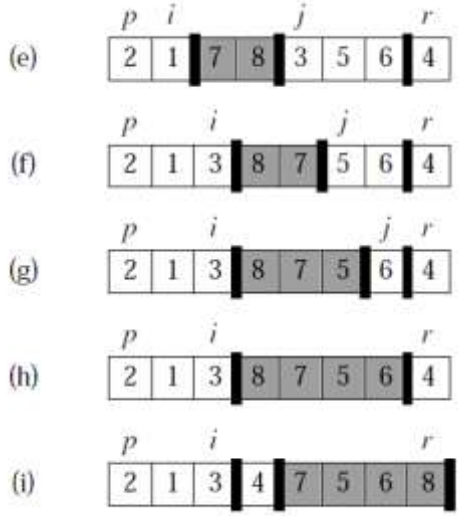
60	40	55	25	50	36

Thus, this is a stable algorithm.

Example - The operation of PARTITION on a sample array. Lightly shaded array elements are all in the first partition with values no greater than x. heavily shaded elements are in the second partition with values greater than x. The un-shaded elements have not yet been put in one of the first two partitions, and the final white element is the pivot.



By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.



(a) The initial array and variable settings. None of the elements have been placed in either of the first two partitions.
(b) The value 2 is "swapped with itself" and put in the partition of smaller values.
(c)–(d) The values 8 and 7 are added to the partition of larger values.
(e) The values 1 and 8 are swapped, and the smaller partition grows.
(f) The values 3 and 7 are swapped, and the smaller partition grows.
(g)–(h) The larger partition grows to include 5 and 6 and the loop terminates.
(i) In lines 7–8, the pivot element is swapped so that it lies between the two partitions.

COMPLEXITY OF QUICK SORT -

The running time of quick sort depends on whether partition is balanced or unbalanced, which in turn depends on which elements of an array to be sorted are used for partitioning.

- A very good partition splits an array up into two equal sized arrays.
- A bad partition splits an array up into two arrays of very different sizes.
- The worst partition puts only one element in one array and all other elements in the other array.

If the partitioning is balanced, the Quick sort runs asymptotically as fast as merge sort. On the other hand, if partitioning is unbalanced, the Quick sort runs asymptotically as slow as insertion sort.

Best Case Partitioning - The best thing that could happen in Quick sort would be that each partitioning stage divides the array exactly in half. In other words, the best to be a median of the keys in A[p . . . r] every time procedure 'Partition' is called. The procedure 'Partition' always splits

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

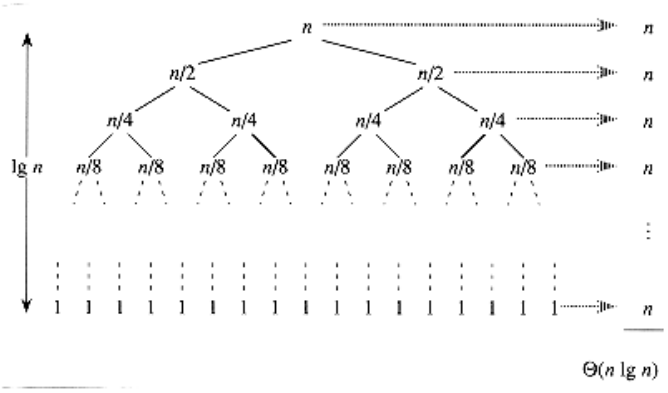
the array to be sorted into two equal sized arrays. If the procedure 'Partition' produces two regions of size n/2, the recurrence relation is then -

$$T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{2}\right) + \theta(n)$$

$$T(n) = 2T\left(\frac{n}{2}\right) + \theta(n)$$

And from case 2 of Master theorem

$$T(n) = \theta(n \lg n)$$



Worst case Partitioning - The worst-case occurs if given array A[1 . . . n] is already sorted.
The PARTITION (A, p. r) call always returns 'p'.
So, successive calls to partition will split arrays of length n, n - 1, n - 2, . . . ,2 and
Running time proportional to n+(n - l) + (n - 2) + . . . + 2 = [(n + 2)(n - 1)/2]= (n2).
The worst-case also occurs if A [1.. n] starts out in reverse order.

Quick sort algorithm is fastest when the median of the array is chosen as the pivot element. This is because the resulting partitions are of very similar size. Each partition splits itself in two and thus the best case is reached very quickly.

Sorting in Linear Time –

Merge sort, quick sort and heap sort algorithms share an interesting property : the sorted order they determine is based only on comparisons between the input elements. We call such sorting algorithms comparison sorts. There are sorting algorithms that run faster but they require special assumption about the input sequence to be sort. Examples that run in linear time - counting sort, radix sort, and bucket sort.

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

Counting sort –

Counting sort assumes that each of the 'n' input elements is an integer in the range 0 to k, for some integer k. When $k = O(n)$, the sort runs in $O(n)$ time. The basic idea of counting sort is to determine, for each input element x, the number of elements less than x. This information can be used to place element x directly into its position in the output array. For example, if there are 17 elements less than x, then x belongs in output position 18. This scheme must be modified slightly to handle the situation in which several elements have the same value, since we don't want to put them all in the same position. In the code for counting sort, we assume that the input is an array $A[1 \ldots n]$, and thus $length[A] = n$. We require two other arrays: the array $B[1 \ldots n]$ holds the sorted output, and the array $C[0 \ldots k]$ provides temporary working storage.(k is the highest number in array A[ ]).

COUNTING-SORT *(A, B, k)*

1. **for** $i \leftarrow 0$ **to** k
2. **do** $C[i] \leftarrow 0$
3. **for** $j \leftarrow 1$ **to** $length[A]$
4. **do** $C[A[j]] \leftarrow C[A[j]] + 1$
5. /* $C[i]$ now contains the number of elements equal to i. */
6. **for** $i \leftarrow 1$ **to** k
7. **do** $C[i] \leftarrow C[i] + C[i-1]$
8. /* $C[i]$ now contains the number of elements less than or equal to i. */
9. **for** $j \leftarrow length[A]$ **downto** 1
10. **do** $B[C[A[j]]] \leftarrow A[j]$
11. $C[A[j]] \leftarrow C[A[j]] - 1$

An important property of counting sort is that it is ***stable*** : numbers with the same value appear in the output array in the same order as they do in the input array. That is, ties between two numbers are broken by the rule that whichever number appears first in the input array appears first in the output array. Normally, the property of stability is important only when satellite data are carried around with the element being sorted.

Example – Find the sorted array by using Counting – sort algorithm.

A[ ] = { 6 , 0 , 2 , 0 , 1 , 3 , 4 , 6 , 1 , 3 , 2 }

Solution -

1	2	3	4	5	6	7	8	9	10	11

By Siddharth S. Shukla (BE, ME, PhD*)

i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352

No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

A	6	0	2	0	1	3	4	6	1	3	2

Here k = 6 (highest number in array A)

For i = 0 to 6

c[i] = 0

	0	1	2	3	4	5	6
c	0	0	0	0	0	0	0

For j = 1 to 11

	0	1	2	3	4	5	6
c	2	2	2	2	1	0	2

For i = 1 to 6

	0	1	2	3	4	5	6
c	2	4	6	8	9	9	11

j	A[j]	c[A[j]]	B[c[A[j]]] = A[j]	c[A[j]] = c[A[j]] – 1
11	2	6	B[6] = 3	c[2] = 5
10	3	8	B[8] = 3	c[3] = 7
9	1	4	B[4] = 1	c[1] = 3
8	6	11	B[11] = 6	c[6] = 10
7	4	9	B[9] = 4	c[4] = 8
6	3	7	B[7] = 3	c[3] = 6
5	1	3	B[3] = 1	c[1] = 2
4	0	2	B[2] = 0	c[0] = 1
3	2	5	B[5] = 2	c[2] = 4
2	0	1	B[1] = 0	c[0] = 0
1	6	10	B[10] = 6	c[6] = 9

	1	2	3	4	5	6	7	8	9	10	11
B	0	0	1	1	2	2	3	3	4	6	6

Thus, the final sorted array is B[ ].

Radix sort –

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

Radix sort is the algorithm that is useful when there is a constant 'd' such that all the keys are 'd' digit numbers. To execute Radix-sort , for $p = 1$ toward 'd' sort the number with respect to the p^{th} digit from the right using any linear – time stable sort.

In a typical computer, which is a sequential random-access machine, radix sort is sometimes used to sort records of information that are keyed by multiple fields. For example, we might wish to sort dates by 3 keys: year, month, and day. We could run a sorting algorithm with a comparison function that, given two dates, compares years, and if there is a tie, compares months, and if another tie occurs, compares days. Alternatively, we could sort the information three times with a stable sort: first on day, next on month, and finally on year.

The code for radix sort is straightforward. The following procedure assumes that each element in the n-element array A has d digits, where digit 1 is the lowest-order digit and digit d is the highest-order digit.

RADIX-SORT *(A, d)*

1. **for** $i \leftarrow 1$ **to** d
2. **do** use a stable sort to sort array A on digit i

The analysis of the running time depends on the stable sort used as the intermediate sorting algorithm. When each digit is in the range 0 to (k – 1) (so that it can take on k possible values), and k is not too large, counting sort is the obvious choice. Each pass over n d-digit numbers then takes time $\theta(n+k)$. There are d passes, so the total time for radix sort is $\theta(d(n+k))$.

Example - The operation of radix sort on a list of seven 3-digit numbers.

The leftmost column is the input. The remaining columns show the list after successive sorts on increasingly significant digit positions. Shading indicates the digit position sorted on to produce each list from the previous one.

Bucket sort -

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

Bucket sort runs in linear time when the input is drawn from a uniform distribution. Like counting sort, bucket sort is fast because it assumes something about the input. Whereas counting sort assumes that the input consists of integers in a small range, bucket sort assumes that the input is generated by a random process that distributes elements uniformly over the interval [0, 1). (Let interval U = [0 , 1))

To sort 'n' input numbers, Bucket – sort –

1. Partitions 'U' into 'n' non – overlapping intervals, called buckets.
2. Put each input number into its bucket.
3. Sort each bucket using a simple algorithm.
4. Concatenates the sorted list.

The idea of bucket sort is to divide the interval $[0, 1)$ into n equal-sized subintervals, or ***buckets***, and then distribute the n input numbers into the buckets. Since the inputs are uniformly distributed over [0, 1), we don't expect many numbers to fall into each bucket. To produce the output, we simply sort the numbers in each bucket and then go through the buckets in order, listing the elements in each.

Our code for bucket sort assumes that the input is an n-element array A and that each element - $A[i]$ in the array satisfies $0 \le A[i] < 1$. The code requires an auxiliary array $B[0 \,..\, n-1]$ of linked lists (buckets) and assumes that there is a mechanism for maintaining such lists.

BUCKET-SORT(A)

1. $n \leftarrow length[A]$
2. **for** $i \leftarrow 1$ **to** n
3. **do** insert $A[i]$ into list $B[\lfloor nA[i] \rfloor]$
4. **for** $i \leftarrow 0$ **to** $n-1$
5. **do** sort list $B[i]$ with insertion sort
6. concatenate the lists $B[0], B[1], \ldots, B[n-1]$ together in order

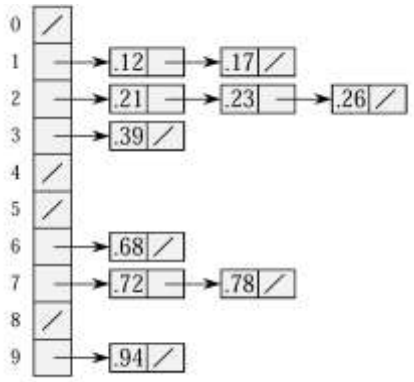
The running time of bucket sort is –

$$T(n) = \Theta(n) + \sum_{i=0}^{n-1} O(n_i^2) .$$

Example – Sort the array using Bucket – sort.

Elements - < 0.78 , 0.17 , 0.39, 0.26 , 0.72 , 0.94 , 0.21 , 0.12 , 0.23 , 0.68 >

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.



Sorted array - < 0.12 , 0.17 , 0.21 , 0.23 , 0.26 , 0.39 , 0.68 , 0.72 , 0.78 , 0.94 >

Heap sort -

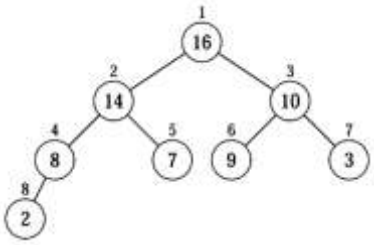
- **Heap** – It is a binary tree with each node containing a data item & satisfy the following properties -
  - It should be always binary tree.
  - The value at any node is always greater than or equal to the values of its children.
  - All the leafs should be at level 'd' or 'd-1' , where d = depth or height of tree.

  OR
- The heap data structure is an array object that can be viewed as a nearly complete binary tree. Each node of the tree corresponds to an element of the array that stores the value in the node. The tree is completely filled on all levels except possibly the lowest, which is filled from the left up to a point.
- Heapsort also introduces another algorithm design technique: the use of a data structure, in this case one we call a "heap," to manage information during the execution of the algorithm. Not only is the heap data structure useful for heapsort, but it also makes an efficient priority queue.
- An array A that represents a heap is an object with two attributes:
  - length[A] - which is the number of elements in the array, and
  - heap-size[A] - the number of elements in the heap stored within array A.
- The root of the tree is A[1], and given the index i of a node, the indices of its parent PARENT(i), left child LEFT(i), and right child RIGHT(i) can be computed as :-

```
PARENT(i)
        return [ i/2 ]
LEFT(i)
        return 2i
RIGHT(i)
        return 2i + 1
```


By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

Example - A binary tree –



An array representing binary tree -

16	14	10	8	7	9	3	2

- There are two kinds of binary heaps :-
  - max-heaps
  - min-heaps
- In a min-heap, a node dominates its children by containing a smaller key than they do, while in a max-heap parent nodes dominate by being bigger.
- **Heap property** –
  - In a max-heap, the max-heap property is that for every node 'i' other than the root, A[PARENT(i)] ≥ A[i]. i,e, the largest element in a max-heap is stored at the root.
  - A min-heap is organized in the opposite way; the min-heap property is that for every node 'i' other than the root, A[PARENT(i)] ≤ A[i]. i.e. the smallest element in a min-heap is at the root.
- The height of a node in a heap to be the number of edges on the longest simple downward path from the node to a leaf.
- The height of the heap to be the height of its root.

Building a heap –

We use the procedure MAX-HEAPIFY in a bottom-up manner to convert an array A[1 . . n], where n = length[A], into a max-heap.

BUILD-MAX-HEAP(A)

1. heap-size[A] ← length[A]
2. for i ← length[A]/2 downto 1
3. do MAX-HEAPIFY(A, i)

MAX-HEAPIFY(A, i)

1. l ← LEFT(i)

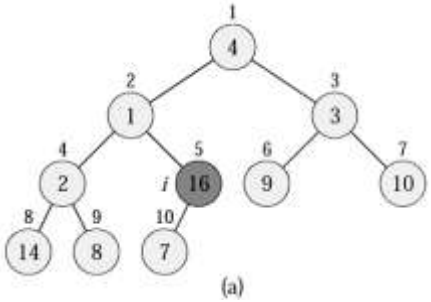
By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

2. r ← RIGHT(i)
3. if l ≤ heap-size[A] and A[l] > A[i]
4. then largest ←l
5. else largest ←i
6. if r ≤ heap-size[A] and A[r] > A[largest]
7. then largest ←r
8. if largest ≠ i
9. then exchange A[i] ↔ A[largest]
10. MAX-HEAPIFY(A, largest)

Example - Action of BUILD-MAX-HEAP.

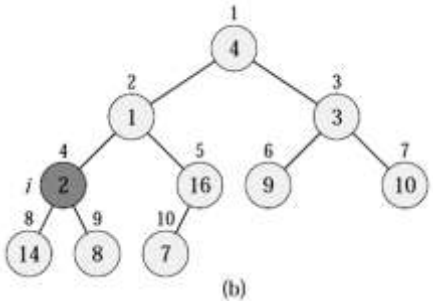
A	4	1	3	2	16	9	10	14	8	7

Solution - Constructing max – heap –



(a)

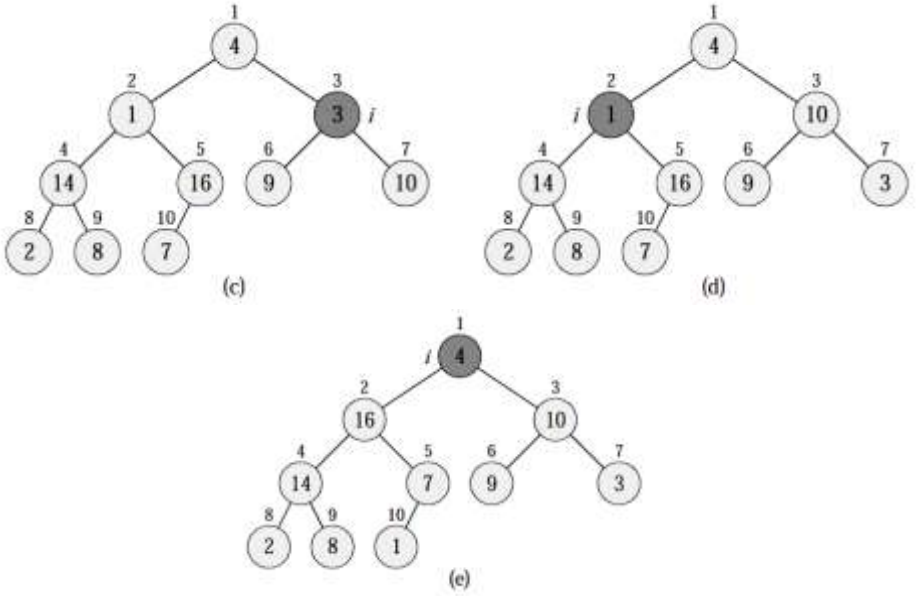
(a) A 10-element input array A and the binary tree it represents. The figure shows that the loop index 'i' refers to node 5 before the call MAX-HEAPIFY(A, i).



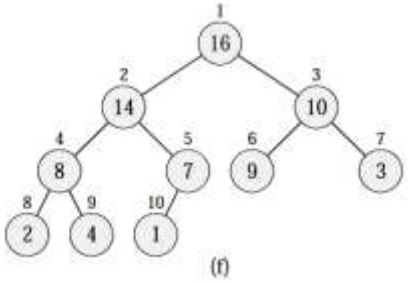
(b)

(b) The data structure that results. The loop index 'i' for the next iteration refers to node 4.

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.



(c)–(e) Subsequent iterations of the for loop in BUILD-MAX-HEAP. Observe that whenever MAX-HEAPIFY is called on a node, the two subtrees of that node are both max-heaps.



(f) The max-heap after BUILD-MAX-HEAP finishes.

Complexity of Build – heap -

The running time of BUILD-MAX-HEAP can be bounded as –

$$\sum_{h=0}^{\lfloor \lg n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil O(h)$$

but –

$$\sum_{h=0}^{\infty} \frac{h}{2^h} = \frac{1/2}{(1-1/2)^2} = 2 .$$

hence –

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

$$O\left(n\sum_{h=0}^{\lfloor \lg n \rfloor}\frac{h}{2^h}\right) = O\left(n\sum_{h=0}^{\infty}\frac{h}{2^h}\right) = O(n)\,.$$

The heap sort algorithm -

The heap sort algorithm starts by using BUILD-MAX-HEAP to build a max-heap on the input array A[1 . . n], where n = length[A].

HEAPSORT(A)

1. BUILD-MAX-HEAP(A)
2. for i ← length[A] downto 2
3. do exchange A[1] ↔ A[i]
4. heap-size[A] ← heap-size[A] − 1
5. MAX-HEAPIFY(A, 1)

The HEAPSORT procedure takes time O(n lg n), since the call to BUILD-MAXHEAP takes time O(n) and each of the n − 1 calls to MAX-HEAPIFY takes time O(lg n).
The procedure HEAP-MAXIMUM implements the MAXIMUM operation in O(1) time.

HEAP-MAXIMUM(A)

1. return A[1]

The procedure HEAP-EXTRACT-MAX implements the EXTRACT-MAX operation. It is similar to the for loop body (lines 3–5) of the HEAPSORT procedure.

HEAP-EXTRACT-MAX(A)

1. if heap-size[A] < 1
2. then error “heap underflow”
3. max ← A[1]
4. A[1] ← A[heap-size[A]]
5. heap-size[A] ← heap-size[A] − 1
6. MAX-HEAPIFY(A, 1)
7. return max

– The running time of HEAP-EXTRACT-MAX is O(lg n), since it performs only a constant amount of work on top of the O(lg n) time for MAX-HEAPIFY.

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

– The procedure MAX-HEAP-INSERT implements the INSERT operation. It takes as an input the key of the new element to be inserted into max-heap A.

MAX-HEAP-INSERT(A, key)

1. heap-size[A] ← heap-size[A] + 1
2. A[heap-size[A]]←−∞
3. HEAP-INCREASE-KEY(A, heap-size[A], key)

HEAP-INCREASE-KEY(A, i, key)

1. if key < A[i]
2. then error "new key is smaller than current key"
3. A[i] ← key
4. while i > 1 and A[PARENT(i)] < A[i]
5. do exchange A[i] ↔ A[PARENT(i)]
6. i ← PARENT(i)

The running time of MAX-HEAP-INSERT on an n-element heap is O(lg n) and the running time of HEAP-INCREASE-KEY on an n-element heap is O(lg n).

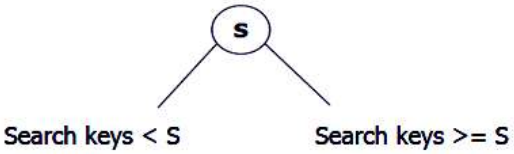
2 – 3 Tree –

A 2 – 3 tree consist of 2 - nodes or 3 - nodes. In other words - A 2-3 tree is a tree in which each internal node (non-leaf) has either 2 or 3 children, and all leaves are at the same level.

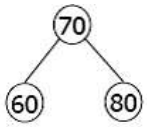
2 – 3 Tree –

- A node may contain 1 or 2 keys .
- All leaf nodes are at the same depth .
- All non-leaf nodes (except the root) have either 1 key and two sub-trees, or 2 keys and three sub-trees.
- The only changes in depth are when the root splits or underflows.
- A 2-3 tree containing n keys is guaranteed to have height O(log n), and its search, insert and delete algorithms all take O(log n) time.
- **2 - node** - A 2 - node is a node with one data and either 2 children or no children.

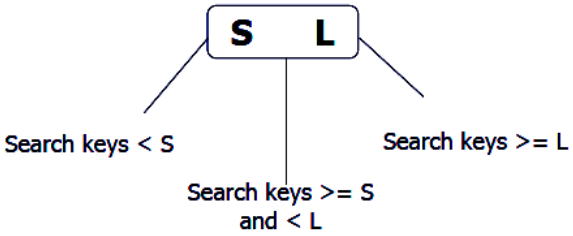
By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.



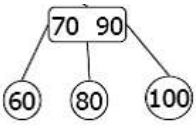
Example –



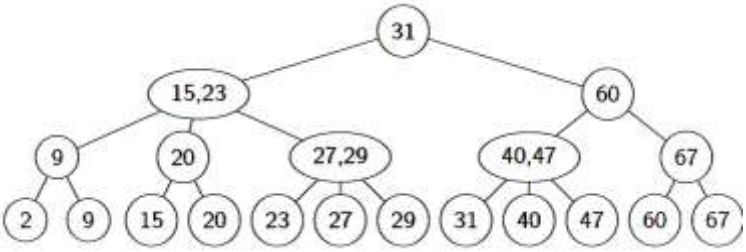
- **3 - node** - A 3 - node is a node with 3 - node item & either 3 children or no children.



Example – All the leaf nodes are at the same level.



Example of a 2 – 3 Tree –



- **Insertion** is at the leaf :-
  If the leaf overflows, split it into two leaves, insert them into the parent, which may also overflow
- **Deletion** is at the leaf :-
  If the leaf underflows (has no items), merge it with a sibling, removing a value and subtree from the parent, which may also underflow

Algorithm to insert an item into 2 – 3 Tree –

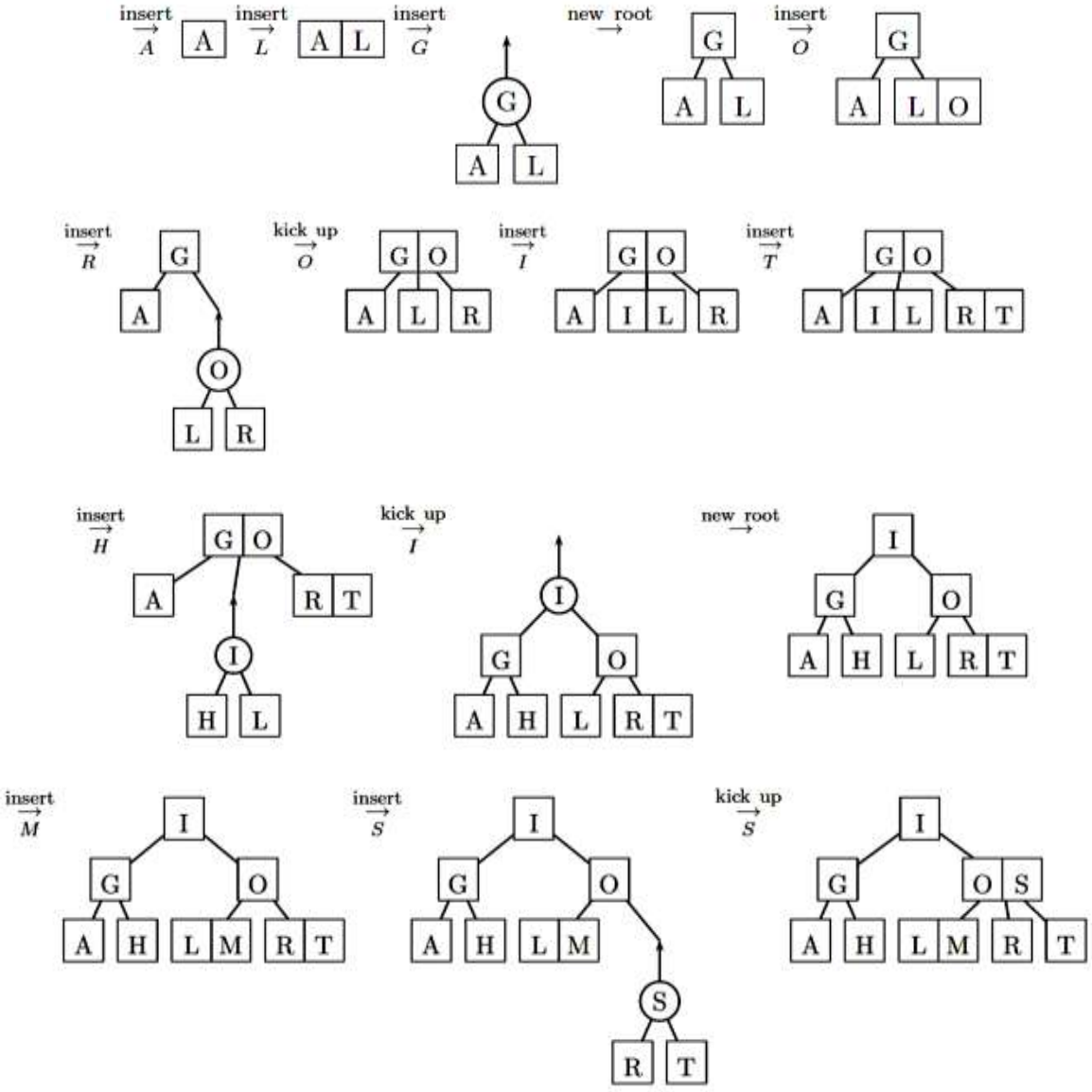
– Look at the leaf node at which the search for 'i' is terminated.

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

- Insert item 'i' into the leaf node, If the leaf as 2 data item, return, If leaf code has 3 data item split the node into 2 node and send the middle item to the parent.
- If the internal node has 3 items, split the node into2 nodes and send the middle item to the parent.
- If the node is roof node & has 3 items then split the node into 2 nodes, make the middle data item as a root.

Tree Insertion Example -

Here we show the letter-by-letter insertion of the letters -
A L G O R I T H M S into an initially empty 2-3 tree.



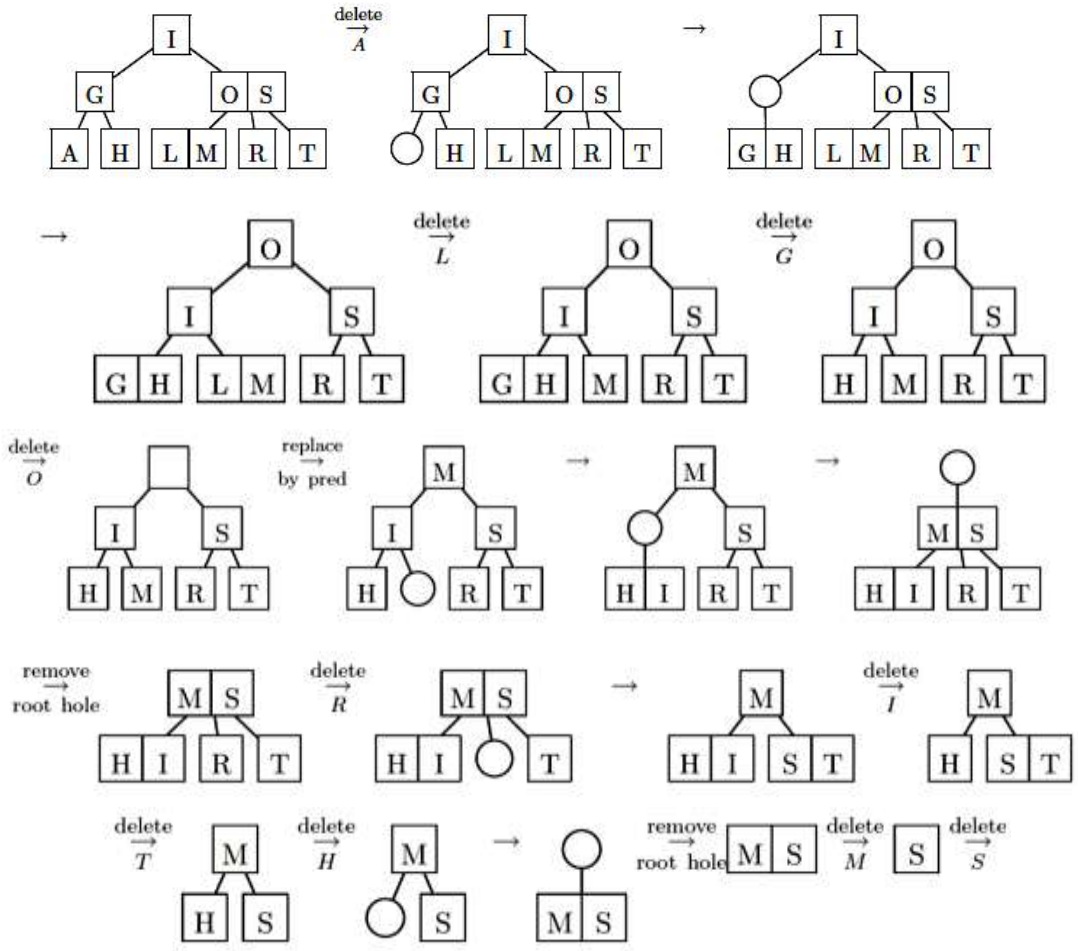
By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

Deletion in 2 - 3 tree -

- delete(2 - 3 tree , item , i)
- Locate the nodes which contain Item i.
- If the node is leaf node swap with its in-order successor.
- If the node contain 2 data item, then simply delete I from the node, else item I from the node, try to redistribute the node from the sibling if not possible merge node.

Tree Deletion Example –

Here we show the letter-by-letter deletion of the letters –
A L G O R I T H M S from the final 2-3 tree of the insertion example:



Height of 2 – 3 Tree –

Height starts from zero '0'.

number of node (n) ≥ maximum number of nodes in height 'h'

$n \geq 2^0 + 2^1 + 2^2 + \ldots\ldots + 2^h$

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

$n \geq 2^{h+1} - 1$

$n + 1 \geq 2^{h+1}$

$\log_2(n + 1) \geq \log_2(2^{h+1})$

$\log (n + 1) \geq h+1$

$h \leq \log (n + 1) - 1$

for 2 nodes : $h = \log (n + 1) - 1$

for 3 nodes : $h \geq \log_3(n + 1) - 1$

Height of 2 – 3 tree –

$$\log_3(n + 1) - 1 \leq h \leq \log (n + 1) - 1$$

Greedy Algorithms –

- It is simple and straight forward method.
- Greedy algorithms build up a solution piece by piece, always choosing the next piece that offers the most obvious and immediate benefit.
- A greedy algorithm always makes the choice that looks best at the moment.
- It makes a locally optimal choice in the hope that this choice will lead to a globally optimal solution.
- Greedy algorithms do not always yield optimal solutions, but for many problems they do.

Greedy approach –

- Minimum spanning tree
  - Prim's algorithm
  - Kruskal's algorithm
- Knapsack problem
- Huffman code

Minimum spanning tree –

- A **spanning tree** of a graph G = (V , E) is a subset of edges from E forming a tree connecting all vertices of 'V' .
- The **minimum spanning tree** - the spanning tree whose sum of edge weights is as small as possible.
- Minimum spanning trees are the answer whenever we need to connect a set of points (representing cities, homes, junctions, or other locations) by the smallest amount of roadway, wire, or pipe.
- Any tree is the smallest possible connected graph in terms of number of edges, while the minimum spanning tree is the smallest connected graph in terms of edge weight.

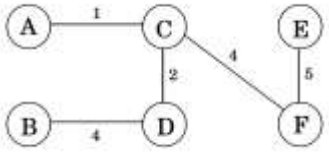
By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

- A minimum spanning tree minimizes the total length over all possible spanning trees.
- There can be more than one minimum spanning tree in a graph.
- All spanning trees of an un-weighted (or equally weighted) graph G are minimum spanning trees, since each contains exactly (n – 1) equal-weight edges.

GENERIC-MST(G,w)

1. A ← ∅
2. while A does not form a spanning tree
3. do find an edge (u, v) that is safe for A
4. A ← A ∪ {(u, v)}
5. return A

Example - The minimum spanning tree has a cost of 16 :



$$\text{weight}(T) = \sum_{e \in E'} w_e$$

weight (T) = 1 + 4 + 2 + 4 + 5 = 16

Prim's algorithm –

Prim's algorithm is a special case of the generic minimum-spanning-tree algorithm. Prim's algorithm has the property that the edges in the set A always form a single tree. The tree starts from an arbitrary root vertex r and grows until the tree spans all the vertices in V.

OR

Prim's algorithm is a greedy algorithm that finds a minimum spanning tree for a connected weighted undirected graph. This means it finds a subset of the edges that forms a tree that includes every vertex, where the total weight of all the edges in the tree is minimized. The algorithm continuously increases the size of a tree, 1 edge at a time starting with a tree consisting of a single vertex, until it span all vertices.

MST-PRIM (G, w, r)

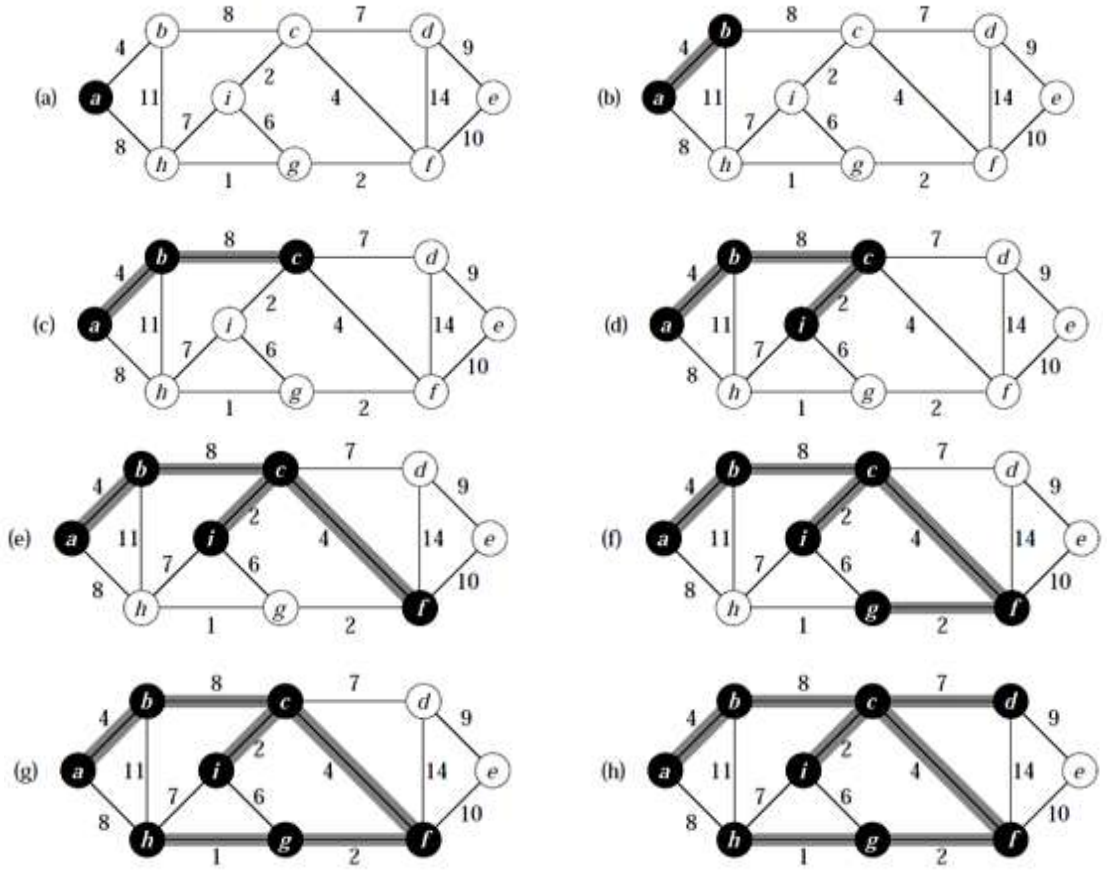
1. for each u ∈ V[G]
2. do key[u]←∞
3. π[u]← NIL

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

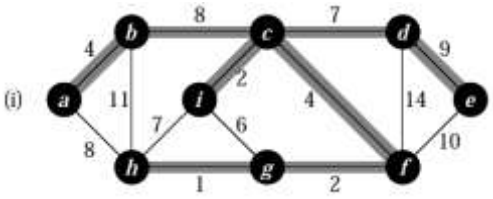
```
4.  key[r] ← 0
5.  Q ← V[G]
6.  while Q ≠ ∅
7.        do u ← EXTRACT-MIN(Q)
8.           for each v ∈ Adj[u]
9.                 do if v ∈ Q and w(u, v) < key[v]
10.                      then π[v]← u
11.                           key[v] ← w(u, v)
```

Prim's algorithm works as –
Lines 1–5 set the key of each vertex to ∞ (except for the root r, whose key is set to 0 so that it will be the first vertex processed), set the parent of each vertex to NIL, and initialize the min-priority queue Q to contain all the vertices.

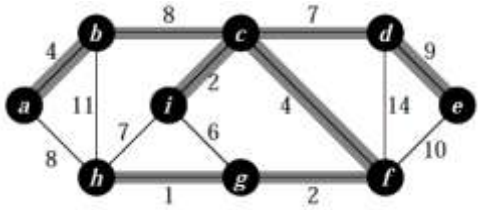
Example – Find minimum spanning tree –



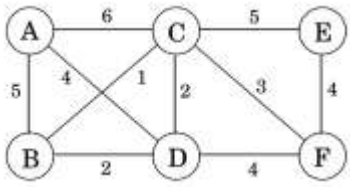
By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.



Final answer -



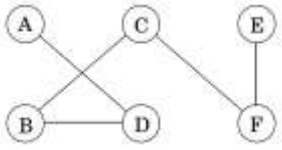
Example – Find the minimum spanning tree of the given graph.



Solution – Table -

Set S	A	B	C	D	E	F
{}	0/nil	∞/nil	∞/nil	∞/nil	∞/nil	∞/nil
A		$5/A$	$6/A$	$4/A$	∞/nil	∞/nil
A, D		$2/D$	$2/D$		∞/nil	$4/D$
A, D, B			$1/B$		∞/nil	$4/D$
A, D, B, C					$5/C$	$3/C$
A, D, B, C, F					$4/F$	

Graph -



Kruskal's algorithm –

Like prim's algorithm, kruskal's algorithm also constructs the minimum spanning tree of a graph by adding edges to the spanning tree one-by-one. Kruskal's algorithm is based directly on the generic minimum-spanning-tree algorithm. At all points during its execution, the set of edges selected by prim's algorithm forms exactly one tree. On the other hand, the set of edges selected by kruskal's algorithm forms a forest of trees. In Kruskal's algorithm, the set A is a forest. The safe edge added to A is always a least-weight edge in the graph that connects two distinct components.

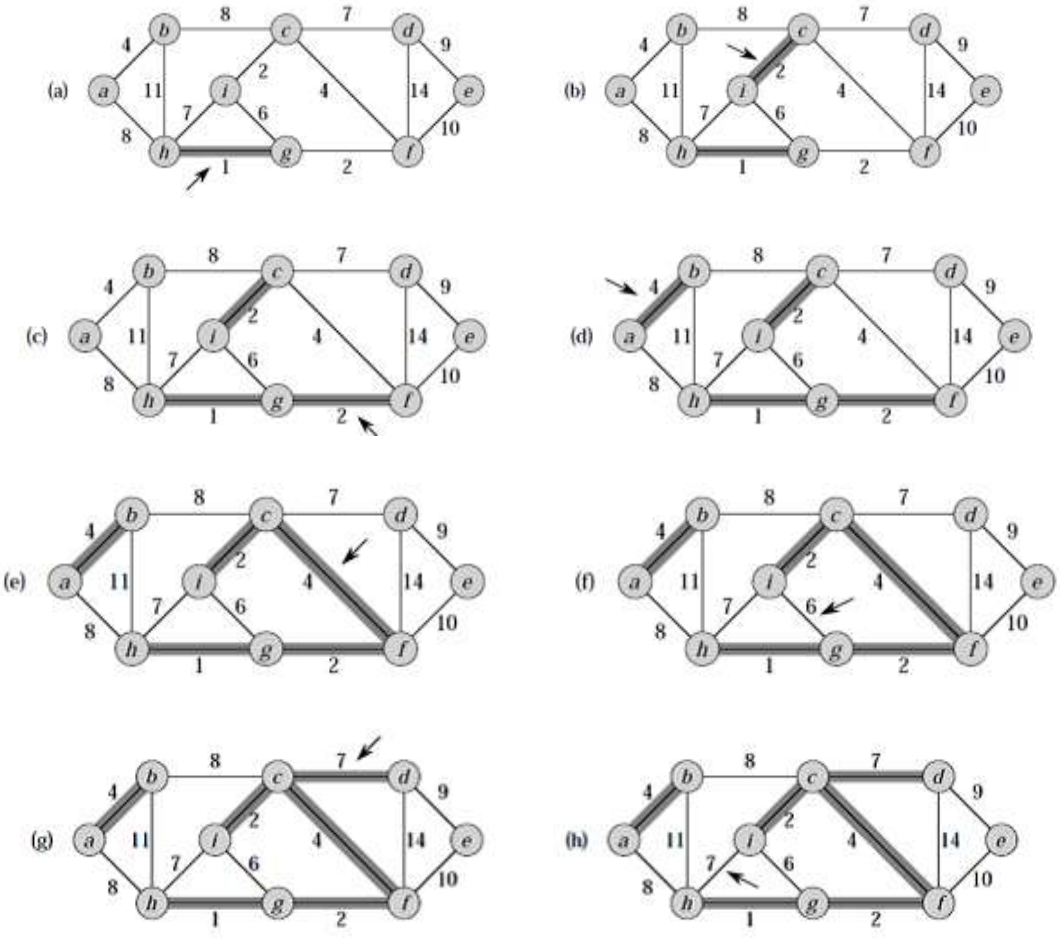
By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

Kruskal's algorithm is conceptually simple. The edges are selected and added to the spanning tree in increasing order of their weights. An edge is added to the tree only if it does not create a cycle.

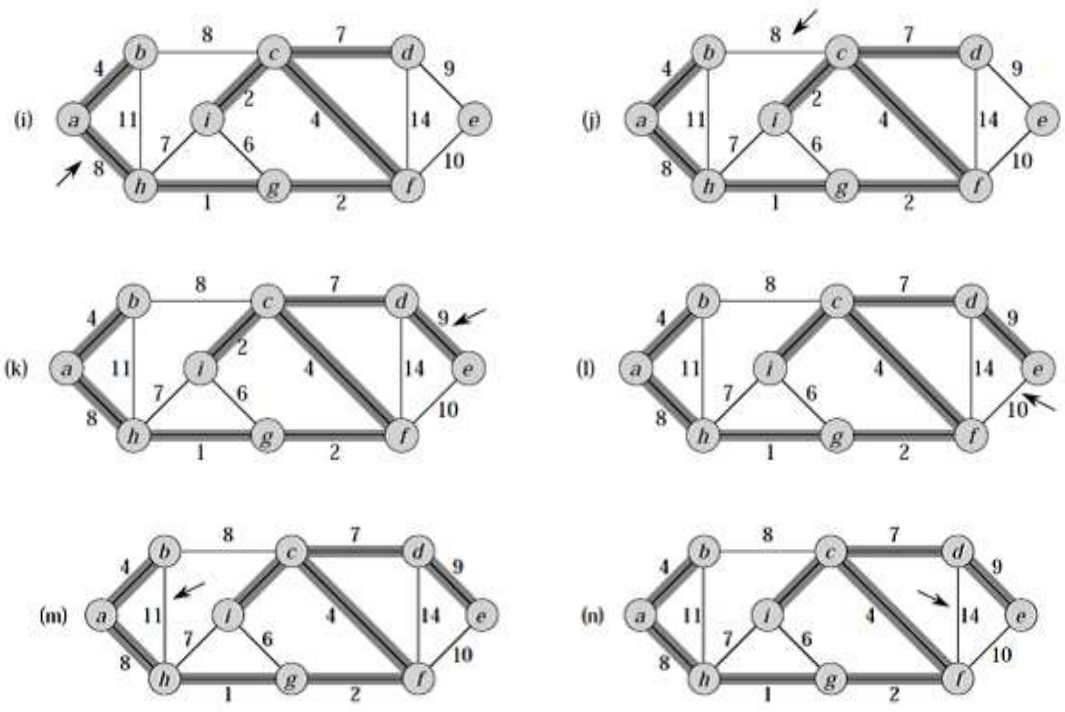
MST-KRUSKAL(G,w)

1. $A \leftarrow \emptyset$
2. for each vertex $v \in V[G]$
3. do MAKE-SET(v)
4. sort the edges of E into nondecreasing order by weight w
5. for each edge $(u, v) \in E$, taken in nondecreasing order by weight
6. do if FIND-SET(u) $\neq$ FIND-SET(v)
7. then $A \leftarrow A \cup \{(u, v)\}$
8. UNION(u, v)
9. return A

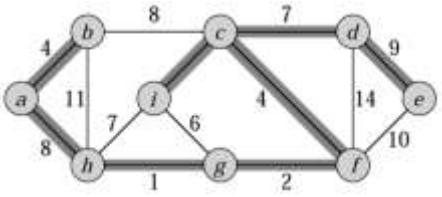
Example – Find minimum spanning tree –



By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.



Final answer –



Knapsack problem –

The knapsack problem is a problem in combinatorial optimization where given a set of items, each with a Weight and a value, determine the number of each item to include in a collection so that the total weight is less than or equal to a given limit and the total value is as large as possible.

We have 'n' kinds of items, 1 through n. Each kind of item 'i' has a value V_i and a weight W_i usually assume that all values and weight are non-negative. The maximum weight that we can carry in the bag is W.

Example - During a robbery, a burglar finds much more loot than he had expected and has to decide what to take. His bag (or "knapsack") will hold a total weight of at most 'W' pounds. There are 'n' items

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

to pick from, of weight $w_1 \ldots\ldots\ldots w_n$, and dollar value $v_1 \ldots\ldots\ldots v_n$. What's the most valuable combination of items he can fit into his bag?

For instance, take W = 10 and –

Item	Weight	Value
1	6	$30
2	3	$14
3	4	$16
4	2	$9

There are two versions of this problem.

- If there are unlimited quantities of each item available, the optimal choice is to pick item 1 and two of item 4 (total: $48).
- If there is one of each item, then the optimal knapsack contains items 1 and 3 (total: $46).

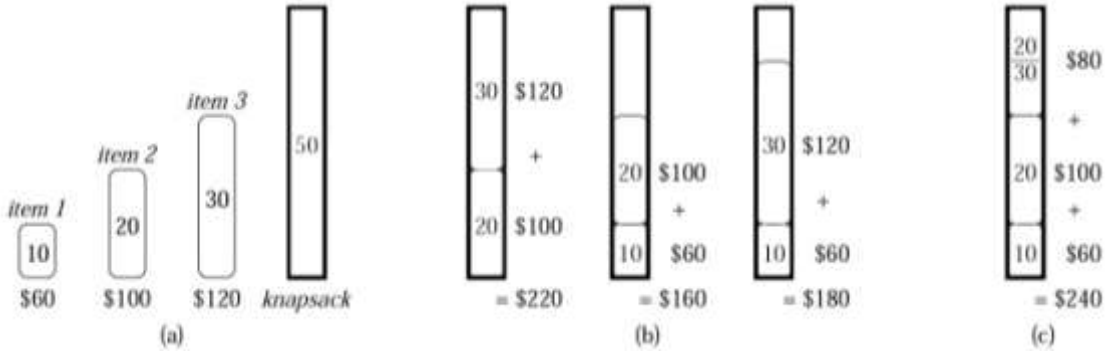
There are two types of Knapsack problem –

- 0-1 knapsack problem
- fractional knapsack problem

0/1 knapsack problem – Here the item can - not be broken into the smaller pieces. We can decide either to take an item or leave it. It is solve by dynamic programming properties.

Fractional knapsack problem – Here we can take fraction of item meaning that the item can be broken into smaller pieces. It can be solve by greedy choice property.

Example -



(a) The thief must select a subset of the three items shown whose weight must not exceed 50 pounds.

(b) The optimal subset includes items 2 and 3. Any solution with item 1 is suboptimal, even though item 1 has the greatest value per pound.

(c) For the fractional knapsack problem, taking the items in order of greatest value per pound yields an optimal solution.

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

Huffman code –

Huffman code is a greedy algorithm. Huffman codes are a widely used and very effective technique for compressing data; savings of 20% to 90% are typical, depending on the characteristics of the data being compressed. Huffman's greedy algorithm uses a table of the frequencies of occurrence of the characters to build up an optimal way of representing each character as a binary string.

Consider the probability of designing a binary character code in which each other is represented by a unique binary string.

There are two types of Binary Character Code –

1. Fixed - length code
2. Variable - length code

Fixed - length code – Each letter represented by an equal number of bits. With a fixed length code, at least 3 bits per character.

Example - Suppose we have 10^5 characters in a data file Normal storage: 8 bits per character (ASCII) $8*10^5$ bits in file. But, we want to compress the file and store it compactly. Suppose only 6 characters appear in the file:

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5

Representation in fixed length code –

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101

Each character represented in 3 bit code –
Total number of character = 10^5
Total number of bits = 1,00,000 * 3 = 3,00,000 bits

Variable - length code – A variable length code can do considerably better than a fixed length code, by giving frequent character code words and infrequent character code words.

Example - Representation in variable length code –

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Variable-length codeword	0	101	100	111	1101	1100

Total number of bits = summation of (number of character * number of bits per character)
= [(45 * 1) + (13 * 3) + (12 * 3) + (16 * 3) + (9 * 4) + (5 * 4)] * 1000

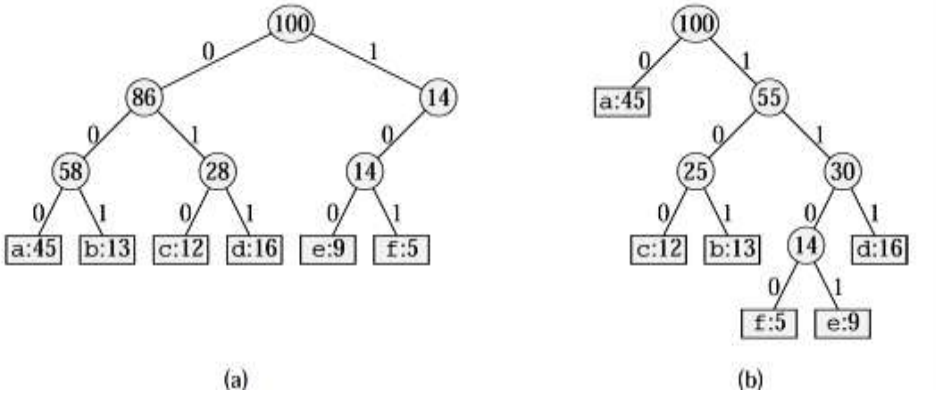
By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

$= [\ 45 + 39 + 36 + 48 + 36 + 20\ ] * 1000$
$= 2.24 * 10^5$

Thus, 224,000 bits required to represent the file.

Prefix codes - Codes in which no codeword is also a prefix of some other codeword. Such codes are called prefix codes. Prefix codes are desirable because they simplify decoding. Since no codeword is a prefix of any other, the codeword that begins an encoded file is unambiguous. We can simply identify the initial codeword, translate it back to the original character, and repeat the decoding process on the remainder of the encoded file. The decoding process needs a convenient representation for the prefix code so that the initial codeword can be easily picked off. A binary tree whose leaves are the given characters provides one such representation. We interpret the binary codeword for a character as the path from the root to that character, where 0 means "go to the left child", and 1 mean "go to the right child." Note that these are not binary search trees, since the leaves need not appear in sorted order and internal nodes do not contain character keys. An optimal code for a file is always represented by a full binary tree, in which every nonleaf node has two children.

Example – According to above data :



(a) (b)

Each leaf is labeled with a character and its frequency of occurrence. Each internal node is labeled with the sum of the frequencies of the leaves in its sub-tree.

(a) The tree corresponding to the fixed-length code a = 000, . . . , f = 101.

(b) The tree corresponding to the optimal prefix code a = 0, b = 101, . . . , f = 1100.

Constructing a Huffman code – Huffman invented a greedy algorithm that constructs an optimal prefix code called a Huffman code.

HUFFMAN(C)

1. $n \leftarrow |C|$

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

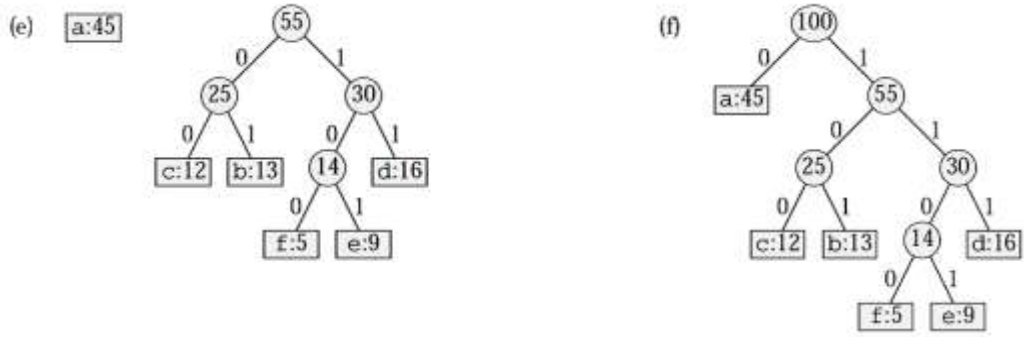
2. Q ← C
3. for i ← 1 to n − 1
4. do allocate a new node z
5. left[z]← x ← EXTRACT-MIN(Q)
6. right[z]← y ← EXTRACT-MIN(Q)
7. f [z]← f [x] + f [y]
8. INSERT(Q, z)
9. return EXTRACT-MIN(Q)

Example – For our example, there are 6 letters in the alphabet, the initial queue size is n = 6, and 5 merge steps are required to build the tree. The final tree represents the optimal prefix code. The codeword for a letter is the sequence of edge labels on the path from the root to the letter.

The steps of Huffman's algorithm for the frequencies –
Each part shows the contents of the queue sorted into increasing order by frequency. At each step, the two trees with lowest frequencies are merged. Leaves are shown as rectangles containing a character and its frequency. Internal nodes are shown as circles containing the sum of the frequencies of its children. An edge connecting an internal node with its children is labeled 0 if it is an edge to a left child and 1 if it is an edge to a right child. The codeword for a letter is the sequence of labels on the edges connecting the root to the leaf for that letter.

(a) The initial set of n = 6 nodes, one for each letter.
(b)–(e) Intermediate stages.
(f) The final tree.

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.



In short - using Huffman codes –

- Each message has a different tree. The tree must be saved with the message.
- Huffman codes are effective for long files where the savings in the message can offset the cost for storing the tree.
- Decode files by starting at root and proceeding down the tree according to the bits in the message (0 = left, 1 = right). When a leaf is encountered, output the character at that
- Huffman codes are also effective when the tree can be pre-computed and used for a large number of messages (e.g., a tree based on the frequency of occurrence of characters in the English language).
- Huffman codes are not very good for random files (each character about the same frequency).

Dynamic programming –

Dynamic programming is a problem solving technique that, like Divide and Conquer, solves problems by dividing them into sub - problems. Dynamic programming is used when the sub - problems are not independent, e.g. when they share the same sub-problems. In this case, divide and conquer may do more work than necessary, because it solves the same sub-problem multiple times.

Dynamic Programming solves each sub-problem once and stores the result in a table so that it can be rapidly retrieved if needed again. It is often used in Optimization Problems: A problem with many possible solutions for which we want to find an optimal (the best) solution. (There may be more than 1 optimal solution). Dynamic Programming is an approach developed to solve sequential, or multi-stage, decision problems; hence, the name “dynamic” programming. But, as we shall see, this approach is equally applicable for decision problems where sequential property is induced solely for computational convenience.

Dynamic programming can be thought of as being the reverse of recursion. Recursion is a top - down mechanism - we take a problem, split it up, and solve the smaller problems that are created.

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

Dynamic programming is a bottom-up mechanism - we solve all possible small problems and then combine them to obtain solutions for bigger problems.

Common Characteristic of Dynamic Programming Problems –

1. The problem can be divided into stages with a decision required at each stage.
2. Each stage has a number of states associated with it.
3. The decision at one stage transforms one slate into state in the next stage.
4. Given the current state, the optimal decision for each of the remaining states does not depend on the previous state or decisions.
5. There exists a recursive relationship that identifies the optimal decision for stage j, given that stage (j + 1) has already been solved.
6. The final stage must be solvable by itself.

Difference between dynamic programming and divide and conquer –

- Divide and conquer algorithms partition the problem into independent sub-problems, solve the sub-problems recursively and then combine their solutions to solve the original problem. In contrast dynamic programming is applicable when sub-problems are not independent, i.e. when sub-problems share sub sub-problems.
- Thus, a divide and conquer does more work than necessary, repeatedly solving the common sub-problems. A dynamic programming algorithm solves every sub-problem just once and then saves this answer in a table, thereby avoiding the work of re-computing the answer every time the sub sub-problem is encountered.

The development of a dynamic programming algorithm –

1. Characterize the structure of an optimal solution.
2. Recursively define the value of an optimal solution.
3. Compute the value of an optimal solution in a bottom-up fashion.
4. Construct an optimal solution from computed information.

Shortest – paths problem –

Single source shortest-paths problem –

Find a shortest path from a given source vertex $s \in V$ to each$v \in V$.

Single destination shortest-paths problem –

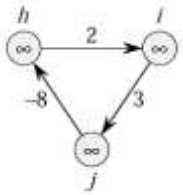
Find a shortest path to a given destination vertex t from each vertex v.

Single pair shortest-path problem –

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

Find a shortest path from u to v for given vertices u and v.

Negative-weight edges – If the graph G = (V, E) contains no negative weight cycles reachable from the source s, then for all v ∈ V, the shortest-path weight δ(s, v) remains well defined, even if it has a negative value.



If there is a negative-weight cycle reachable from s, however, shortest-path weights are not well defined. No path from s to a vertex on the cycle can be a shortest path.

The Bellman-Ford algorithm, allow negative-weight edges in the input graph and produce a correct answer as long as no negative-weight cycles are reachable from the source.

Cycles – A shortest path cannot contain a negative-weight cycle. Nor can it contain a positive-weight cycle, since removing the cycle from the path produces a path with the same source and destination vertices and a lower path weight.

Relaxation – The process of relaxing1 an edge (u, v) consists of testing whether we can improve the shortest path.

INITIALIZE-SINGLE-SOURCE(G, s)

1. for each vertex v ∈ V[G]
2. do d[v]←∞
3. π[v]← NIL
4. d[s] ← 0

RELAX(u, v,w)

1. if d[v] > d[u] + w(u, v)
2. then d[v] ← d[u] + w(u, v)
3. π[v]← u

The Bellman-Ford algorithm –

The Bellman-Ford algorithm solves the single-source shortest-paths problem in the general case in which edge weights may be negative. The Bellman-Ford algorithm returns a Boolean value

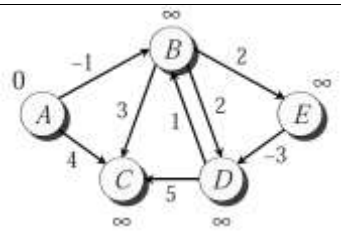
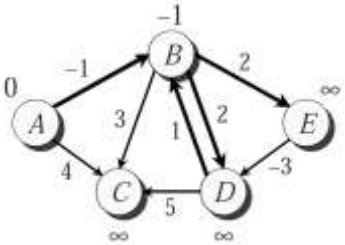
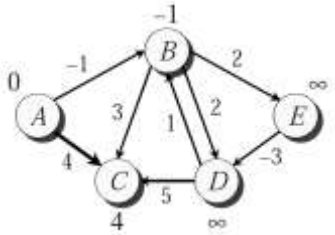
By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

indicating whether or not there is a negative-weight cycle that is reachable from the source. If there is such a cycle, the algorithm indicates that no solution exists. If there is no such cycle, the algorithm produces the shortest paths and their weights. The algorithm uses relaxation. The algorithm returns TRUE if and only if the graph contains no negative-weight cycles that are reachable from the source.

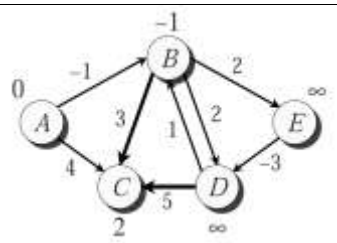
BELLMAN-FORD(G,w, s)

1. INITIALIZE-SINGLE-SOURCE(G, s)
2. for i ← 1 to |V[G]| − 1
3. do for each edge (u, v) ∈ E[G]
4. do RELAX(u, v,w)
5. for each edge (u, v) ∈ E[G]
6. do if d[v] > d[u] + w(u, v)
7. then return FALSE
8. return TRUE

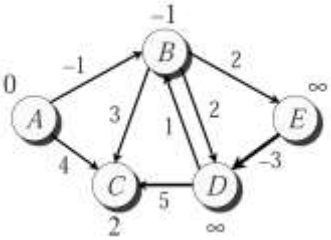
Example – Order of edges: (B,E), (D,B), (B,D), (A,B), (A,C), (D,C), (B,C), (E,D)

Graph	Table
	A B C D E 0 ∞ ∞ ∞ ∞
	A B C D E 0 ∞ ∞ ∞ ∞ 0 −1 ∞ ∞ ∞
	A B C D E 0 ∞ ∞ ∞ ∞ 0 −1 ∞ ∞ ∞ 0 −1 4 ∞ ∞

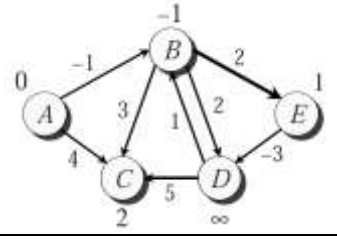
By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.



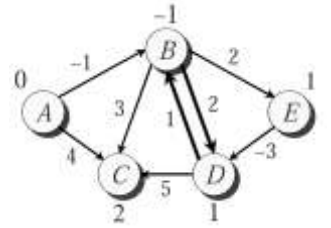
A	B	C	D	E
0	∞	∞	∞	∞
0	−1	∞	∞	∞
0	−1	4	∞	∞
0	−1	2	∞	∞



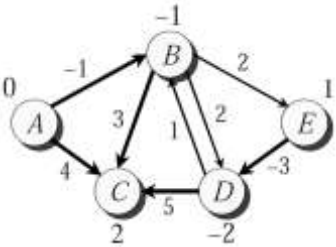
A	B	C	D	E
0	∞	∞	∞	∞
0	−1	∞	∞	∞
0	−1	4	∞	∞
0	−1	2	∞	∞



A	B	C	D	E
0	∞	∞	∞	∞
0	−1	∞	∞	∞
0	−1	4	∞	∞
0	−1	2	∞	∞
0	−1	2	∞	1

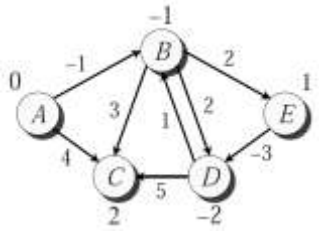


A	B	C	D	E
0	∞	∞	∞	∞
0	−1	∞	∞	∞
0	−1	4	∞	∞
0	−1	2	∞	∞
0	−1	2	∞	1
0	−1	2	1	1



A	B	C	D	E
0	∞	∞	∞	∞
0	−1	∞	∞	∞
0	−1	4	∞	∞
0	−1	2	∞	∞
0	−1	2	∞	1
0	−1	2	1	1
0	−1	2	−2	1

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.



A	B	C	D	E
0	∞	∞	∞	∞
0	−1	∞	∞	∞
0	−1	4	∞	∞
0	−1	2	∞	∞
0	−1	2	∞	1
0	−1	2	1	1
0	−1	2	−2	1

Note: Values decrease monotonically.

Dijkstra's algorithm –

Dijkstra's algorithm solves the single-source shortest-paths problem on a weighted, directed graph G = (V, E) for the case in which all edge weights are nonnegative. In this section, therefore, we assume that w(u, v) ≥ 0 for each edge (u, v) ∈ E. Dijkstra's algorithm relaxes edges.

DIJKSTRA(G,w, s)

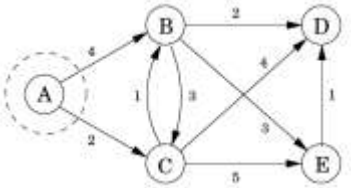
1. INITIALIZE-SINGLE-SOURCE(G, s)
2. S ← ∅
3. Q ← V[G]
4. while Q ≠ ∅
5. do u ← EXTRACT-MIN(Q)
6. S ← S ∪ {u}
7. for each vertex v ∈ Adj[u]
8. do RELAX(u, v,w)

Note –

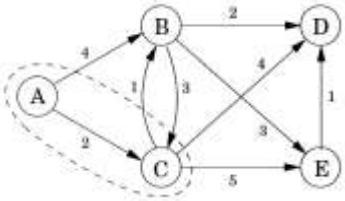
1. Dijkstra's algorithm, run on a weighted, directed graph with nonnegative weight edge.
2. The running time of Dijkstra's algorithm depends on how the min-priority queue is implemented.
3. The running time of the Dijkstra's algorithm is $O(E.\log_2 V)$.
4. Running time of this algorithm is lower than The Bellman-Ford algorithm.

Example – A complete run of Dijkstra's algorithm, with node 'A' as the starting point. Also shown are the associated 'dist' values and the final shortest-path tree.

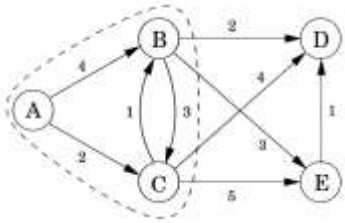
By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.



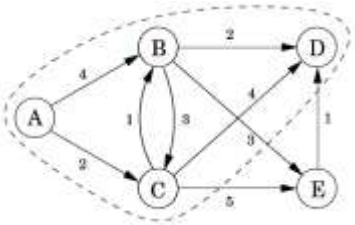
A: 0	D: ∞
B: 4	E: ∞
C: 2	



A: 0	D: 6
B: 3	E: 7
C: 2	

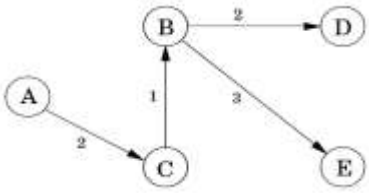


A: 0	D: 5
B: 3	E: 6
C: 2	



A: 0	D: 5
B: 3	E: 6
C: 2	

Final path -



The Floyd - Warshall algorithm –

Floyd's algorithm is best employed on an adjacency matrix data structure, which is no extravagance since we must store all n^2 pair - wise distances anyway.

Advantages :

- Much easier to code.
- You get more. All pairs of shortest paths are solved.

Disadvantages :

- Slower. O(V^3).
- Harder to understand.

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

Let $d_{ij}^{(k)}$ be the weight of a shortest path from vertex 'i' to vertex 'j' for which all intermediate vertices are in the set {1, 2, . . . , k}. When k = 0, a path from vertex 'i' to vertex 'j' with no intermediate vertex numbered higher than 0 has no intermediate vertices at all. Such a path has at most one edge, and hence $d_{ij}^{(0)} = w_{ij}$. A recursive definition following the above discussion is given by –

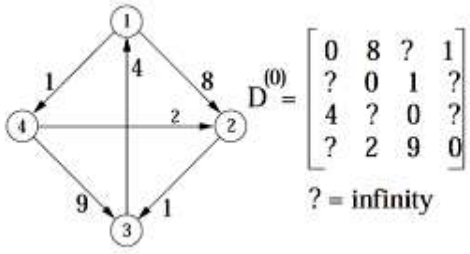
$$d_{ij}^{(k)} = \begin{cases} w_{ij} & \text{if } k = 0 , \\ \min\left(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\right) & \text{if } k \geq 1 . \end{cases}$$

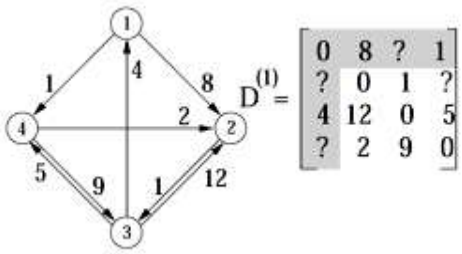
Because for any path, all intermediate vertices are in the set {1, 2, . . . , n}, the matrix $D^{(n)} = \left(d_{ij}^{(n)}\right)$ gives the final answer: $d_{ij}^{(n)} = \delta(i, j)$ for all i, j ∈ V.

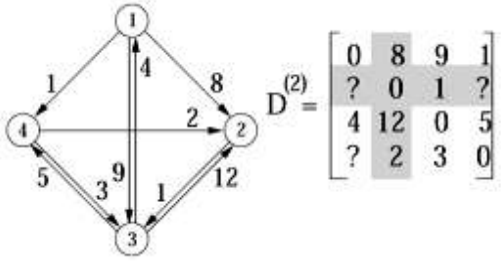
FLOYD-WARSHALL(W)

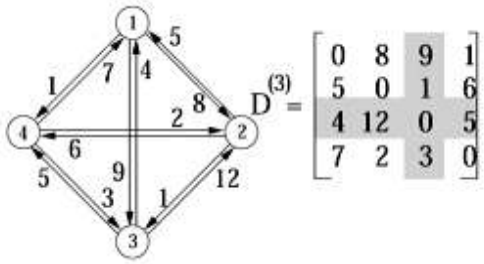
1. n ← rows[W]
2. $D^{(0)}$ ← W
3. for k ← 1 to n
4. do for i ← 1 to n
5. do for j ← 1 to n
6. do $d_{ij}^{(k)} \leftarrow \min\left(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\right)$
7. return $D^{(n)}$

Example –

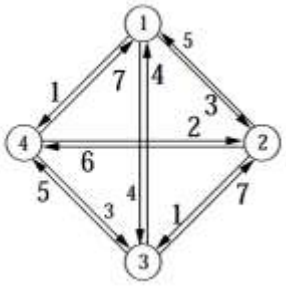








By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.



$$D^{(4)} = \begin{bmatrix} 0 & 3 & 4 & 1 \\ 5 & 0 & 1 & 6 \\ 4 & 7 & 0 & 5 \\ 7 & 2 & 3 & 0 \end{bmatrix}$$

Longest Common Subsequence (LCS) –

The Longest Common Subsequence (LCS) problem is as follows:

A common subequence of two strings is a subsequence that appears in both strings. A longest common subequence is a common subsequence of maximal length.

OR -

We are given two strings: string S of length n, and string T of length m. Our goal is to produce their longest common subsequence: the longest sequence of characters those appear left-to-right (but not necessarily in a contiguous block) in both strings.

Example – string - ACTTGCG

- ACT , ATTC , T , ACTTGC are all subsequences.
- TTA is not a subequence.

In the longest-common-subsequence problem, we are given two sequences $X = (x_1, x_2, \ldots , x_m)$ and $Y = (y_1, y_2, \ldots , y_n)$ and wish to find a maximum-length common subsequence of X and Y . The LCS problem can be solved efficiently using dynamic programming.

Example – S1 = AAACCGTGAGTTATTCGTTCTAGAA
S2 = CACCCCTAAGGTACCTTTGGTTC
LCS is - ACCTAGTACTTTG

Characterizing a longest common subsequence –

A brute-force approach to solving the LCS problem is to enumerate all subsequences of X and check each subsequence to see if it is also a subsequence of Y , keeping track of the longest subsequence found. Each subsequence of X corresponds to a subset of the indices {1, 2, . . . ,m} of X. There are 2^m subsequences of X, so this approach requires exponential time. The LCS problem has an optimal-substructure property.

given a sequence $X = (x_1, x_2, \ldots , x_m)$, we define the i^{th} prefix of X, for $i = 0, 1, \ldots ,m$, as $X_i = (x_1, x_2, \ldots , x_i)$. For example, if X = (A, B, C, B, D, A, B) , then X_4 = (A, B, C, B) and X_0 is the empty sequence.

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

Theorem - Optimal substructure of an LCS –

Let $X = (x_1, x_2, \ldots, x_m)$ and $Y = (y_1, y_2, \ldots, y_n)$ be sequences, and let $Z = (z_1, z_2, \ldots, z_k)$ be any LCS of X and Y .

1. If $x_m = y_n$, then $z_k = x_m = y_n$ and Z_{k-1} is an LCS of X_{m-1} and Y_{n-1}.
2. If $x_m \neq y_n$, then $z_k \neq x_m$ implies that Z is an LCS of X_{m-1} and Y.
3. If $x_m \neq y_n$, then $z_k \neq y_n$ implies that Z is an LCS of X and Y_{n-1}.

Computing the length of an LCS –

Procedure LCS-LENGTH takes two sequences $X = (x_1, x_2, \ldots, x_m)$ and $Y = (y_1, y_2, \ldots, y_n)$ as inputs. It stores the c[i, j] values in a table c[0 . . m, 0 . . n] whose entries are computed in row-major order. It also maintains the table b[1 . . m, 1 . . n] to simplify construction of an optimal solution. Intuitively, b[i, j] points to the table entry corresponding to the optimal subproblem solution chosen when computing c[i, j]. The procedure returns the b and c tables; c[m, n] contains the length of an LCS of X and Y .

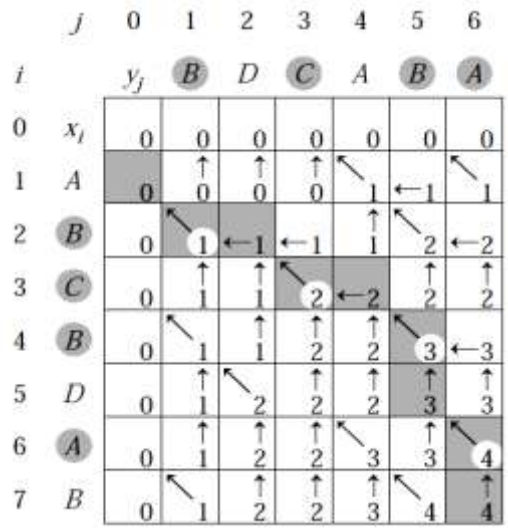
LCS-LENGTH(X, Y)

```
1.  m ← length[X]
2.  n ← length[Y ]
3.  for i ← 1 to m
4.       do c[i, 0] ← 0
5.  for j ← 0 to n
6.       do c[0, j ] ← 0
7.  for i ← 1 to m
8.       do for j ← 1 to n
9.            do if x_i = y_j
10.                 then c[i, j ] ← c[i − 1, j − 1] + 1
11.                      b[i, j ] ← "↖"
12.                 else if c[i − 1, j ] ≥ c[i, j − 1]
13.                      then c[i, j ] ← c[i − 1, j ]
14.                           b[i, j ] ← "↑"
15.                      else c[i, j ] ← c[i, j − 1]
16.                           b[i, j ] ← "←"
17. return c and b
```

Example – The c and b tables computed by LCS-LENGTH on the sequences X = (A, B, C, B, D, A, B) and Y = (B, D, C, A, B, A). The square in row i and column j contains the value of c[i, j] and the appropriate arrow for the value of b[i, j]. The entry 4 in c[7, 6]—the lower right-hand corner of the table—is

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

the length of an LCS (B, C, B, A) of X and Y. For i, j > 0, entry c[i, j] depends only on whether $x_i = y_j$ and the values in entries c[i − 1, j], c[i, j − 1], and c[i − 1, j − 1], which are computed before c[i, j]. To reconstruct the elements of an LCS, follow the b[i, j] arrows from the lower right-hand corner; the path is shaded. Each "↖" on the path corresponds to an entry (highlighted) for which $x_i = y_j$ is a member of an LCS.



		j	0	1	2	3	4	5	6
i			y_j	B	D	C	A	B	A
0	x_i		0	0	0	0	0	0	0
1	A		0	↑ 0	↑ 0	↑ 0	↖ 1	← 1	↖ 1
2	B		0	↖ 1	← 1	← 1	↑ 1	↖ 2	← 2
3	C		0	↑ 1	↑ 1	↖ 2	← 2	↑ 2	↑ 2
4	B		0	↖ 1	↑ 1	↑ 2	↑ 2	↖ 3	← 3
5	D		0	↑ 1	↖ 2	↑ 2	↑ 2	↑ 3	↑ 3
6	A		0	↑ 1	↑ 2	↑ 2	↖ 3	↑ 3	↖ 4
7	B		0	↖ 1	↑ 2	↑ 2	↑ 3	↖ 4	↑ 4

Constructing an LCS –

The b table returned by LCS-LENGTH can be used to quickly construct an LCS of $X=(x_1, x_2, \ldots, x_m)$ and $Y = (y_1, y_2, \ldots, y_n)$. We simply begin at b[m, n] and trace through the table following the arrows. Whenever we encounter a "↖" in entry b[i, j], it implies that $x_i = y_j$ is an element of the LCS. The elements of the LCS are encountered in reverse order by this method. The following recursive procedure prints out an LCS of X and Y in the proper, forward order. The initial invocation is PRINT-LCS(b, X, length[X], length[Y]).

PRINT-LCS(b, X, i, j)

1. if i = 0 or j = 0
2. then return
3. if b[i, j] = "↖"
4. then PRINT-LCS(b, X, i − 1, j − 1)
5. print x_i
6. elseif b[i, j] = "↑"
7. then PRINT-LCS(b, X, i − 1, j)
8. else PRINT-LCS(b, X, i, j − 1)

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

For the b table this procedure prints "BCBA." The procedure takes time O(m + n), since at least one of i and j is decremented in each stage of the recursion.

Graph algorithms –

- A **graph** is a collection of vertices (V) and edges (E). An edge connects two vertices.
- A simple path is a path with no vertex repeated.
- The distance between two nodes is the length of the shortest path between them.
- A simple cycle is a simple path except the first and last vertex is repeated and, for an undirected graph, number of vertices $> = 3$.
- A tree is a graph with no cycles.
- A complete graph is a graph in which all edges are present.
- A sparse graph is a graph with relatively few edges.
- A dense graph is a graph with many edges.
- An undirected graph has no specific direction between the vertices.
- A directed graph has edges that are 'one way'. We can go from one vertex to another, but not vice versa.
- A weighted graph has weight associated with each edge. The weights can represent distances, costs etc.

Representations of graphs –

There are two standard ways to represent a graph G = (V, E):

- as a collection of adjacency lists
- as an adjacency matrix.

- An **adjacency list** is an array which contains a list for each vertex. The list for a given vertex contains all the other vertices that are connected to the first vertex by a single edge.
- An **adjacency matrix** is a matrix with a row and column for each vertex. The matrix entry is 1 if there is an edge between the row vertex and the column vertex. The diagonal may be zero or one. Each edge is represented twice.

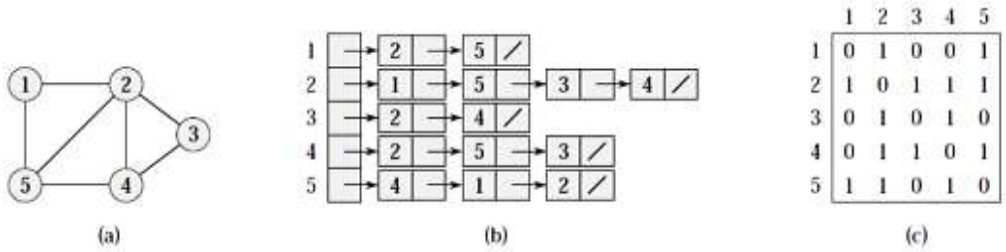
Example - Two representations of an undirected graph.

(a) An undirected graph G having five vertices and seven edges.

(b) An adjacency-list representation of G.

(c) The adjacency-matrix representation of G.

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

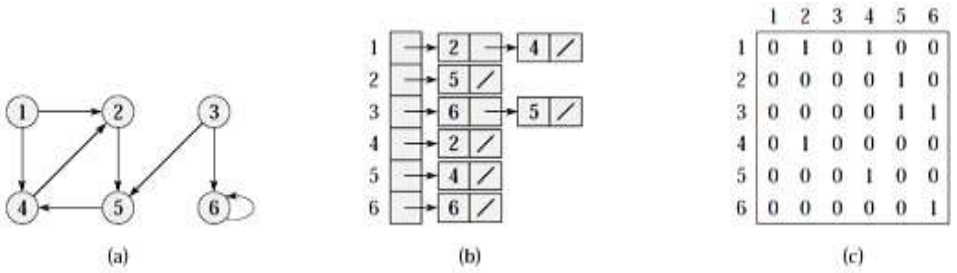


Example - Two representations of a directed graph.

(a) A directed graph G having six vertices and eight edges.

(b) An adjacency-list representation of G.

(c) The adjacency-matrix representation of G.



Backtracking – Backtracking is a general algorithm technique that considers searching every possible combination in order to solve an optimization problem.

Backtracking is also known as depth first search (or) branch and bound. Backtracking is an important tool for solving constraint satisfaction problems, such as crosswords, verbal arithmetic, Sudoku and many other puzzles. It is often the more convenient technique for parsing, for the knapsack problem and other combinational optimization problems.

The advantage of backtracking algorithm is that they are complete, that is they are guaranteed to find every solution to every possible puzzle.

Graph Traversal – To traverse a graph is to process every node in the graph exactly once, because there are many paths leading from one node to another, the hardest part about traversing a graph is making sure that you do not process some node twice.

General solutions for this difficulty -

1. When you first encounter a node, mark it as REACHED. When you visit a node, check if its marked REACHED, if it is, just ignore it. This is the method our algorithms will use.
2. When you process a node, delete it from the graph. Deleting the node causes the deletion of all the arcs that lead to the node, so it will be impossible to reach more than once.

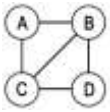
General traversal strategy –

1. Mark all nodes in the graph as NOT REACHED.

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

2. Pick a starting node mark it as REACHED, and place it on the READY list.
3. Pick a node on the READY list. Process it remove it from READY. Find all its neighbours; Those that are NOT REACHED should marked as REACHED and added to READY.
4. Repeat 3 until READY is empty.

Example - Consider graph -



Step 1: A = B = C = D = NOT REACHED
Step 2: READY = {A} . A = REACHED
Step 3: Process A. READY = {B, C}. B = C = REACHED
Step 3: Process C . READY = {B, D} . D = REACHED
Step 3: Process B. READY = {D}
Step 3: Process D . READY = { }

There are 2 common elementary algorithms for tree searching are –

1. Breadth-first search (BFS)
2. Depth-first search (DFS)

Both of these algorithms work on directed or undirected graphs. Many advanced graph algorithms are based on the ideas of BFS or DFS. Each of these algorithms traverses edges in the graph, discovering new vertices as it proceeds. The difference is in the order in which each algorithm discovers the edge.

BREADTH FIRST SEARCH –

Breadth first search trees have a nice property: Every edge of G can be classified into one of three groups. Some edges are in T themselves. Some connect two vertices at the same level of T. And the remaining ones connect two vertices on two adjacent levels. It is not possible for an edge to skip a level. Therefore, the breadth first search tree really is a shortest path tree starting from its root. Every vertex has a path to the root, with path length equal to its level (just follow the tree itself), and no path can skip a level so this really is a shortest path.

Procedure –

First, the starting vertex is enqueued.
Then, the following steps are repeated until the queue is empty.

1. Remove the vertex at the head of the queue and call it vertex.

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

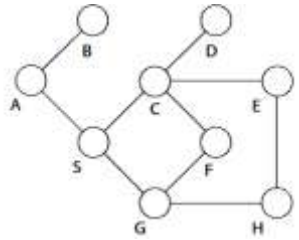
2. visit vertex
3. Follow each edge emanating from vertex to find the adjacent vertex and call it 'to' If 'to' has not already been put into the queue, enqueue it.

Notice, that a vertex can be put into the queue at most once. Therefore, the algorithm must somehow keep track of the vertices that have been enqueued.

BFS(G, s)

```
1.  for each vertex u ∈ V[G] − {s}
2.        do color[u] ← WHITE
3.              d[u]←∞
4.              π[u]← NIL
5.  color[s] ← GRAY
6.  d[s] ← 0
7.  π[s] ← NIL
8.  Q ← ∅
9.  ENQUEUE(Q, s)
10. while Q = ∅
11.       do u ← DEQUEUE(Q)
12.             for each v ∈ Adj[u]
13.                   do if color[v] = WHITE
14.                         then color[v] ← GRAY
15.                              d[v] ← d[u] + 1
16.                              π[v]← u
17.                              ENQUEUE(Q, v)
18.                   Dequeue(Q)
19.                   color[u] ← BLACK
```

Example - Take S to be the source vertex.

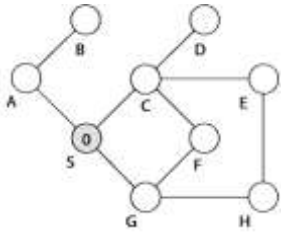


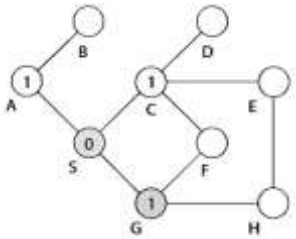
Adjacancy Lists:

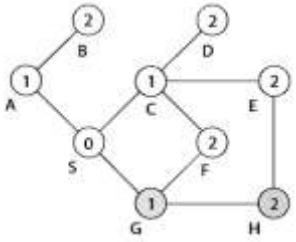
S = {A,C,G}

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

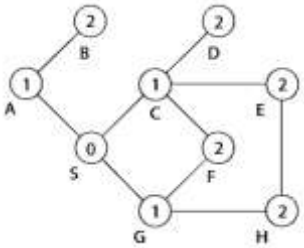
A = {B, S}
B = {A}
C = {D,E, F, S}
D = {C}
E = {C,H}
F = {C,G}
G = {F,H,S}
H = {E,G}







Finally – we get –



v	w	Action	Queue
–	–	Start	{S}
S	A	set d(A) = 1	{A}
S	C	set d(C) = 1	{A,C}
S	G	set d(G) = 1	{A,C,G}
A	B	set d(B) = 2	{C,G,B}
A	S	none, as d(S) = 0	{C,G,B}
C	D	set d(D) = 2	{G,B,D}
C	E	set d(E) = 2	{G,B,D,E}
C	F	set d(F) = 2	{G,B,D,E, F}
C	S	none	{G,B,D,E, F}
G	F	none	{B,D,E, F}
G	H	set d(H) = 2	{B,D,E, F,H}
G	S	none	{B,D,E, F,H}
B	A	none	{D,E,F,H}

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

D	C	none	{E, F,H}
E	C	none	{F,H}
E	H	none	{F,H}
F	C	none	{H}
F	G	none	{H}
H	E	none	{ }
H	G	none	{ }

Analysis – The **while-loop** in breadth-first search is executed at most |V| times. The reason is that every vertex enqueued at most once. So, we have O(V). The **for-loop** inside the while-loop is executed at most |E| times if G is a directed graph or 2|E| times if G is undirected. The reason is that every vertex dequeued at most once and we examine (u, v) only when u is dequeued. Therefore, every edge examined at most once if directed, at most twice if undirected. So, we have O(E). Therefore, the total running time for breadth-first search traversal is O(V + E).

DEPTH-FIRST SEARCH –

Depth-first search is a systematic way to find all the vertices reachable from a source vertex. In depth-first search, edges are explored out of the most recently discovered vertex v that still has unexplored edges leaving it. When all of v's edges have been explored, the search "backtracks" to explore edges leaving the vertex from which v was discovered. This process continues until we have discovered all the vertices that are reachable from the original source vertex. If any undiscovered vertices remain, then one of them is selected as a new source and the search is repeated from that source. This entire process is repeated until all vertices are discovered.

As in breadth-first search, whenever a vertex v is discovered during a scan of the adjacency list of an already discovered vertex u, depth-first search records this event by setting v's predecessor field π[v] to u. Unlike breadth-first search, whose predecessor subgraph forms a tree, the predecessor subgraph produced by a depth-first search may be composed of several trees, because the search may be repeated from multiple sources.

Each vertex v has two timestamps: the first timestamp d[v] records when v is first discovered (and grayed), and the second timestamp f [v] records when the search finishes examining v's adjacency list (and blackens v). For every vertex u, $d[u] < f[u]$.

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

DFS (V, E)

```
1.  for each vertex u in V [G]
2.    do color[u] ← WHITE
3.          π[u] ← NIL
4.  time ← 0
5.  for each vertex u in V[G]
6.    do if color[u] ← WHITE
7.          then DFS-Visit(u)
```

DFS-Visit(u)

```
1.  color[u] ← GRAY                ▷ discover u
2.  time ← time + 1
3.  d[u] ← time
4.  for each vertex v adjacent to u    ▷ explore (u, v)
5.    do if color[v] ← WHITE
6.          then π[v] ← u
7.               DFS-Visit(v)
8.  color[u] ← BLACK
9.  time ← time + 1
10. f[u] ← time
```

Analysis – The analysis is similar to that of BFS analysis. The DFS-Visit is called (from DFS or from itself) once for each vertex in V[G] since each vertex is changed from white to gray once. The **for-loop** in DFS-Visit is executed a total of |E| times for a directed graph or 2|E| times for an undirected graph since each edge is explored once. Moreover, initialization takes $\Theta(|V|)$ time. Therefore, the running time of DFS is $\Theta(V + E)$.

Consider vertex u and vertex v in V[G] after a DFS. Suppose vertex v in some DFS-tree. Then we have $d[u] < d[v] < f[v] < f[u]$ because of the following reasons:

a. Vertex u was discovered before vertex v; and
b. Vertex v was fully explored before vertex u was fully explored.

Note that converse also holds: if $d[u] < d[v] < f[v] < f[u]$ then vertex v is in the same DFS-tree and a vertex v is a descendent of vertex u.

Suppose vertex u and vertex v are in different DFS-trees or suppose vertex u and vertex v are in the same DFS-tree but neither vertex is the descendent of the other. Then one vertex was discovered and fully explored before the other was discovered i.e., $f[u] < d[v]$ or $f[v] < d[u]$.

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

Parenthesis Theorem - For all u, v, exactly one of the following holds:

1. $d[u] < f[u] < d[v] < f[v]$ or $d[v] < f[v] < d[u] < f[u]$ and neither of u and v is a descendant of the other.
2. $d[u] < d[v] < f[v] < f[u]$ and v is a descendant of u.
3. $d[v] < d[u] < f[u] < f[v]$ and u is a descendant of v.

So, $d[u] < d[v] < f[u] < f[v]$ cannot happen.

Like parentheses: ( ) [], ([ ]), and [( )] are OK but ([ )] and [( ]) are not OK.

Suppose G be an undirected graph, then we have following edge classification:

1. **Tree Edge** - An edge connects a vertex with its parent.
2. **Back Edge** - A non-tree edge connects a vertex with an ancestor..
3. **Forward Edge** - There is no forward edges because they become back edges when considered in the opposite direction.
4. **Cross Edge** - There cannot be any cross edge because every edge of G must connect an ancestor with a descendant.

Example – Consider graph (a) -

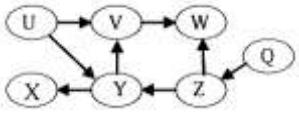
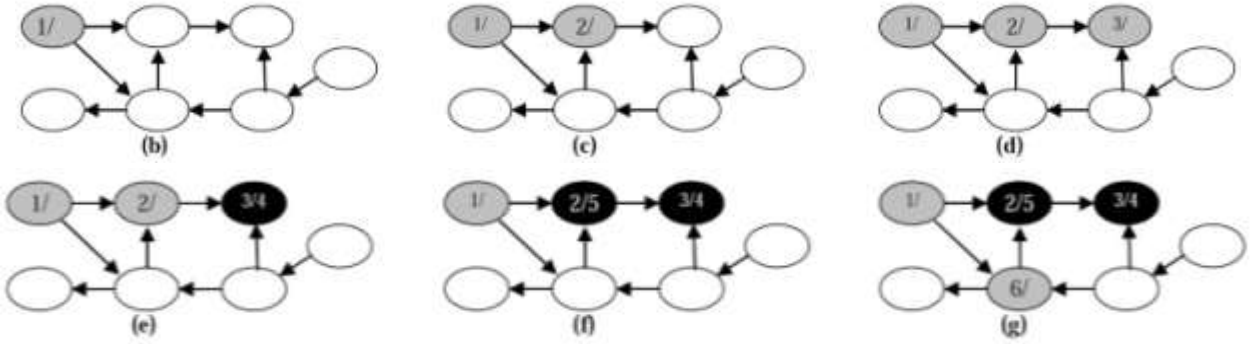
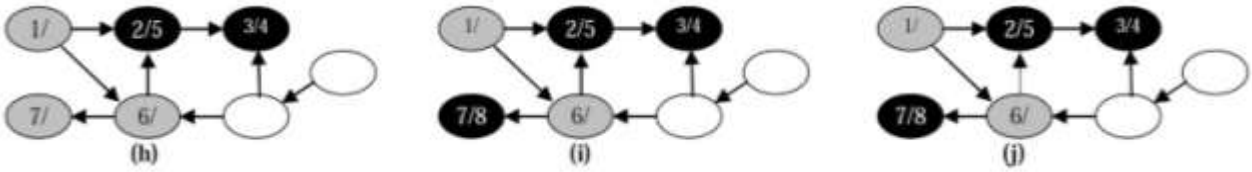


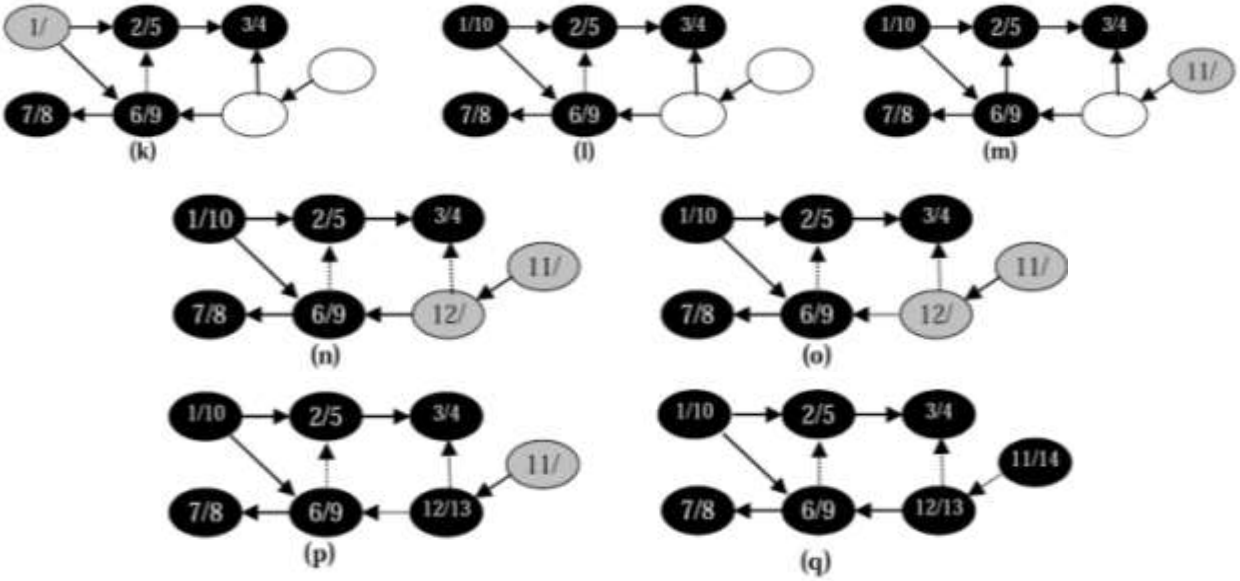
Figure (b) shows the progress of the DFS algorithm for the graph of (a). Starting from node U we can either discover node V or Y . Suppose that we discover node V , which has a single outgoing edge to node W. W has no outgoing edges, so this node is finished, and we return to V . From V there is no other choice, so this node is also finished and we return to U. From node U we can continue to discover Y and its descendants, and the procedure continues similarly.



By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.



At stage (l) we have discovered and finished nodes U, V , W, X, Y . Selecting node Q as a new starting node, we can discover the remaining nodes (in this case Z).



Topological sort –

Depth-first search can be used to perform a topological sort of a directed acyclic graph (also called a "dag"). A topological sort of a dag G = (V, E) is a linear ordering of all its vertices such that if G contains an edge (u, v), then u appears before v in the ordering.

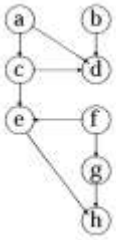
TOPOLOGICAL-SORT(G)

1. call DFS(G) to compute finishing times f [v] for each vertex v
2. as each vertex is finished, insert it onto the front of a linked list
3. return the linked list of vertices.

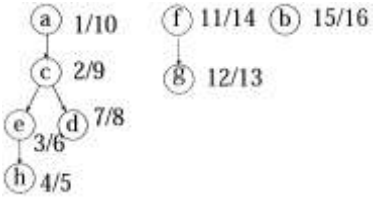
We can perform a topological sort in time $\theta(V + E)$, since depth-first search (DFS) takes $\theta(V + E)$ time and it takes O(1) time to insert each of the |V| vertices onto the front of the linked list. Topologically sorted dag as an ordering of vertices along a horizontal line such that all directed edges go from left to right.

Example – Consider graph –

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

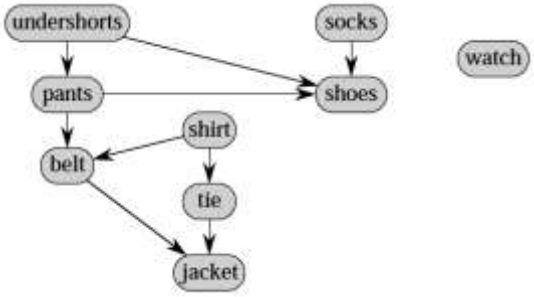


Solution – After DFS traversal –

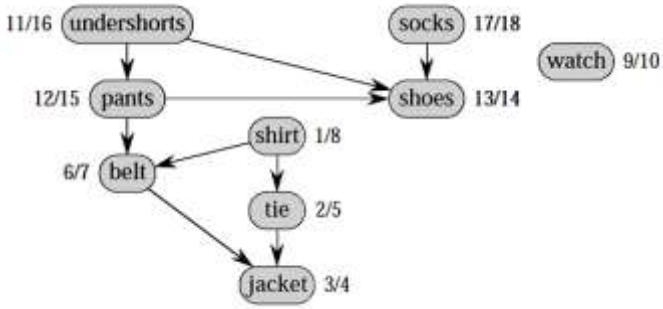


Final Topological order – (b , f , g , a , c , d , e , h)

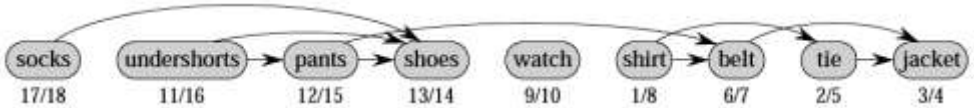
Example – Consider graph –



Solution - Applying - DFS

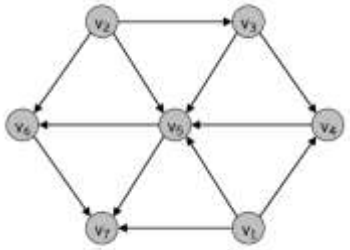


Topological sort –

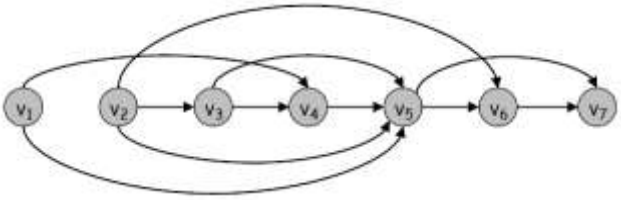


Example – Consider graph –

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.



Topological graph –



Strongly connected components –

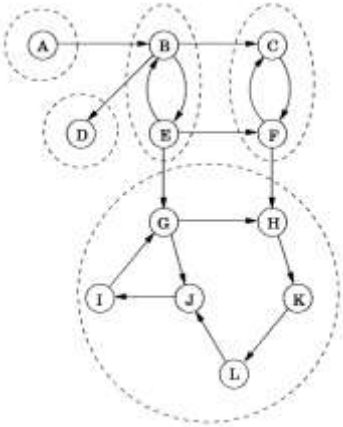
Every directed graph is a dag of its strongly connected components. Two nodes u and v of a directed graph are connected if there is a path from u to v and a path from v to u.
A directed graph is called strongly connected if there is a path in each direction between each pair of vertices of the graph. In a directed graph G that may not itself be strongly connected, a pair of vertices u and v are said to be strongly connected to each other if there is a path in each direction between them.

STRONGLY-CONNECTED-COMPONENTS(G)

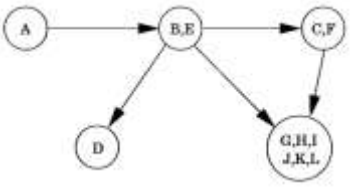
1. call DFS(G) to compute finishing times f [u] for each vertex u
2. compute GT
3. call DFS(GT), but in the main loop of DFS, consider the vertices in order of decreasing f [u] (as computed in line 1)
4. Output the vertices of each tree in the depth-first forest formed in line 3 as a separate strongly connected component.

Example – A directed graph and its strongly connected components–

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.



The meta-graph –



Dictionary (Hash Tables) –

Dictionary - In a dictionary, we separate the data into two parts. Each item stored in a dictionary is represented by a key/value pair. The key is used to access the item. With the key you can access the value, which typically has more information.

A dictionary stores key element pairs (k , e) which we call items.

where, k = key

e = element

There are two types of dictionaries –

1. Unordered Dictionary
2. Ordered Dictionary

- The dictionary ADT models a searchable collection of key element items.
- The main operations of a dictionary are searching, inserting, and deleting items.
- Multiple items with the same key are allowed.
- Dictionary ADT methods:
- **find(k)**: if the dictionary has an item with key k, returns the position of this element, else, returns a null position.
- **insertItem(k , o)**: inserts item (k, o) into the dictionary
- **removeElement(k)**: if the dictionary has an item with key k, removes it from the dictionary and returns its element. An error occurs if there is no such element.

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

- **size( ).**
- **isEmpty( ).**
- **keys( )** : return an iterator of the keys in M.
- **Elements( )** : return an iterator of the values in M.

Log File – A log file is a dictionary implemented by means of an unsorted sequence. We store the items of the dictionary in a sequence (based on a doubly-linked lists or a circular array), in arbitrary order.

Performance –

insertItem takes O(1) time since we can insert the new item at the beginning or at the end of the sequence. **find** and **removeElement** take O(n) time since in the worst case (the item is not found) we traverse the entire sequence to look for an item with the given key.

The log file is effective only for dictionaries of small size or for dictionaries on which insertions are the most common operations, while searches and removals are rarely performed (e.g., historical record of logins to a workstation).

Hash Functions and Hash Tables – A hash function h maps keys of a given type to integers in a fixed interval [0, N - 1].

Example:

$h(x) = x \bmod N$

is a hash function for integer keys

The integer h(x) is called the hash value of key x

A hash table for a given key type consists of -

- Hash function - h
- Array (called table) of size N

- When implementing a map with a hash table, the goal is to store item (k, o) at index $i = h(k)$.
- A hash function is usually specified as the composition of two functions:
  - Hash code map: h1: keys → integers
  - Compression map: h2: integers → $[0, N-1]$
- The hash code map is applied first, and the compression map is applied next on the result, i.e., $h(x) = h2(h1(x))$
- The goal of the hash function is to "disperse" the keys as uniformly as possible.
- There are two types of Hash Tables:
  - Open-addressed Hash Tables
  - Separate-Chained Hash Tables
- **An Open-addressed Hash Table** is a one-dimensional array indexed by integer values that are computed by an index function called a **hash function.**

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

- **A Separate-Chained Hash Table** is a one-dimensional array of linked lists indexed by integer values that are computed by an index function called a **hash function**.
- Hash tables are sometimes referred to as **scatter tables**..
- Typical hash table operations are:
  - Initialization
  - Insertion
  - Searching
  - Deletion

Hash Codes –

Memory address:

- We reinterpret the memory address of the key object as an integer (default hash code of all Java objects).
- Good in general, except for numeric and string keys.

Integer cast:

- We reinterpret the bits of the key as an integer.
- Suitable for keys of length less than or equal to the number of bits of the integer type (e.g., byte, short, int and float in Java).

Component sum:

- We partition the bits of the key into components of fixed length (e.g., 16 or 32 bits) and we sum the components (ignoring overflows).
- Suitable for numeric keys of fixed length greater than or equal to the number of bits of the integer type (e.g., long and double in Java).

Types of Hashing –

There are two types of hashing :

1. **Static hashing**: In static hashing, the hash function maps search-key values to a fixed set of locations.
2. **Dynamic hashing**: In dynamic hashing a hash table can grow to handle more items. The associated hash function must change as the table grows.

- The load factor of a hash table is the ratio of the number of keys in the table to the size of the hash table.

Some Applications of Hash Tables –

- **Database systems**: Specifically, those that require efficient random access. Generally, database systems try to optimize between two types of access methods: sequential and random. Hash tables are an important part of efficient random access because they provide a way to locate data in a constant amount of time.

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

- **Symbol tables**: The tables used by compilers to maintain information about symbols from a program. Compilers access information about symbols frequently. Therefore, it is important that symbol tables be implemented very efficiently.
- **Data dictionaries**: Data structures that support adding, deleting, and searching for data. Although the operations of a hash table and a data dictionary are similar, other data structures may be used to implement data dictionaries. Using a hash table is particularly efficient.
- **Network processing algorithms**: Hash tables are fundamental components of several network processing algorithms and applications, including route lookup, packet classification, and network monitoring.
- **Browser Cashes**: Hash tables are used to implement browser cashes.

Hashing methods –

1. **Division method** – In the division method for creating hash functions, we map a key 'k' into one of 'm' slots by taking the remainder of "k divided by m".
   That is, the hash function is - $h(k) = k \bmod m$.

Example – If the hash table has size, $m = 12$ and
the key, $k = 100$,
then $h(k) = 4$.

Since it requires only a single division operation, hashing by division is quite fast. When using the division method, we usually avoid certain values of 'm'.
For example, m should not be a power of 2, since if $m = 2^p$, then h(k) is just the 'p' lowest-order bits of 'k'.

Disadvantage – A potential disadvantage of the division method is due to the property that consecutive keys map to consecutive hash values -

$h(i)=i$
$h(i + 1) = i + 1 \pmod M$
$h(i \div 2) = i + 2 \pmod M$

While this ensures that consecutive keys do not collide, it does not mean that consecutive array locations will be occupied. We will see that in certain implementations this can lead to degradation in performance.

2. **Multiplication method** – The multiplication method for creating hash functions operates in two steps. First, we multiply the key k by a constant 'A' in the range $0 < A < 1$ and extract the fractional part of 'kA'. Then, we multiply this value by 'm' and take the floor of the result.

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

The hash function is - h(k) =⌊ $m\ (k\ \ A\ mod\ 1)$ ⌋
Where "$k\ A\ mod\ 1$" means the fractional part of 'k A', that is, $k\ A - \ \lfloor kA \rfloor$.

Advantage – The value of m is not critical.
Practical issues –

- Easy to implement.
- On most machines multiplication is faster than division.
- We can substitute one multiplication by shift operation.
- We don't need to do floating - point operations.
- If successive keys have a large interval, A = 0.6125423371 can be recommended.

3. **Mid – square method** – A good hash function to use with integer key values is the mid - square method. The mid square method squares the key value, and then takes out the middle 'r' bits of the result, giving a value in the range 0 to (2^r) - 1. This works well because most (or) all bits of the key value contribute to the result.

Example - Consider records whose keys are 4 - digit numbers in base 10. The goal is to hash these key values to a table of size 100(i.e., a range of 0 to 99). This range is equivalent to two digits in base 10. That is r = 2. If the input is the number 4567, squaring yields an 8 - digit number, 20857489. The middle two digits of this result are 57. All digits of the original key value (equivalently all bits when the number is viewed in binary) contribute to the middle two digits of the squared value. Thus, the result is not dominated by the distribution of the bottom or the top digit of the original key value. Of course, if the key values all tend to be small numbers, then their squares will only affect the low order digits of the hash value.

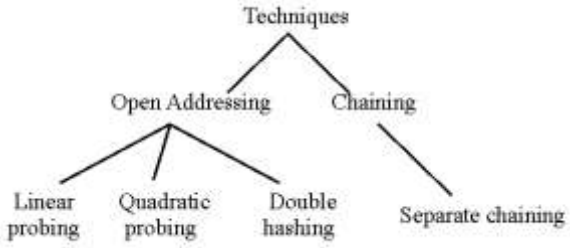
Example -1 : Key = 123 - 45 - 6789
(123456789)^2 = 15241578750790521
h(123 – 45 - 6789) = 8750

Example -2: To map the key 3121 into a hash table of size 1000,
we square it (3121)^2 = 9740641 and extract 406 as the hash value.

Resolving Collisions –

In collision resolution strategy algorithms and data structures are used to handle two hash keys that hash to the same hash keys. There are a number of collision resolution techniques but the most popular are open addressing and chaining.

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

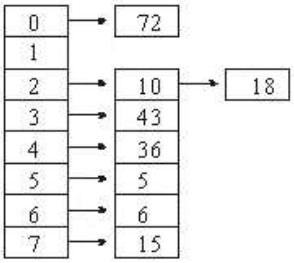


Chaining –

Separated chaining – Every linked list has each element that Collides to the similar slot. Insertion need to locate the accurate slot, and appending to any end of the list in that slot wherever, deletion needs searching the list and removal.

Example – Hash key = key % table size
Insert keys – 36 , 18 , 72 , 43 , 6 , 10 , 5 , 15

Solution - Hash key = 36 % 8 = 4
18 % 8 = 2
72 % 8 = 0
43 % 8 = 3
6 % 8 = 6
10 % 8 = 2
5 % 8 = 5
15 % 8 = 7



Open addressing –

In open addressing, all elements are stored in the hash table itself. That is, each table entry contains either an element of the dynamic set or NIL. When searching for an element, we systematically examine table slots until the desired element is found or it is clear that the element is not in the table. There are no lists and no elements stored outside the table, as there are in

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

chaining. Thus, in open addressing, the hash table can "fill up" so that no further insertions can be made; the load factor α can never exceed 1.

Advantage –It avoids pointers altogether. Instead of following pointers, we compute the sequence of slots to be examined. The extra memory freed by not storing pointers provides the hash table with a larger number of slots for the same amount of memory, potentially yielding fewer collisions and faster retrieval.

To perform insertion using open addressing, we successively examine, or probe, the hash table until we find an empty slot in which to put the key.

HASH-INSERT(T, k)

```
1.  i ← 0
2.  repeat j ← h(k, i )
3.          if T [ j ] = NIL
4.                  then T [ j ]← k
5.                          return j
6.          else i ←i + 1
7.    until i = m
8.  error "hash table overflow"
```

The algorithm for searching for key 'k' probes the same sequence of slots that the insertion algorithm examined when key 'k' was inserted. Therefore, the search can terminate (unsuccessfully) when it finds an empty slot, since 'k' would have been inserted there and not later in its probe sequence. (note - this argument assumes that keys are not deleted from the hash table.) The procedure HASH-SEARCH takes as input a hash table T and a key 'k', returning 'j' if slot 'j' is found to contain key 'k', or NIL if key 'k' is not present in table T .

HASH-SEARCH(T, k)

```
1.  i ← 0
2.  repeat j ← h(k, i )
3.          if T [ j ] = k
4.                  then return j
5.          i ←i + 1
6.     until T [ j ] = NIL or i = m
7.  return NIL
```

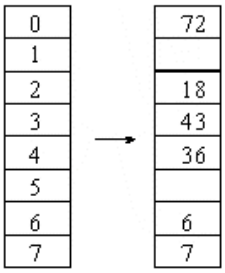

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

Deletion from an open-address hash table is difficult. When we delete a key from slot 'i' , we cannot simply mark that slot as empty by storing NIL in it. Doing so might make it impossible to retrieve any key 'k' during whose insertion we had probed slot 'i' and found it occupied. One solution is to mark the slot by storing in it the special value DELETED instead of NIL. We would then modify the procedure HASH-INSERT to treat such a slot as if it were empty so that a new key can be inserted. No modification of HASH-SEARCH is needed, since it will pass over DELETED values while searching. When we use the special value DELETED, however, search times are no longer dependent on the load factor 'α', and for this reason chaining is more commonly selected as a collision resolution technique when keys must be deleted.

1. Linear probing –Linear probing method is used for resolving hash collisions of values of hash functions by sequentially searching the hash table for a free location. The item will be stored in the next available slot in the table in linear probing also an assumption is made that the table is not already full. This is implemented via a linear search for an empty slot, from the point of collision.

If the physical end of table is reached during the linear search, the search will again get start around to the beginning of the table and continue from there. The table is considered as full, if an empty slot is not found before reaching the point of collision.

Example – Given table – with keys = 72 , 18 , 43 , 36 , 6 , 7



Add keys – 10 , 5 , 15 into above table.

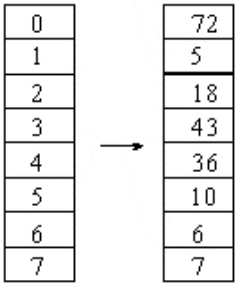
Solution – Hash key = key % table size

Hash key = 10 % 8 = 2

5 % 8 = 5

15 % 8 = 7

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.



Limitation - A problem with linear probe method is primary clustering. In primary clustering block of data may possibly be able to form collision. Several attempts may be required by any key that hashes into the cluster to resolve the collision.

2. Quadratic probing – To resolve the primary clustering problem, quadratic probing can be used. With quadratic probing, rather than always moving one spot, move 'i^2' Spots form the point of collision where 'i' is the number of attempts needed to resolve the collision.

Example – Insert the following keys into table where table size = 10 , keys = 89 , 18 , 49 , 58 , 69

Solution – Hash key = key % table size

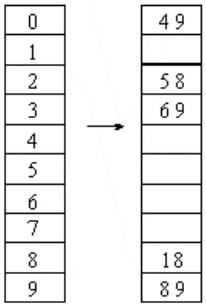
Hash key = 89 % 10 = 9

18 % 10 = 8

49 % 10 = 9 → collision → $(49+1^2)\%10 = 0$

58 % 10 = 8 → collision → $(58+1^2)\%10 = 9$ → collision → $(58+2^2)\%10 = 2$

69 % 10 = 9 → collision → $(69+1^2)\%10 = 0$ → collision → $(69+2^2)\%10 = 3$



Limitation - Maximum half of the table can be used as substitute locations to resolve collisions. Once the table gets more than half full, its really hard to locate an unfilled spot. This new difficulty is recognized as secondary clustering because elements that hash to the same hash key will always probe the identical substitute cells.

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

3. Double hashing – Double hashing uses the idea of applying a second hash function to the key when a collision occurs, the result of the second hash function will be the number of positions from the point of collision to insert. There are some requirements for the second function.

1. It must never evaluate to zero.
2. Must make sure that all cells can be probed.

A popular second hash function is:

Hash key = R (Key mod R) ,

where R is a prime number smaller than the size of the table.

Example - Insert the following keys into table where table size = 10 , keys = 89 , 18 , 49 , 58 , 69

Solution – Hash key1 = key % table size

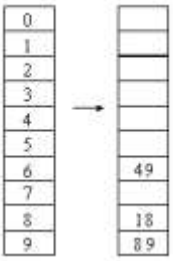
Hash key2 = R – (key % R)

where, R is a prime number less than the table size.

Hash key = 89 % 10 = 9

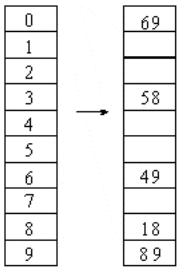
18 % 10 = 8

49 % 10 = 9 → collision → 7 – (49 % 7) = 7 position from 9



58 % 10 = 8 → collision → 7 – (58 % 7) = 5 position from 8

69 % 10 = 9 → collision → 7 – (69 % 7) = 1 position from 9



NP-hard and NP-complete –

A mathematical problem for which, even in theory, no shortcut, or smart algorithm is possible that would lead to a simple or rapid solution. Instead the only way to find an Optimal solution is a

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

computationally intensive, exhaustive analysis in which all possible outcomes are tested. Examples of NP - hard problems include the travelling salesman problem.

P - Problem – A problem is assigned to the P (polynomial time) class if there exists at least one algorithm to solve that problem, such that no. of steps of the algorithm is bounded by a polynomial in 'n', where 'n' is the length of the input.

NP - Problem – A problem is assigned to the NP (non - deterministic polynomial time) class if it is solvable in polynomial time by a non - deterministic Turing machine.

A P - Problem (whose solution time is bounded by a polynomial) is always also NP. If a problem is known to be NP, and a solution to the problem is somehow known, then demonstrating the correctness of the solution can always be reduced to a single P(polynomial time) verification. If P and NP are not equivalent then the solution of NP - problems requires (in the worst case) an exhaustive search.
A problem is said to be NP - hard if an algorithm for solving it can be translated into one for solving any other NP - problem. It is much easier to show that a problem is NP than to show that it is NP - hard. A problem which is both NP and NP - hard is called an NP - complete problem.

P versus NP problem - The P versus NP problem is the determination of Whether all NP - Problems are actually P - Problems, if P and NP are not equivalent then the solution of NP - Problem requires an exhaustive search, While if they are, then asymptotically faster algorithms may exist.

NP – completeness – Polynomial time reductions provide a formal means for showing that one problem is at least as hard as another, to within polynomial - time factor. That is, if $L_1 \leq_p L_2$, then L1 is not more than a polynomial factor harder than L_2, which is why the “less than or equal to " notation for reduction is mnemonic. We can now define the set of NP - Complete languages, which are the hardest problem in NP.

A language --- that is NP-complete, if it satisfies the following 2 properties -

1. $L \in NP$;and
2. For every $L' \in NP$, $L' \leq_p L$

If a language L satisfies property 2 but not necessary property 1, we say that L is NP - hard. We use the notation L NPC to denote that L is NO - Complete.

NP - complete problem – A problem which is both NP (verifiable in non deterministic polynomial time) and NP - hard (any Np - problem can be translated into this problem) examples of NP - hard problems include the Hamiltonian cycle and travelling sales man problems.

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

Technique for NP - complete problem -

Techniques for dealing with NP-complete problems in practice are -

1. Backtracking
2. Branch and Bound
3. Approximation Algorithms

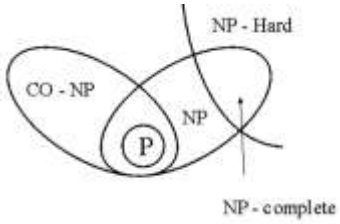
Examples of NP-complete problems :

- Circuit satisfiability
- Formula satisfiability
- 3 - CNF satisfiability
- Clique
- Vertex cover
- Subset - sum
- Hamiltonian cycle
- Traveling salesman
- Sub graph isomorphism
- Integer programming
- Set partition
- Graph coloring

Example - Circuit satisfiability is a good example of problem that we don't know how to solve in polynomial time. In this problem, the input is a Boolean circuit. A collection of and, or and not gates connected by wires. The input to the circuit is a set of 'm' Boolean (true/false) values x1 , x2 , . . . xm. The output is a single Boolean value, the circuit satisfiability problem asks, given a circuit, whether there is an input that makes. The circuit output TRUE, or conversely, whether the circuit always outputs FLASE. Nobody knows how to Solve this problem faster than just trying all 2^m possible inputs to the circuit but this requires exponential time.

- A problem is NP - complete if it is both NP - hard and an element of NP. NP-complete problems are the hardest problems in NP. If anyone finds a polynomial - time algorithm for even one NP - complete problem, then that would imply a polynomial - time algorithm for every NP - complete problem. Literally thousands of problems have been shown to be NP-complete, so a polynomial time algorithm for one (i.e., all) of them seems incredibly unlikely.

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.



- To prove that a problem is NP - hard, we use a reduction argument, exactly like we're trying to prove a lower bound.
- To prove that problem 'A' is NP - hard, reduce a known NP - hard problem to 'A'.
- Saying that a problem is NP - hard is like Saying 'If I own a dog, then it can speak fluent English' you probably don't know whether or not I own a dog, but you are probably pretty sure that I don't own a talking dog. Nobody has a mathematical proof that dogs can't speak English the fact that no one has ever heard a dog speak English is evidence as per the hundreds of examinations of dogs that lacked the proper mouth shape and brain power, but mere evidence is not a proof nevertheless, no sane person would believe me if I said I owned a dog that spoke fluent English. So the statement 'If I own a dog then it can speak fluent English' has a natural corollary: No one in their right mind should believe that I own a dog ! likewise if a problem is NP - hard no one in their right mind should believe it can be solved in polynomial time.

----------◄►----------

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

Questions

Q.1 The complete binary tree at level 'd' contains ------- leaves and ------- non leaf nodes (Root is in level 0)?

(A) 2^d-1, 2^d (B) 2^d, 2^d-1 (C) 2^d+1, 2^d (D) 2^d-1, 2^d+1

Q.2 Convert the following expression from reverse polish notation to infix notation ABCDE+*-/

(A) A/(B-C*D+E) (B) A/(B-C*(D+E)) (C) (A/(B*C)-(D+E)) (D) A/(B-C)*(D+E)

Q.3
```
int rec(int n){     if(n==1)
                        return 1;
                    else
                        return(rec(n-1) + rec(n-1));
```

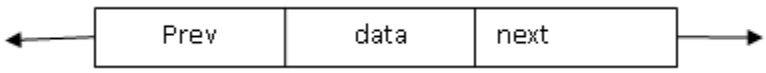
The complexity of above code is

(A) $O(2^n)$ (B) O(n) (C) O(n!) (D) O(log n)

Q.4 What is the prefix of the following infix expression: log x + log y

(A) + log x log y (B) +x log y log (C) x log + y log (D) + log log x y

Q.5 If a DLL is declared as



Then to insert a node in the middle of the list, requires how many changes to various next and prev pointers?

(A) 1 next, 2 prev (B) 2 next, 2 prev (C) 2 next, 1 prev (D) 3 next, 2 prev

Q.6 How many times the following 'c' program would print'i-GATE'

```
main(){
        print("\ni-GATE"); main(); }
```


By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

(A) infinite number of times (B) give compile time error

(C) till the stack does not overflow (D) give run time error

Q.7 What is the post order of BST whose preorder traversal is given as: Preoredr 6,2,1,4,3,5,7

(A) 1,3,5,4,2,7,6 (B) 7,5,3,4,1,2,6 (C) 1,2,3,4,5,6,7 (D) 1,3,2,4,5,6,7

Q.8 The time complexity of linear search algorithm over an array of n element is

(A) $O(\log_2 n)$ (B) O (n) (C) O ($n \log_2 n$) (D) O $\left(n^2\right)$

Q.9 Match the following:Data: The keys are inserted into an AVL tree: 1, 2, 3, 8, 6

List-I	List-II
P: The root node	A: 1
Q: Number of nodes in LST of root	B: 2
R: Number of nodes in RST of root	C: 3
	D: 4

(A) P-A,Q-B,R-C (B) P-B,Q-D,R-C (C) P-B,Q-A,R-C (D) P-C,Q-B,R-A

Q.10 Given the input {71, 23, 73, 99, 44, 79, 89} and a hash function h(x) = x mod 10 and quadratic probing, then in which location the last element is placed:

(A) 2 (B) 6 (C) 7 (D) 8

Q.11 The time taken by binary search algorithm to search a key in a sorted array of n elements is

(A) $O(\log_2 n)$ (B) O (n) (C) O ($n \log_2 n$) (D) O $\left(n^2\right)$

Q.12 The time required to search an element in a linked list of length n is

(A) $O(\log_2 n)$ (B) O (n) (C) O ($n \log_2 n$) (D) O $\left(n^2\right)$

Q.13 How many maximum keys can be there in 3rd level of 4-5 B trees?

(A) 124 (B) 17 (C) 24 (D) 126

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

Q.14 If number of leaf nodes is 2(n-1)+3 where 'n' is a number of internal nodes.Then which of the following is true?

(A) The give tree is binary tree (B) The give tree is 3-ary tree

(C) The give tree is 4-ary tree (D) None of the above

Q.15 In a perfect hash function what is the time required to sort all the items?

(A) O (1) (B) O (n) (C) O (log n) (D) O (n/n)

Q.16 Which one of the following is not a valid recursive procedure ?

(A) Linear recursion (B) Mutual recursion

(C) Binary recursion (D) Polynomial recursion

Q.17 In "Towers of Hanoi", for N=5, After how many invocation, first move take place.

(A) 6 (B) 5 (C) 7 (D) 9

Q.18 Nine nodes labeled 1,2,3,4……9 are used to construct different binary trees. How many such binary trees can be

constructed whose pre order traversal is 1,2,3…….9.

(A) 2^9 (B) (1/10) 18 C_9 (C) 9! (D) 18 C_9

Q.19 Insert the following elements in AVL tree 3, 2, 1, 4, 5, 6, 7 The number of rotation required in AVL tree is

(A) 4 right rotations (B) 2 right rotations, 2 left rotations

(C) 1 Left rotations, 3 Right rotations (D) 4 left rotations

Q.20 Consider a linked list of *n* element. What is the time taken to insert an element *n* after element pointed by some pointer?

(A) O (1) (B) O ($\log_2 n$) (C) O (n) (D) O ($n \log_2 n$)

Q.21 What is the post-order traversal of the binary tree having preorder traversal as 2, 40, 6, 5, 7, 11, 37, 9, 4 and in-order traversal as 40, 6, 2, 11, 7, 37, 5, 4, 9?

(A) 9, 4, 5, 37, 7, 11, 2, 6, 40 (B) 4, 9, 37, 11, 7, 5, 6, 40, 2

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

(C) 6, 40, 11, 37, 7, 4, 9, 5, 2 (D) 2, 5, 9, 4, 7, 37, 11, 40, 6

Q.22 Identify the true statements regarding insertion of node in a linear linked list

I- Setting the field of the new node means allocating memory to newly created node.

II- If the node precedes all others in the list, then insert it at the front and return its address

III- creating a new node depends upon free memory space.

IV- count the number of nodes in the list

(A) I & II (B) I, II & III (C) I & III (D) II & III

Q.23 Stack A has the entries a,b,c(with a on the top). Stack B is empty. An entry popped out of stack A can be printed immediately or push to stack B can only be printed. In this arrangement, which of the following permutation of a, b, c are not possible?

(A) b a c (B) b c a (C) c a b (D) a b c

Q.24 The average successful search time taken by binary search on a sorted array of 10 items is

(A) 2.6 (B) 2.7 (C) 2.8 (D) 2.9

Q.25 The time required to search an element in a BST having n element is

(A) O(1) (B) O(logn) (C) O(n) (D) $O(n^2)$

Q.26 The time required to insert an element in a stack with linked implementation is

(A) O (1) (B) O ($\log_2 n$) (C) O (n) (D) O ($n \log_2 n$)

Q.27 Which of the following option insertion an item x after current pointer:

(A) cur = newnode(x, next) (B) cur.next = newnode(x, cur)

(C) cur.next = newnode(x, cur.next) (D) cur = newnode(x, cur.next)

Q.28 Consider the following recursive method (Ackermann's function):

```
int acker(int m, int n){      int result;
                              if(m==0) result= n+1;
```


By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

```
else if(n==0) result=acker(m-1, 1);
        else result=acker(m-1,acker(m, n-1));
result;}
```

What is acker(1, 2)?

(A) 2 (B) 3 (C) 4 (D) 6

Q.29 Consider the following pseudo code for the traversal of a binary tree

1. Create the stack

2. Push the root node on the stack

3.while stack is NOT empty

Pop a node

If the node is NOT NULL

Print it value

Print the nodes right child on the stack

Print the nodes left child on the stack

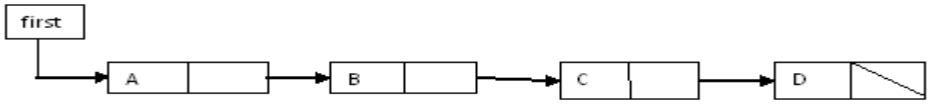
Identify the above iterative code for the traversal of a binary tree

(A) Converse preorder (B) Preorder (C) Inorder (D)PostOrder

Q.30 Calculate m for a m-array tree where leaves L=41 and internal node i=10.

(A) 4 (B) 5 (C) 6 (D) 7

Q.31 Here is the code for this single linked list:




```
void Do(struct node *beforep)
```


By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

{

struct node *p, *afterp;

p = before→next; afterp = p→next; p→next = afterp →next;

before →next = afterp; afterp →next = p; }

What is the output after invoking Do(first)?

(A) d c b a (B) d b c a (C) a c b d (D) b a c d

Q.32 Algorithm Do(n)

if(n is 1)

return 1;

else

return (1/n + Do(n-1));

End

Identify the following series:

(A) 1+2+3+…….N (B) 1+ 1/ 2 + 1/ 3 + ……1/ N

(C) 1- 1/ 2 - 1/ 3 - ……1/ N (D) 1+ 1/ 2 + 2 + 1/ 3 + 3 +……1/ N + N

Q.33 Suppose the 6 weights 4, 15, 25, 5, 8, 16 are given. Find a 2-tree T with the given weights and a minimum weighted path length P.

(A) 172 (B) 177 (C) 180 (D) 231

Q.34 Consider the following algorithm on stack

Algorithm whatamldoing

1. loop(Stack.popStack(data))

Temp1.pushStack(data)

2. loop(Temp1.popStack(data))

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

Temp2.pushStack(data)

3. loop(Temp2.popStack(data))

Stack.pushStack(data)

End.

(A) Stack content remains intact

(B) Reversing stack contents

(C) Pop out stack contents and pushed in alternate elements

(D) Test the stack contents whether it is palindrome

Q.35 A two dimension array data [5][6] is stored in column major order with base address 301.What is the address of data[2][4]?

(A) 323 (B) 326 (C) 321 (D) 423

Q.36 In a compact single dimensional array representation for lower triangular matrices (i.e. all the elements of the lower

triangle) of each row are stored one after another, starting from the first row, the index of the (i, j) th element of the lower

(A) $i + j$ (B) $i + j - 1$ (C) $j + \frac{i(i-1)}{2}$ (D) $i + \frac{j(j-1)}{2}$

Q.37 Consider the following routine to traverse on circular singly linked list with head node

```
void Do(struct node *head)
{ struct node *current = head → next, *nextNext;
  while(current → next != head)
  {  if(current → data == current → next → data)
     {  nextNext = current → next → next;
        free(current → next);   current → next = nextNext;     }
   else
```


By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

current = current → next; } }

What does Do() do on sorted circulary linked lint?

(A) insert duplicates (B)insert duplicate at alternate place

(C) remove duplicates (D) remove alternate duplicats

Q.38 Give the correct matching for the following pairs:

(A) O (log n) (P) Selection

(B) O (n) (Q)Insertion sort

(C) O (n log n) (R)Binary search

(D) O (n^2) (S) Merge sort

(A) A – R B – P C – Q D – S (B)A – R B – P C – S D – Q

(C) A – P B – R C – S D – Q (D)A – P B – S C – R D – Q

Q.39 Match the following:

P : Linear probing	1) Deletion possible
Q : Random probing	2) Secondary clustering
R : Quadratic probing	3) Deletion not possible
S : Separate chaining	4) Overflow
	5) Primary clustering

(A)	(B)	(C)	(D)
P – 5	P – 5	P – 2	P – 5
Q – 2	Q – 2	Q – 2	Q – 2
R – 2	R – 2	R – 5	R – 2
S – 1	S – 3	S – 1	S – 3

Q.40 Considering the following keys to create AVL tree

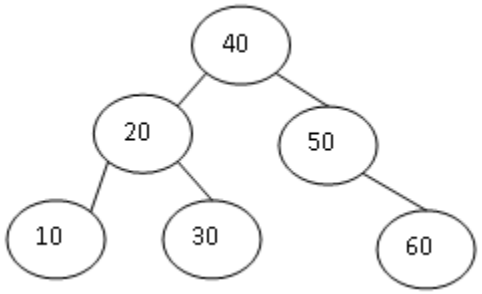
A, B, C, D, E, F

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

P : The number of internal nodes	1) 1
Q : The number of leaves	2) 2
R : The number of nodes having one child	3) 3
S : The number of null leaf	4) 4
	5) 6
	6) 7

(A)	(B)	(C)	(D)
P – 3	P – 3	P – 3	P – 3
Q – 1	Q – 3	Q – 3	Q – 1
R – 3	R – 1	R – 5	R – 3
S – 5	S – 6	S – 6	S – 6

Q.41 What is rank of node 50 in indexed BT

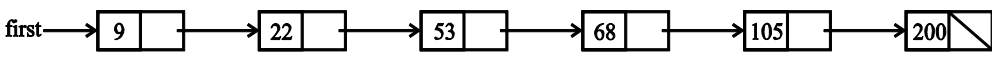


(A) 1 (B) 2 (C) 3 (D) 4

Q.42 Consider the hash table of size seven, with starting index zero, and a has function (3x+4) mod 7.Assuming the hash table is initially empty, which of the following is the contents of the table when the sequence 1,3,8,10 is inserted into the table using closed hashing? Note that- denote an empty location in the table

(A) 8, -, -, -, -, -, 10 (B) 1, 8, 10, -, -, -, 3 (C) 1, -, -, -, -, -, 3 (D) 1, 10, 8, -, -, -, 3

Q.43 What would be the output of the following after executing the sequence of operations?



By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

struct node *p (i) $p = first \rightarrow link$ (ii) $first \rightarrow link = p \rightarrow link \rightarrow link \rightarrow link$

(iii) $first \rightarrow link \rightarrow link \rightarrow link = p$

(iv) $print(p \rightarrow link \rightarrow link \rightarrow link \rightarrow link \rightarrow link \rightarrow data)$

(A) 9 (B) 22 (C) 53 (D) 68

Q.44 Find how many collision occurs when following keys 10, 100, 32, 45, 58, 126, 3, 29, 200, 400, 0 are reduced to hash table size=13.

(A) 4 (B) 5 (C) 6 (D) 7

Q.45 Keys 9, 19, 29, 39, 49, 59, 69 are inserted into the side table hash function H=K mode 10 and quadratic probing is used for collision revolution. What is the index into 59 will be inserted?

(A) 3 (B) 4 (C) 8 (D) 5

Q.46 Determine True(T) / False(F) of the following equalities:

i) n!=O(n log n) ii) $3^n = Ø(2^n)$

iii) $n^a = Ø(n^b)$ a, b constant and $a \neq b$ iv) $n^2+3n+\sin n = O(n3)$

(A) i-F ii-F iii-F iv-T (B) i-T ii-F iii-F iv-T (C) i-T ii-T iii-F iv-T
(D) i-F ii-T iii-T iv-T

Q.47 Consider an undirected graph G with n vertices and e edges. What is the time taken by Depth First Search (DFS) if the graph is represented by (i) Adjacency matrix and (ii) Adjacency list?

(A) $O(n^2)$, O (n) (B) $O(n^2)$, O (e) (C) O (e), $O(n^{1.2})$ (D) O ($e+n$), O (e)

Q.48 Create Binary search tree for the keys : 30, 40, 5, 80, 2, 35, 60, 32, 85, 31 and 33s

What is the height of the BST (root in at level 1)

(A) 3 (B) 4 (C) 5 (D) 6

Q.49 How many values can be held by an array A (-1..m , 1..m)?

(A) m (B) m^2 (C) $m(m+1)$ (D) $m(m+2)$

Q.50 Which of the following sorting procedure is the slowest?

(A) Quick sort (B) Heap sort (C) Shell sort (D) Bubble sort

Q.51 The worst case time complexity of binary insertion sort algorithm to sort n elements is

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

(A) O (n) (B) O n ($\log_2 n$) (C) O ($n^{1.2}$) (D) O (n^2)

Q.52 Faster access to non – local variable is achieved using an array of pointers to activation records called a

(A) stack (B) heap (C) display (D) activation tree

Q.53 A full binary tree with n-non-leaf nodes contains

(A) $\log_2 n$ nodes (B) $n + 1$ nodes (C) $2n - 1$ (D) $2n + 1$ nodes

Q.54 A sort which uses the binary tree concept such that any number is larger than all the numbers in the subtree below it is called

(A) selection sort (B) insertion sort (C) heap sort (D) quick sort

Q.55 In a binary max heap containing n numbers the smallest element can be found in time

(A) Ø (n log n) (B) Ø (n) (C) Ø (log n) (D) Ø (1)

Q.56 A complete n-ary tree is a tree in which each node has n children or no children. Let I be the number of internal nodes and L be the number of leaves in a complete n-ary tree. If L=41, and I=10,What is the value of n

(A) 3 (B) 4 (C) 5 (D) 6

Q.57 The average time required to perform a successful sequential search for an element in an array A (1 : n) is given by

(A) $(n + 1)/2$ (B) $\log_2 n$ (C) $n(n - 1)/2$ (D) n^2

Q.58 an algorithm is made up of 2 modules M_1 M_2. If order of M_1 is $f(n)$ and M_2 is $g(n)$ then the order of the algorithm is

(A) max ($f(n)$, $g(n)$) (B) min ($f(n)$, $g(n)$) (C) $f(n) + g(n)$ (D) $f(n) * g(n)$

Q.59 Consider the hash table of size seven, with starting index zero, and a has function (3x+4) mod 7. Assuming the hash table is initially empty, which of the following is the contents of the table when the sequence 1,3,8,10 is inserted into the table using closed hashing? Note that- denote an empty location in the table

(A) 8, -, -, -, -, -, 10 (B) 1, 8, 10, -, -, -, 3 (C) 1, -, -, -, -, -, 3 (D) 1, 10, 8, -, -, -, 3

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

Q.60 int xyz(struct bstnode *p)

```
{ if(p==0) return0;
  if(p->right!=0)
   return(xyz(p->left)+xyz(p->rigth)+1);
  else
   return(xyz(p->left)); }
```

What does the above function do on Binary Search Tree?

(A) Count the number of leaves (B) Count the number of right children

(B) Count the number of nodes (D) Count the number of left children

Q.61 The linked list implementation of sparse matrices is superior to the generalized dope vector method because, it is

(A) conceptually easier and completely dynamic (B) efficient, if the sparse matrix is a band matrix

(C) efficient in accessing an entry (D) all of these

Q.62 The number of vertices of odd degree in a graph is

(A) always even (B) always odd (C) regular graph (D) always zero

Q.63 A graph in which all nodes are of equal degree is known as

(A) multigraph (B) non regular graph (C) regular graph (D) complete graph

Q.64 A circuit is a connected graph which includes every vertex of the graph is known as

(A) Euler (B) Unicursal (C) Hamiltonian (D) Clique

Q.65 If B is a circuit matrix of a graph with k components, e edges and n vertices, the rank of B is

(A) $n - k$ (B) $e - n - k$ (C) $e - n + k$ (D) $e + n - k$

Q.66 If $T_1 = O(1)$, give the correct matching for the following pairs:

(M) $T_n = T_{n-1} + n$ (U) $T_n = O(n)$

(N) $T_n = T_{\frac{n}{2}} + n$ (V) $T_n = O(n \log n)$

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

(O) (P) $T_n = T_{n-1} + \log n$ (W) $T_n = O(n^2)$

(A) M – W N – V O – U (B) M – W N – U O – V

(C) M – V N – W O – U (D) M – W N – U O – V

Q.67 If one uses straight two-way merge sort algorithm to sort the following elements in ascending order:

20, 47, 15, 8, 9, 4, 40, 30, 12 17

then the order of these elements after second pass of the algorithm is:

(A) 8, 9, 15, 20, 47, 4, 12, 17, 30, 40 (B) 8, 15, 20, 47, 4, 9, 30, 40, 12, 17

(C) 15, 20, 47, 4, 8, 9, 12, 30, 40, 17 (D) 4, 8, 9, 15, 20, 47, 12, 17, 30, 40

Q.68 A sorting technique is called stable if

(A) it takes O (nlog n) time

(B) it maintains the relative order of occurrence of non-distinct elements

(C) it uses divide and conquer paradigm (D) it takes O(n) space

Q.69 Using Huffman coding Algorithm, consider 8 items with frequency. What will be the code value for item G.

metI	A	B	C	D	E	F	G	H
ycneuqerF	5	1	1	7	8	2	3	6

(A) 001 (B) 101 (C) 0001 (D) 00001

Q.70 The best searching algorithm to search an element if the list of data is not sorted, is

(A) First sort the data then do a binary search using an array (B) Linear search

(C) binary search using array (D) Binary search using tree

Q.71 Which of the following are true for a binary max-heap with n node?

i) The height is O(logn)

ii) The smallest key must be in the leftmost position of its level.

iii) All leaves appear only at the bottom level

By Siddharth S. Shukla (BE, ME, PhD*)

i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352

No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

What is the prefix of the following infix expression: log x + log y

(A) i (B) i & ii (C) ii (D) i & iii

Q.72 What is the maximum height of a 2-3 tree with n nodes?

(A) log n (B) n (C) $\log_3 n$ (D) n/2

Q.73 Match the following

A. Strassen's matrix multiplication P. Logic programming

B. Kruskal minimum spanning tree Q. Dynamic programming

C. Bi connected component R.Divide and Conquer

D. Floyd's shortest path algorithm S. Depth first search

(A) A-R,B-P,C-S,D-Q (B) A-R,B-P,C-Q,D-S

(C) A-R,B-S,C-Q,D-P (D) A-S,B-P,C-Q,D-R

Q.74 Supply the missing factor in the following recursive definition of the number of nodes on height H of a binary tree.

N(H)= 1 H=1

N(H-1)+? H>=2

(A) N(H-2)+1 (B) 2^{H-1} (C) 2 (D) N(H-1)+1

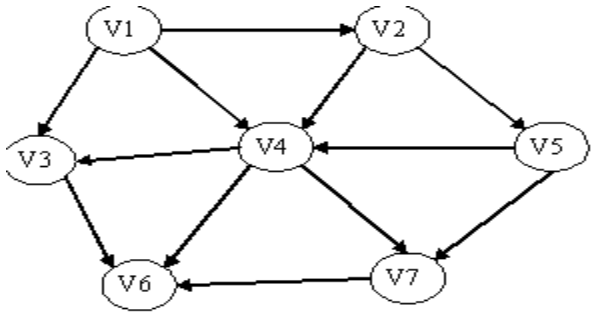
Q.75 If f(n) = O(log n) + O(n) + O(n log n), then f(n) is

i) O(log n) ii) O(n) iii) O(n log n) iv) $O(n^2)$

(A) iii (B) iii & iv (C) i and ii (D) None

Q.76 What is the topological sort order of the given graph?

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.



(A) V1, V2, V5, V4, V7, V3, V6 (B) V1, V2, V4, V5, V3, V7, V6

(C) V1, V2, V4, V5, V3, V7, V6 (D) NoNE

Q.77 The depth of a complete binary tree is given by

(A) $D_n = n \log_2 n$ (B) $D_n = n \log_2 n+1$ (C) $D_n = \log_2 n$ (D) $D_n = \log_2 n+1$

Q.78 How may maximum number of multiplication require to calculating the following product of matrices?

A[m * n] * B[n * n] * C[n * n] * D[n * 1]

(A) $n^2 + m\ n$ (B) $2n^2 + m\ n$ (C) $2m\ n^2 + m\ n$ (D) None

Q.79 $T(n) = 2T(\sqrt{n}) + \log n$ has T(n) equal to

(A) O (log n. log log n) (B) O(log log n) (C) O (n log n) (D) O(log log n)

Q.80 Consider the following instance of Knapsack problem:

N=7, m=15, $(P_1, P_2, P_3, P_4, P_5, P_6, P_7) = (10, 5, 15, 7, 6, 18, 3)$ and $(W_1, W_2, W_3, W_4, W_5, W_6, W7) = (2, 3, 5, 7, 1, 4, 1)$. Get the optimal solutions to fill the knapsack exactly using greedy about profit and weight.

(A) 55 (B) 55.33 (C) 55.66 (D) 60

Q.81 Determine True(T) / False(F) of the following equalities:

i) $n!=O(n \log n)$ ii) $3^n = \emptyset(2^n)$

iii) $n^a = \emptyset(n^b)$ a, b constant and $a \neq b$ iv) $n^2+3n+\sin n = O(n3)$

(A) i-F ii-F iii-F iv-T (B) i-T ii-F iii-F iv-T

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

(C) i-T ii-T iii-F iv-T (D) i-F ii-T iii-T iv-T

Q.82 If one uses straight two-way merge sort algorithm to sort the following elements in ascending order: 20, 47, 15, 8, 9, 4, 40, 30, 12 17 then the order of these elements after second pass of the algorithm is:

(A) 8, 9, 15, 20, 47, 4, 12, 17, 30, 40 (B) 8, 15, 20, 47, 4, 9, 30, 40, 12, 17

(C) 15, 20, 47, 4, 8, 9, 12, 30, 40, 17 (D) 4, 8, 9, 15, 20, 47, 12, 17, 30, 40

Q.83 If $T_1 = O(1)$, give the correct matching for the following pairs:

(M) $T_n = T_{n-1} + n$ (U) $T_n = O(n)$

(N) $T_n = T_{\frac{n}{2}} + n$ (V) $T_n = O(n \log n)$

(O) (P) $T_n = T_{n-1} + \log n$ (W) $T_n = O(n^2)$

(A) M – W N – V O – U (B) M – W N – U O – V

(C) M – V N – W O – U (D) M – W N – U O – V

Q.84 In a heap with n elements with the smallest element at the root, the 7th smallest element can be found in time

(A) Ø (n log n) (B) Ø (n) (C) Ø (log n) (D) Ø (1)

Q.85 In a binary max heap containing n numbers the smallest element can be found in time

(A) Ø (n log n) (B) Ø (n) (C) Ø (log n) (D) Ø (1)

Q.86 How many undirected graphs (not necessarily connected) can be constructed out of a given set $V = \{v_1, v_2, \ldots\ldots v_n\}$ of n vertices?

(A) $\frac{n(n-1)}{2}$ (B) 2^n (C) n! (D) $2^{(n(n-1))/2}$

Q.87 Let w be the minimum weight among all edge weights in an undirected connected graph. Let *e* be a specific edge of weight *w*. Which of the following is FALSE?

(A) There is an minimum spanning tree containing *e*.

(B) If *e* is not in a minimum spanning tree T, then in the cycle formed by adding *e* to T, all edges have the same weight.

(C) Every minimum spanning tree has an edge of weight *w*.

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

(D) e is present in every minimum spanning tree.

Q.88 Let G(V,E) be an undirected graph with positive edge weights. Dijkstra's single source shortest path algorithm can be implemented using the binary heap data structure with time complexity:

(A) $O(|V|^2)$ (B) $O(|E|+|V|\log|V|)$ (C) $O(|V|\log|V|)$ (D) $O((|E|+|V|)\log|V|)$

Q.89 Consider the polynomial $p(x)= a_0+a_1x+a_2x^2+a_3x^3$, where $a_i \neq 0, \forall i$. The minimum number of multiplications needed to evaluate p on an input x is.

(A) 3 (B) 4 (C) 9 (D)9

Q.90 The asymptotic behavior of polynomial in 'n' of the form

$f(n)=\sum_{i=0}^{m} a_i n^i$ where $a_m > 0$ then f(n) is

(A) O(log m) (B) $O(n^m)$ (C) O(n log m) (D) None

Q.91 Quick sort is run on two inputs shown below to sort in ascending order

i- 1, 2, 3, ---------- n ii- n, n-1, ---------- 2, 1

Let C_1 and C_2 be the number of comparisons made for the input (i) and (ii) respectively. Then,

(A) $C_1 < C_2$ (B) $C_1 > C_2$ (C) $C_1 = C_2$ (D) can't say

Q.92 Calculate m for a m-array tree where leaves L=41 and internal node i=10.

(A) 4 (B) 5 (C) 6 (D) 7

Q.93 An algorithm that calls itself directly or indirectly is known as

(A) Sub algorithm (B) Recursion (C) Polish notation (D) Traversal algorithm

Q.94 A connected graph T without any cycles is called

(A) a tree graph (B) free tree (C) a tree (D) All

Q.95 Suppose the 6 weights 4, 15, 25, 5, 8, 16 are given. Find a 2-tree T with the given weights and a minimum weighted path length P.

(A) 172 (B) 177 (C) 180 (D) 231

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

Q.96 What is the smallest and largest number of entries for 2_3 Btree (B2_3 Tree) of height 8 (i.e., 8 levels)?

(A) 255 & 6560 (B) 127 & 2186 (C) 6561 & 127 (D) 255 & 2186

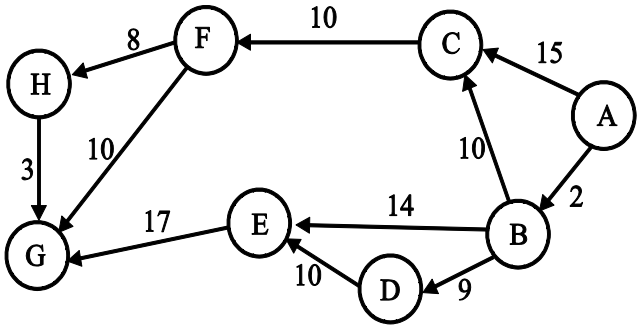
Q.97 Match the following recurrence relations with their algorithms.

Recurrence	Algorithm
A. $T(n) = T(n/2) + O(1)$	p. Tree traversal
B. $T(n) = T(n-1) + O(1)$	q. Merge sort
C. $T(n) = 2T(n/2) + O(1)$	r. Selection sort
D. $T(n) = T(n-1) + O(n)$	s. Sequential search
E. $T(n) = 2T(n/2) + O(n)$	t. Binary search

(A) A-t, B-s, C-q, D-r, E-p (B) A-p B-q C-t D-r E-s

(C) A-r B-s C-t D-q E-p (D) A-t B-s C-p D-r E-q

Q.98 If we run Dijkstra's single source shortest path algorithm on the following edge-weighted directed graph with vertex 5 as the source



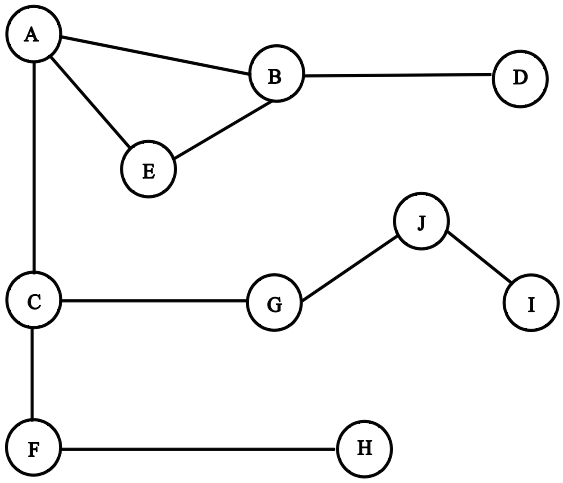
In what order do the nodes get included into the set of vertices for which the shortest path distances are finalized.

(A) A, B, D, C, E, H, G, F (B) A, B, D, C, E, F, H, G

(C) A, B, D, C, E, F, G, H (D) A, B, D, C, E, G, F, H

Q.99 Consider the following graph

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.



If you start from vertex A, then in what order will you traverse the graph using a depth first search?

(i) A B D E C F H G J I (ii) A C F H G J I E B D

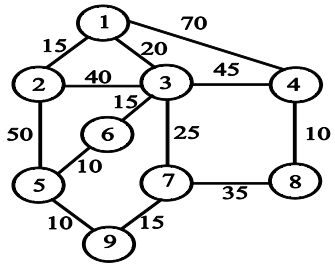
(iii) A E B D C G J I F H (iv) A B D E C G J I F H

(v) A C G J I F H E D B

(A) Only (i), (ii) and (iii) (B) Only (i), (ii), (iii) and (iv)

(C) Only (ii), (iii), (iv) and (v) (D) All of the above

Q.100 Consider the following graph :



The difference of cost between minimum cost spanning tree (using kruskal algorithm) and cost of shortest spanning tree from vertex 1 will be

(A) 15 units (B) 20 units (C) 25 units (D) 30 units

Q.101 The running time $T(n)$, where (n) is the input size of a recursive algorithm is given as follows

$T(n) = C + T(n-1)$; if $n > 1$

d, if $n \le 1$

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

the order of algorithm is

(A) n^2 (B) n (C) n^3 (D) n

Q.102 Consider the Huffman codes that are used to compress the data. Six messages are there and the their frequency is given in the table.

egasseM	M1	M2	M3	M4
ycneuqrF	0.53	0.27	0.11	0.09

What will be the sequence of message for binary sequence?

10100000101001000011

(A) $M_1\ M_2\ M_4\ M_1\ M_2\ M_3\ M_4\ M_1\ M_1$ (B) $M_1\ M_2\ M_4\ M_2\ M_2\ M_3\ M_1\ M_4\ M_1$

(C) $M_1\ M_2\ M_4\ M_3\ M_2\ M_3\ M_1\ M_4\ M_1$ (D) $M_1\ M_2\ M_4\ M_3\ M_2\ M_3\ M_4\ M_2\ M_1$

Q.103 Average successful search time for sequential search on 'n' items is

(A) n/2 (B) (n-1)/2 (C) (n+1)/2 (D) None

Q.104 A machine needs a minimum of 100 sec to sort 1000 names by quick sort. The minimum time needed to sort 1000 names by quick sort. The minimum time needed to sort 100 names will be approximately

(A) 50.2 sec (B) 6.7 sec (C) 72.7sec (D) 11.2 sec

Q.105 A machine took 200sec to sort 200 names, using bubble sort, In 800 sec, it can approximately sort

(A) 400 names (B) 800 names (C) 750 names (D) 1800 names

Q.106 Match the following columns :

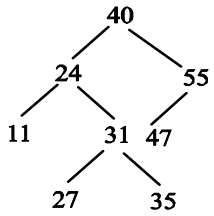
Column A	Column B
1. Max – Heapify	A. O (n)
2. Build max – heap	B. O (log n)

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

3. Heap sort C. O (n log n)

(A) 1-A, 2-B, 3-C (B) 3-A, 1-B, 2-C (C) 2-A, 1-B, 3-C (D) 3-A, 2-B, 1-C

Q.107 Consider the following T.



Find the rank of node 55?

(A) 0 (B) 1 (C) 2 (D) 3

Q.108 Given n d-digit numbers in which each digit can take up to k possible values, RADIX-SORT correctly sorts these numbers in

(A) O (d (n + k)) (B) O (n + k) (C) O (dn + k) (D) O (n + dk)

Q.109 A postfix expression is merely the reverse of the prefix expression

(A) true (B) false (C) cant say (D) sometimes

Q110 In a stack , the data item placed on the stack first is.

(A) the first data item to be removed (B) the last data item to be removed

(C) given as index zero (D) not given as index number.

Q.111 which of the following data structure may give overflow error, even though the current number of element in it is less than its size

(A) Simple queue (B) circular queue (C) stack (D) none of these

Q.112 what can be said about the array representation of a circular queue when it contains only one element?

(A) front=rear=NULL (B) front=rear+1 (C) front=rear-1 (D) front=rear!=NULL

Q.113 A linear list in which elements can be added or removed at either end but not in the middle is

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

known as

(A) queue (B) deque (C) stack (D) tree

Q.114 Queues serve a major role in

(A) Simulation of recursion (B) Simulation of arbitrary linked list

(C) Simulation of limited resource allocation (D)Expression evaluation

Q.115 In queue an element can be inserted and deleted by using

(A) Rear and front (B) front and rear (C) Top (D) none

Q.116 A list with no nodes is called

(A) Error list (B) empty list (C) unique list (D) none of the above

Q.117 Linked list are not suited for

(A) Insertion sort (B) Binary search (C) Radix sort (D) Polynomial function

Q.118 A list can be initialized to the empty list by which operation

(A) List = 1; (B) List =0; (C) List = NULL (D) None of the above

Q.119 Stack cannot be used to

(A) Evaluate an arithmetic expression in postfix form

(B) Implement recursion

(C) Convert a given arithmetic expression in infix form to its equivalent postfix form

(D) Allocation resource (like CPU) by the OS

Q.120 Stack is useful for implementing

(A) Radix sort (B) breadth first search (C) recursion (D) none of these

Q.121 Stack is not used in

(A) Complier (B) System programming (C) operating system (D) process scheduling

Q.122 Queue can be used to implement

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

(A) Radix sort (B) quick sort (C) recursion (D) depth first search

Q.123 which of the following is useful in implementing quick sort?

(A) Stack (B) set (C) list (D) queue

Q.124 which of the following is useful in traversing a given graph by breadth first search?

(A) Stack (B) set (C) list (D) queue

Q.125 The result of illegal attempt to remove an element from an empty queue is called

(A) Over flow (B) underflow (C) array based exception (D) none of the above

Q.126 Insert operation on queue has to be done after testing

(A) Overflow (B) memory space (C) underflow (D) no need to test anything

Q.127 Linked list can be used to implement

(A) Stack (B) queue (C) both (a) and (b) (D) neither (a) or nor (b)

Q.128 In linked list data is stored in

(A) Adjacent location (B) different location in memory

(C) neither (A) nor (B) (D) none of the above

Q.129 ----------is called self-referential structure.

(A) Linked list (B) stack (C) queue (D) graph

Q.130 r = malloc(sizeof (struct node))

In this expression what should be written before malloc for appropriate type casting

(A) (int*) (B) (char*) (C) (struct node*) (D) (node*)

Q.131 Reverse polish notation is often called

(A) Postfix (B) prefix (C) Infix (D) none of the above

Q.132 The time complexity for evaluating a postfix expression is

(A) O(n) (B) O(n logn) (C) O(logn) (D) O(n2)

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

Q.133 The correct push function is

(A) S -> arr[S->top + +] = data (B) S -> arr[+ +S->top] = data

(C) S -> arr[+ +(S->top)] = data (D) none of the above

Q.134 The correct pop function is

(A) S -> arr[(S->top)- -]; (B) S -> arr[- -(S->top)];

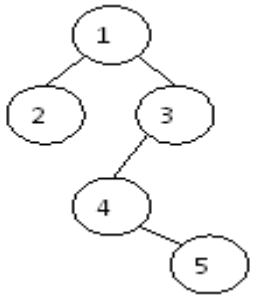
(C) S -> arr[(S->top)]; (D) none of the above

Q.135 In an empty queue rear and front can be initialized as

(A) rear = front = -1 (B) rear = front = 1

(C) rear = 0 , front = -1 (D) rear = -1 , front = 0

Q.136 In the given binary tree, using array (the array start from index 1), you can store the node 4 at location,



(A) 6th (B) 4th (C)5th (D) 3rd

Q.137 Which of the following will have efficient space and time complexities?

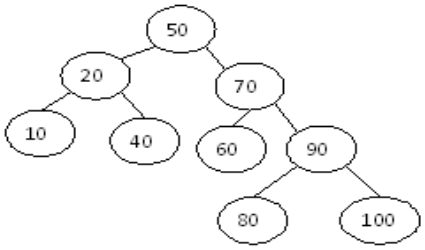
(A) Incomplete binary tree (B) Complete binary (C) Full binary tree (D) None of these

Q.138 In a binary search tree if the number of nodes of a tree is 16 then the minimum height of the tree is,

(A) 16 (B) 4 (C) 3 (D) None of the above

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

Q.139 If we delete 50 from the given binary search tree the root of new binary tree will be,



(A) 40 (B) 60 (C) Both (a) and (b) is possible (D) neither (a) nor (b) is possible

Q.140 The worst-case height of AVL tree with n nodes is

(A) 2 lon n (B) n log (n+1) (C) 1.44 log (n+2) (D) 1.44 n logn

Q.141 The worst-case complexity for searching an element in a binary search tree is

(A) O(n) (B) O(log N) (C) O(N log N) (D) O(N^2)

Q.142 Witch one is true about binary search tree?

(A) Key of every node of right sub-tree of tree is greater than the root.

(B) Key of every node in the left sub-tree of tree is less than the key at the root.

(C) Both of them are true.

(D) None of the above

Q.143 Find the average number of comparisons in a binary search on a sorted array of 10 consecutive integers starting 1.

(A) 2.6 (B) 2.7 (C) 2.8 (D) 2.9

Q.144 Heap can be represented as a

(A) a binary tree (B) an array (C) 2-3 tree (D) None of the above

Q.145 What is the complexity of heapify procedure

(A) Φ (log N) (B) Φ (N) (C) Φ (N log N) (D) Φ (N^2)

Q.146 Which of the following sorting algorithm has average sorting behavior

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

(A) Binary heap (B) Merge Sort (C) Heap Sort (D) Exchange Sort

Q.147 The implementation of priority queue is

(A) Binary heap (B) Hashing (C) Deque (D) None of the above

Q.148 A heap is a

(A) Complete BT (B) Strictly BT (C) Full BT (D) B-tree

Q.149 If there are n elements in heap, what is the time complexity of inserting a new element in the heap in worst case?

(A) Φ (log N) (B) Φ (N) (C) Φ (N log N) (D) Φ (N^2)

Q.150 The right child of the node in position 10 will in the position

(A) 9 (B) 11 (C) 21 (D) 20

Q.151 The parent of a node in heap of position 10 will be in position

(A) 5 (B) 6 (C) 20 (D) 21

Q.152 Avg. Number of comparisons in searching an element in a binary searching tree is

(A) O (n) (B) O (log n) (C) O (n^3) (D) O (n log n)

Q.153 In a complete binary tree of n nodes, the maximum distance between two nodes is, (assume search in the path, cost 1)

(A) $\log_2 n$ (B) $2 \log_2 n$ (C) $3 \log_2 n$ (D) $4 \log_2 n$

Q.154 A complete binary tree of depth 5 has,

(A) 15 nodes (B) 25 nodes (C) 63 nodes (D) 33 nodes

Q.155 In tree construction, which one will be suitable and efficient data structure?

(A) Array (B) Linked list (C) Stack (D) Queue

Q.156 How many-ordered tree possible with 6 nodes?

(A) 10 (B) 12 (C) 13 (D) None of these

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

Q.157 If binary tree T has n leaf nodes, the number of nodes of degree 2 in T is,

(A) $\log_2 n$ (B) n-1 (C) n (D) 2^n

Q.158 Average case complexity if search, insertion and deletion operation is

(A) $O(\log_2 n)$ (B) $O(n \log_2 n)$ (C) O(n) (D) $O(n^n)$

Q.159 How many nodes a complete binary tree of depth 4 has?

(A) 29 (B) 31 (C) 30 (D) 32

Q.160 A full binary tree with n non leaf nodes contain,

(A) $\log_2 n$ nodes (B) n+1 nodes (C) 2n node (D) 2n+1 node

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

1-B	2-B	3-A	4-A	5-B	6-C
7-A	8-B	9-C	10-D	11-A	12-B
13-A	14-B	15-A	16-D	17-A	18-A
19-C	20-A	21-C	22-A	23-C	24-D
25-B	26-A	27-C	28-B	29-A	30-B
31-C	32-B	33-A	34-B	35-A	36-C
37-C	38-B	39-A	40-B	41-A	42-B
43-B	44-C	45-C	46-B	47-B	48-C
49. D	50. D	51. D	52. C	53. D	54. C
55. B	56. C	57.A	58.A	59. B	60. B
61. D	62. A	63. C	64. C	65. C	66. B
67. B	68. B	69.A	70.D	71.A	72.A
73.A	74.B	75.B	76.A	77.D	78.C
79.A	80.B	81.B	82.B	83.B	84.C
85.B	86.D	87.D	88.B	89.A	90.B
91.C	92.B	93.B	94.D	95.A	96.A
97.D	98.B	99.B	100.B	101.B	102.D
103.C	104.B	105.A	106.C	107.C	108.A
109.B	110.B	111.B	112.B	113.B	114.C
115.A	116.B	117.C	118.C	119.D	120.C
121.D	122.A	123.A	124.D	125.B	126.A

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.

127.C	128.B	129.A	130.C	131.A	132.A
133.C	134.A	135.A	136.A	137.C	138.B
139.C	140.C	141.A	142.C	143.D	144.B
145.A	146.B	147.A	148.A	149.A	150.C
151.A	152.B	153.B	154.C	155.B	156.D
157.B	158.A	159.B	160.D		

By Siddharth S. Shukla (BE, ME, PhD*)
i-GATE , B-713, Street 22, Smriti Nagar, Bhilai- 490020, Contact Mobile 98271-62352
No part of this booklet may be reproduced or utilized in any form without the written permission. All right are reserved.